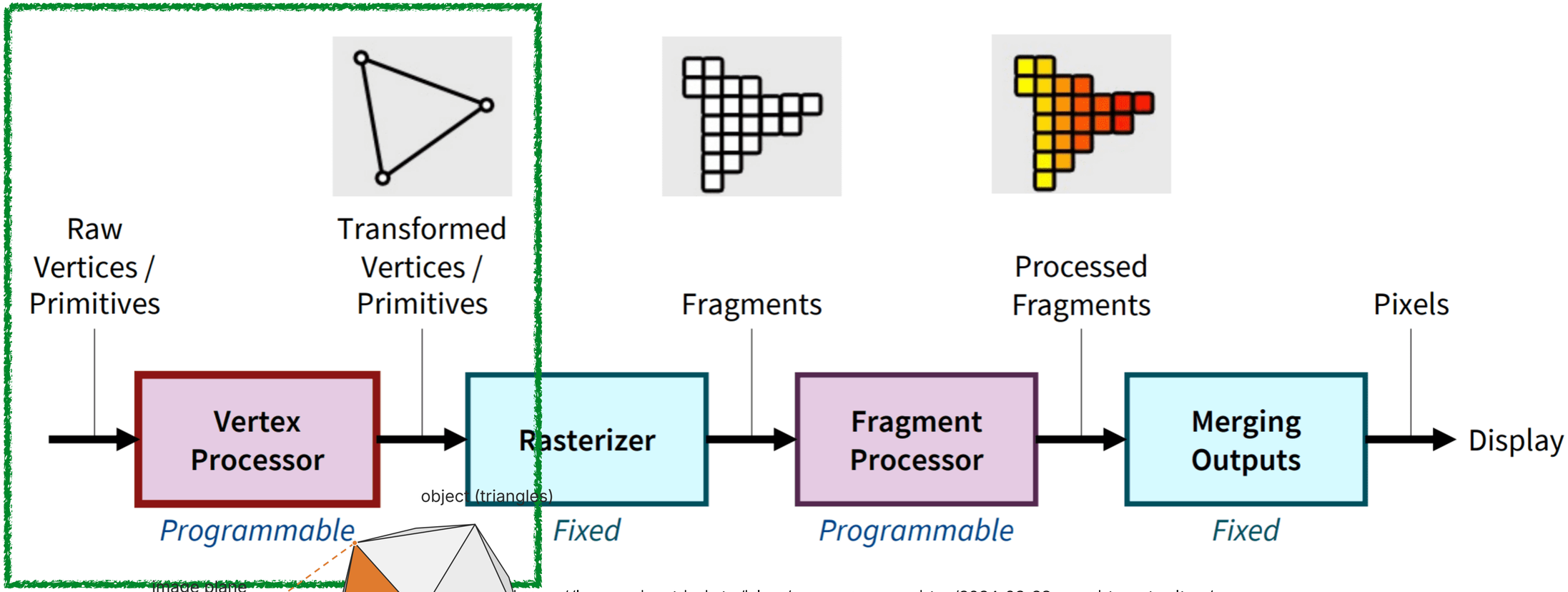
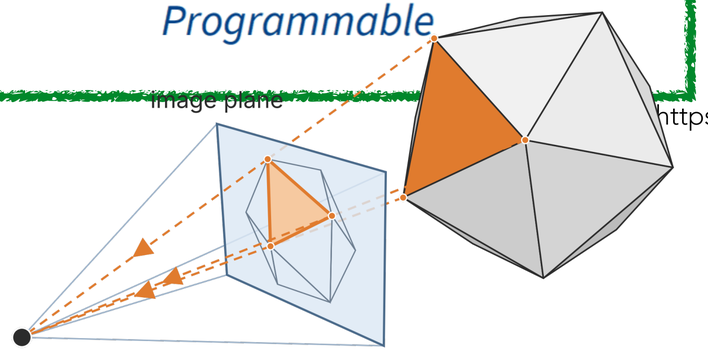
Viewing Transformations

University of Victoria | Computer Science

Ray tracing → Rasterization

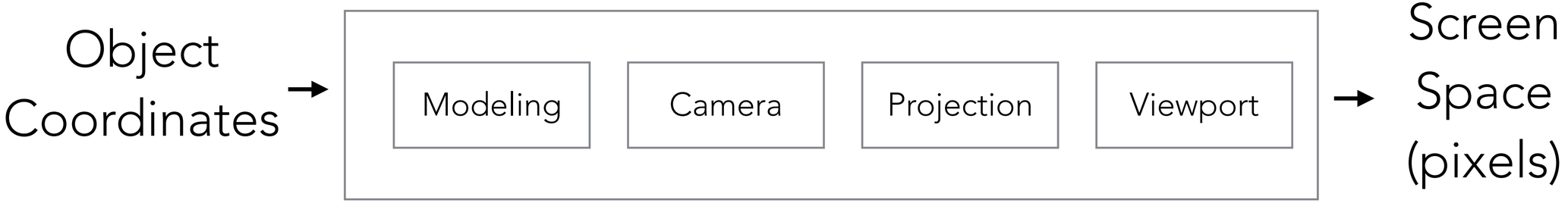


https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

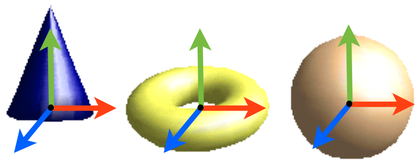


University of Victoria | Computer Science

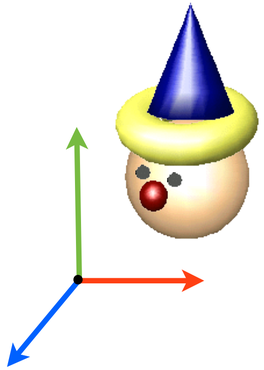
Viewing transformations



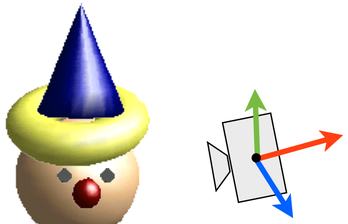
Coordinate Systems



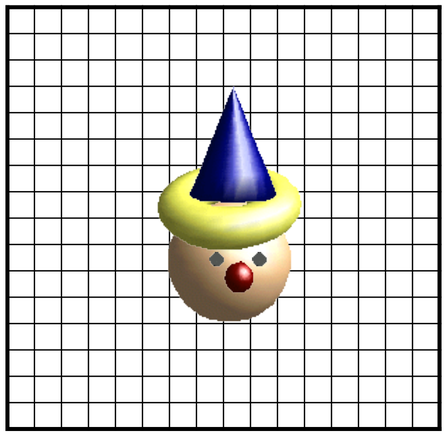
object coordinates



world coordinates

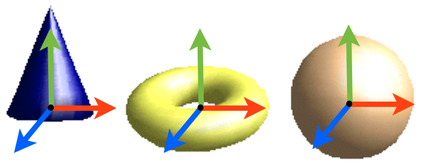


camera coordinates

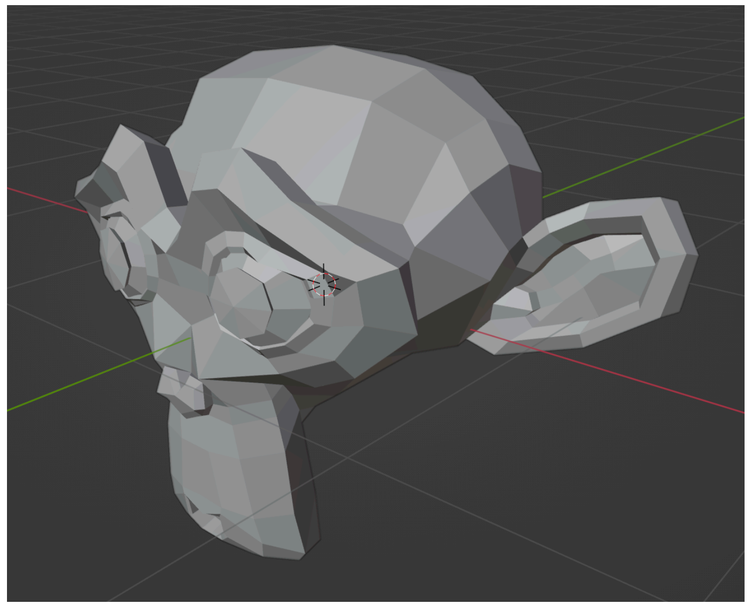


screen coordinates

Object Coordinates

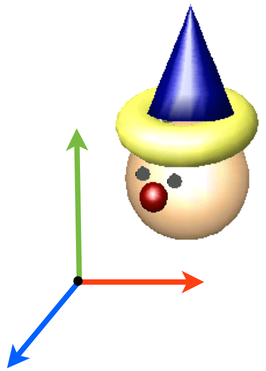


object coordinates

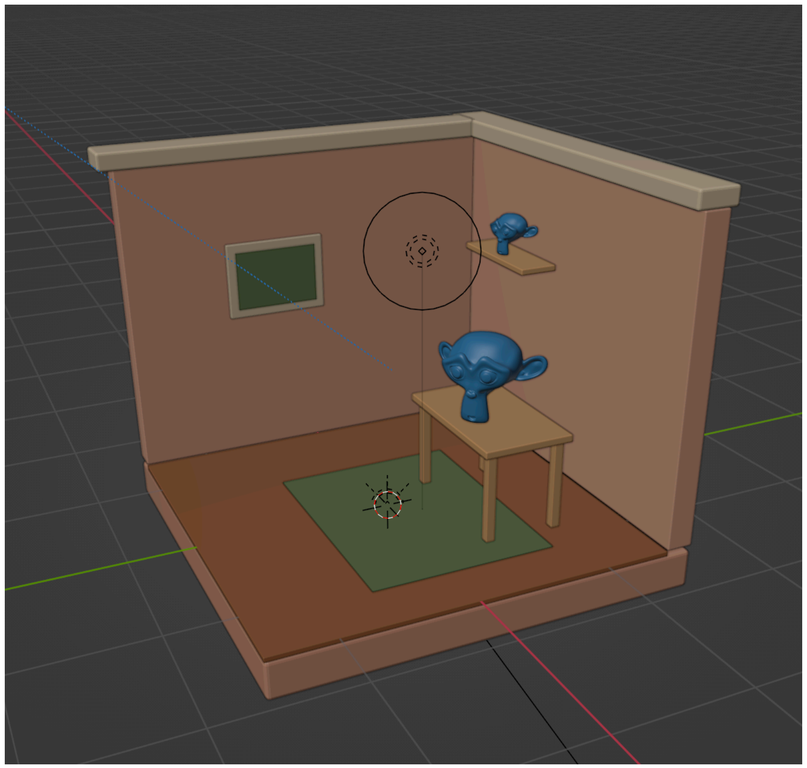


- Easier to model
  - E.g., rotation center can be set to the object center
  - Reuse (instancing)

World Coordinates



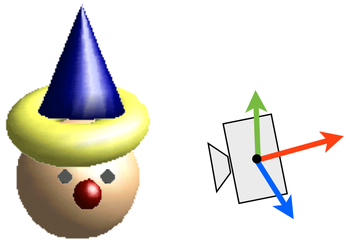
world coordinates



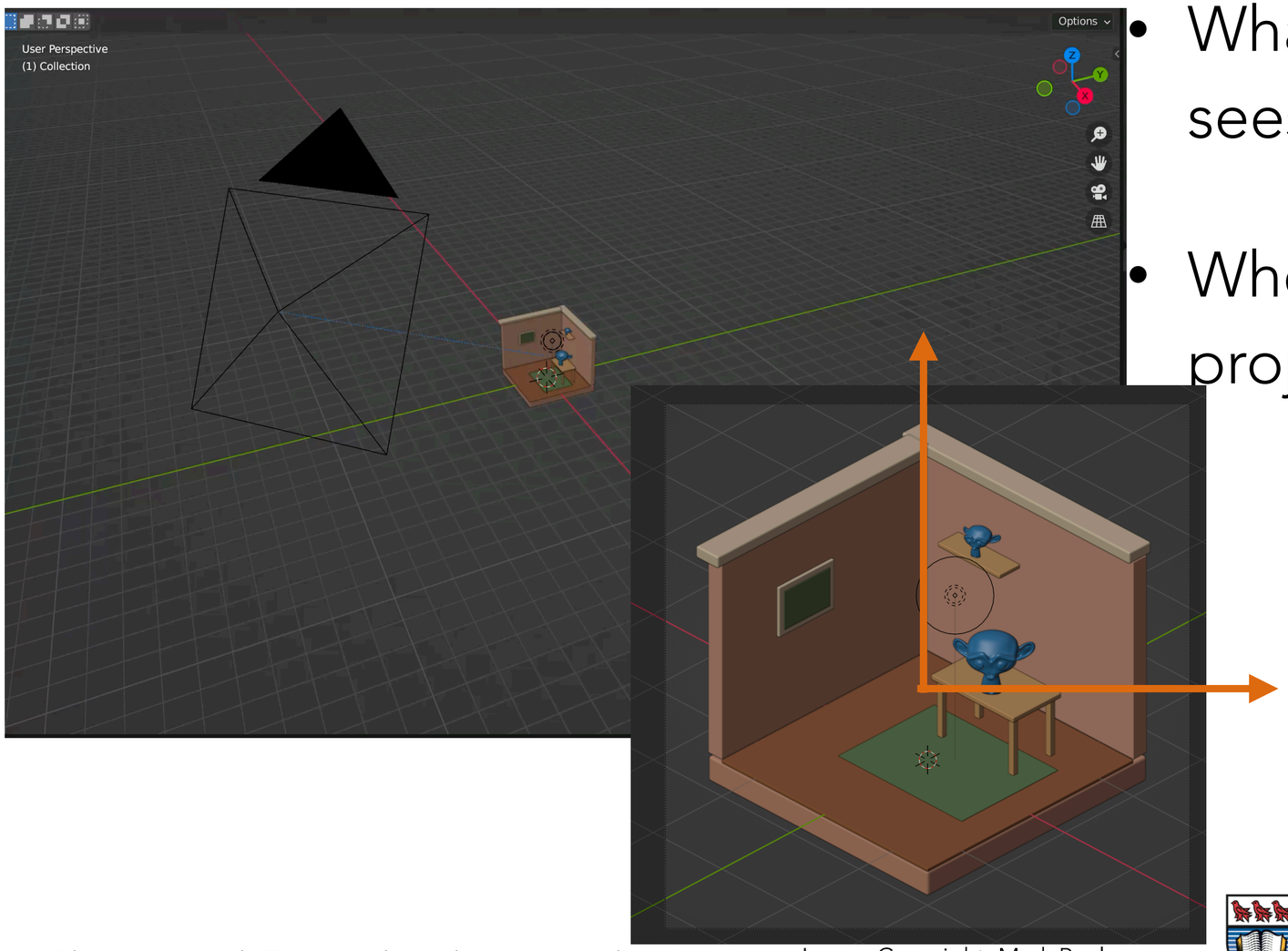
- Actual scene to assemble objects
- Also where the camera moves

Image Copyright: Mark Pauly

Camera Coordinates



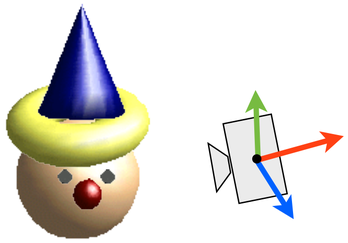
camera coordinates



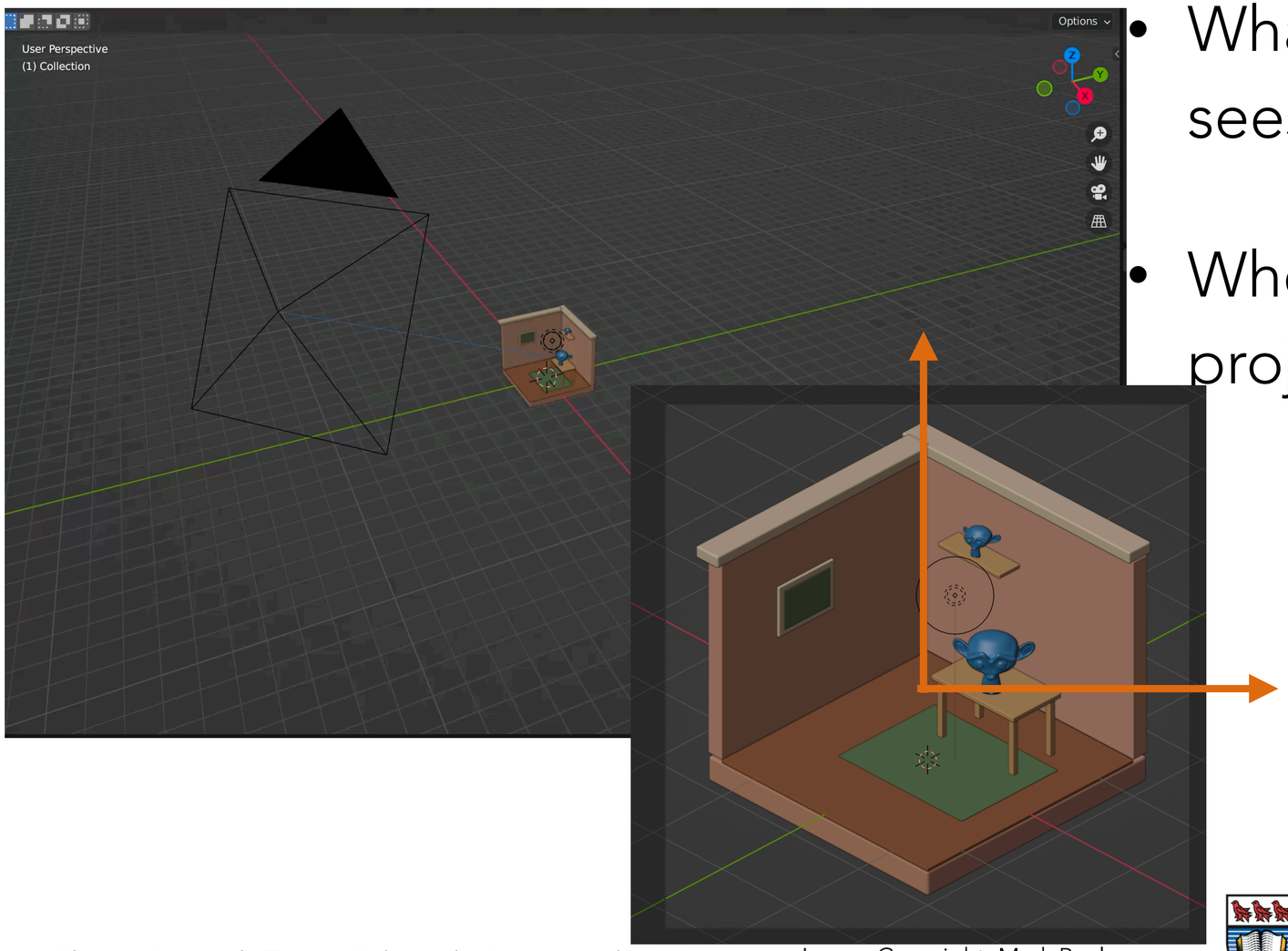
- What the camera sees
- When camera projection happens

Image Copyright: Mark Pauly

Camera Coordinates



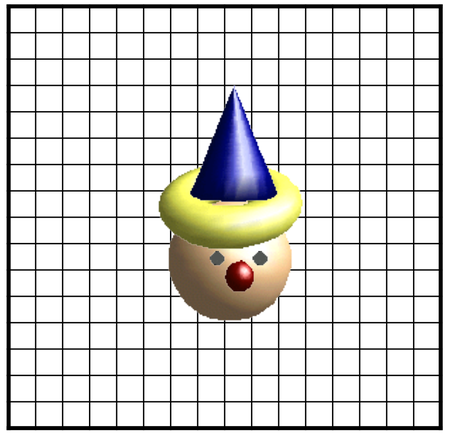
camera coordinates



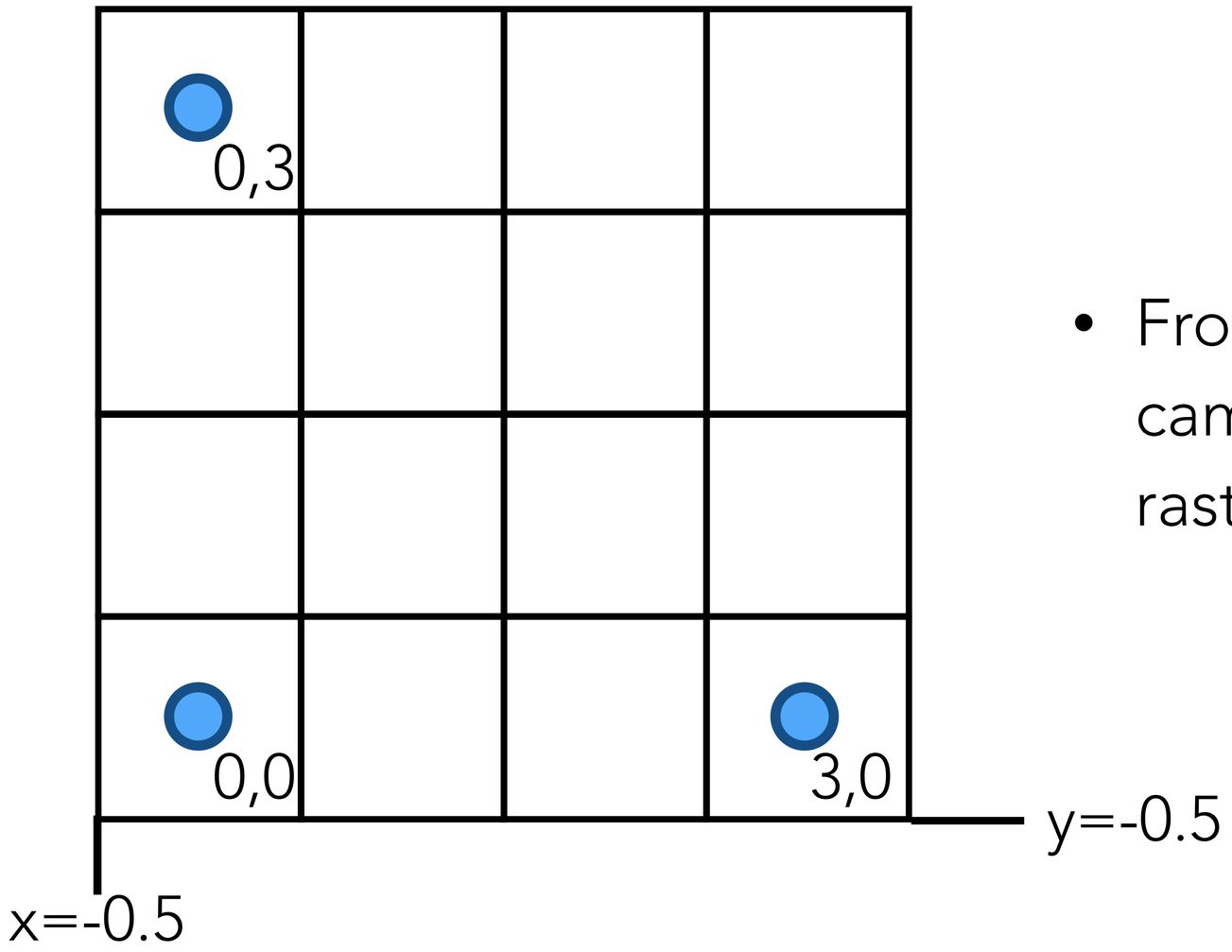
- What the camera sees
- When camera projection happens

Image Copyright: Mark Pauly

Screen Coordinates

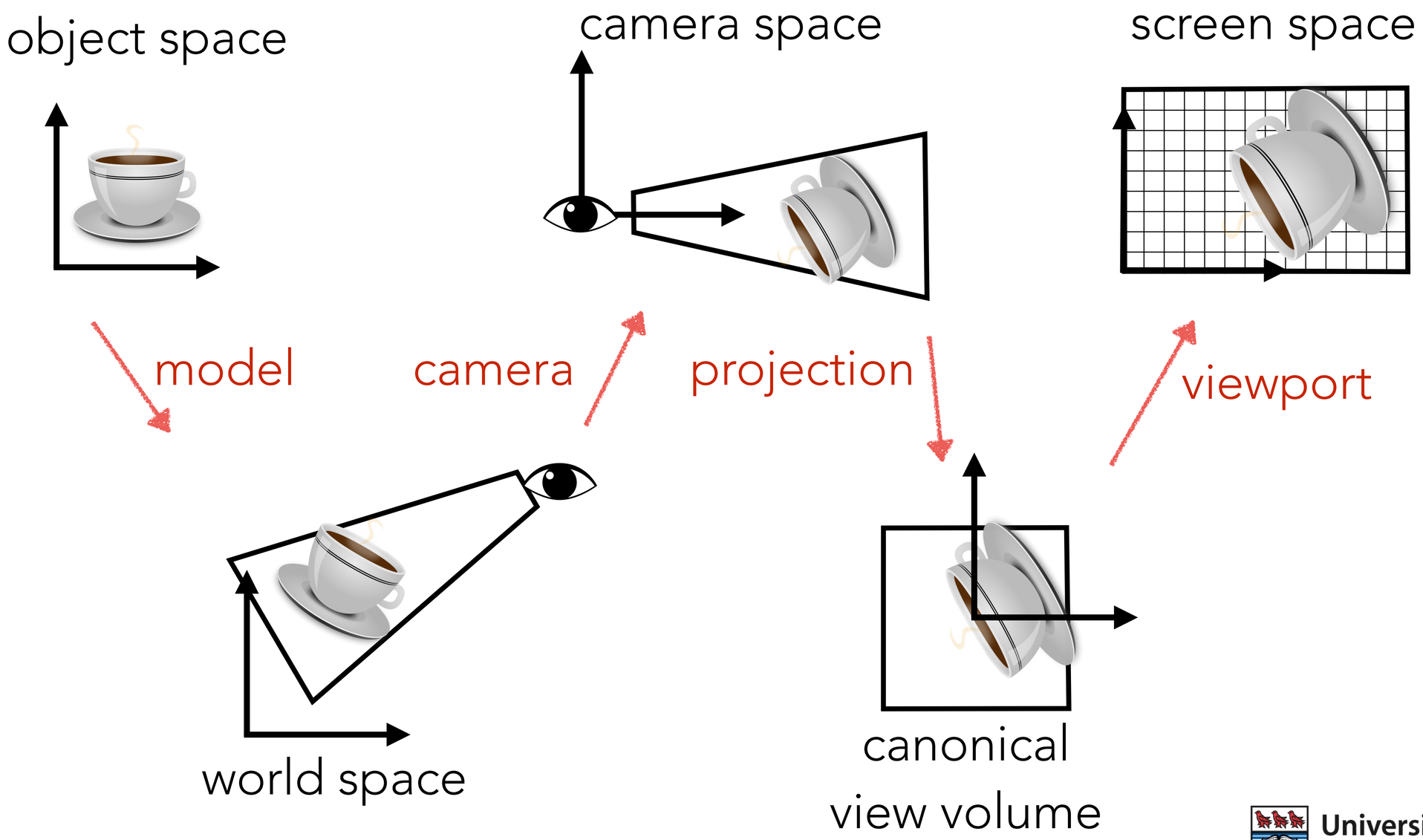


screen coordinates

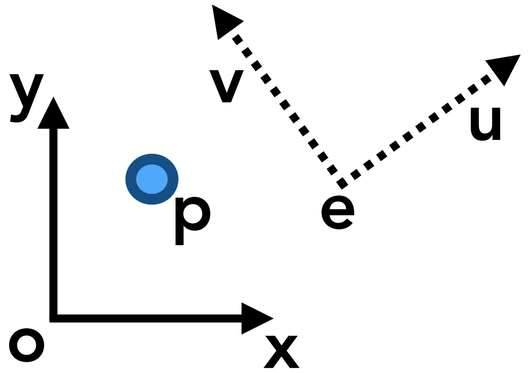


- From the canonical camera to actual raster image space

Viewing Transformation

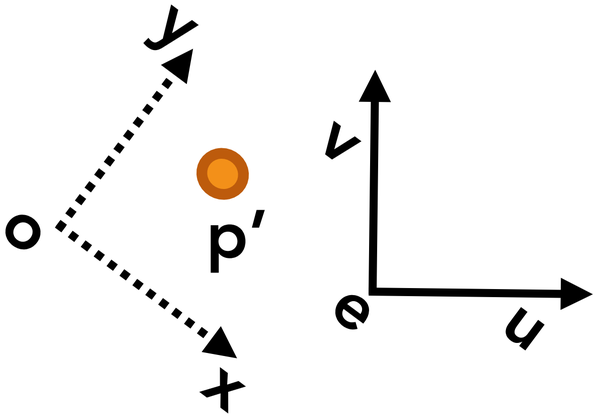


Change of frame



world coordinates

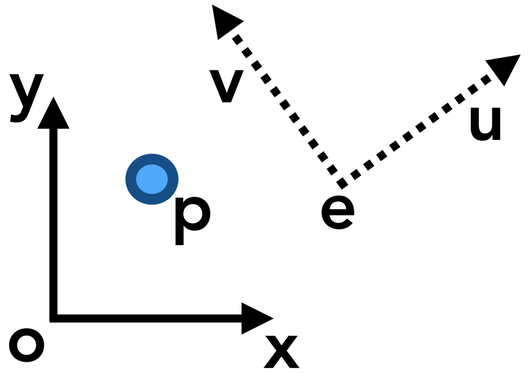
Transform the camera coordicate p’ to the world coordinate p



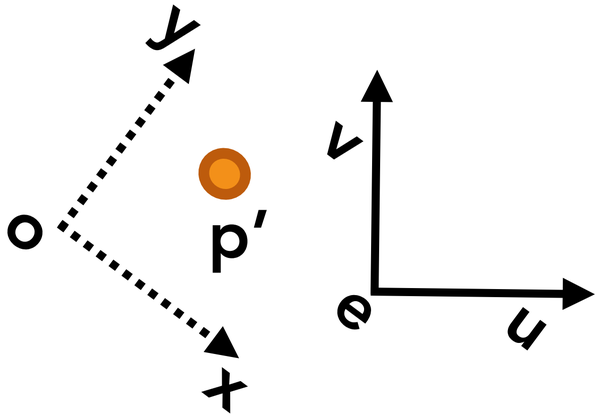
camera coordinates

University of Victoria | Computer Science

Change of frame



$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

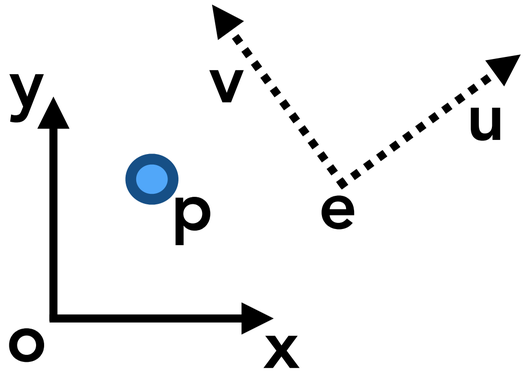


world coordinates

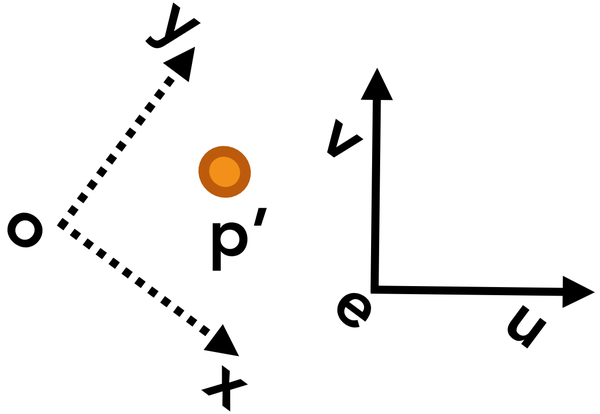
camera coordinates

University of Victoria | Computer Science

Change of frame



$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$



Camera frame: origin + basis

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$

world coordinates

camera coordinates

University of Victoria | Computer Science

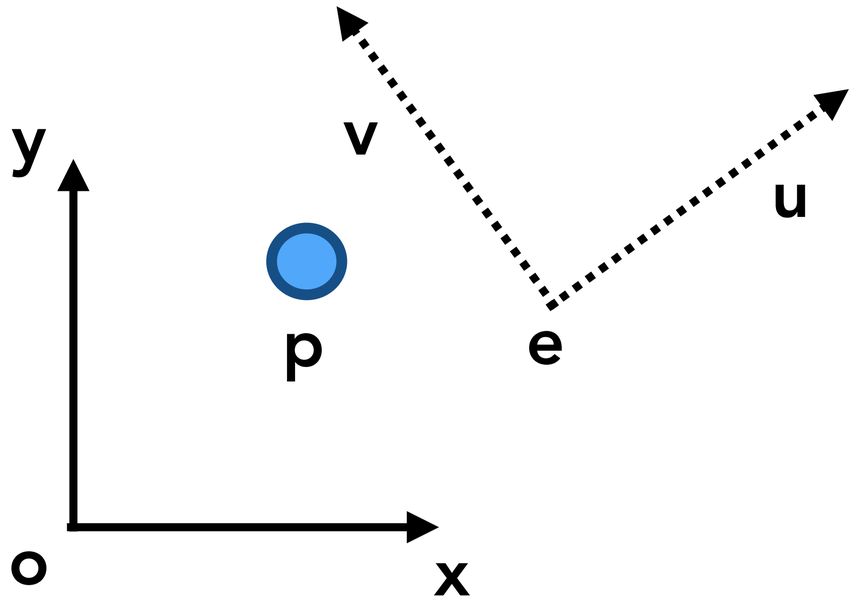
Change of frame

$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

$$\mathbf{p} = \qquad p'_u \mathbf{u}$$

Camera frame: origin + basis
in world coordinates

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$



University of Victoria | Computer Science

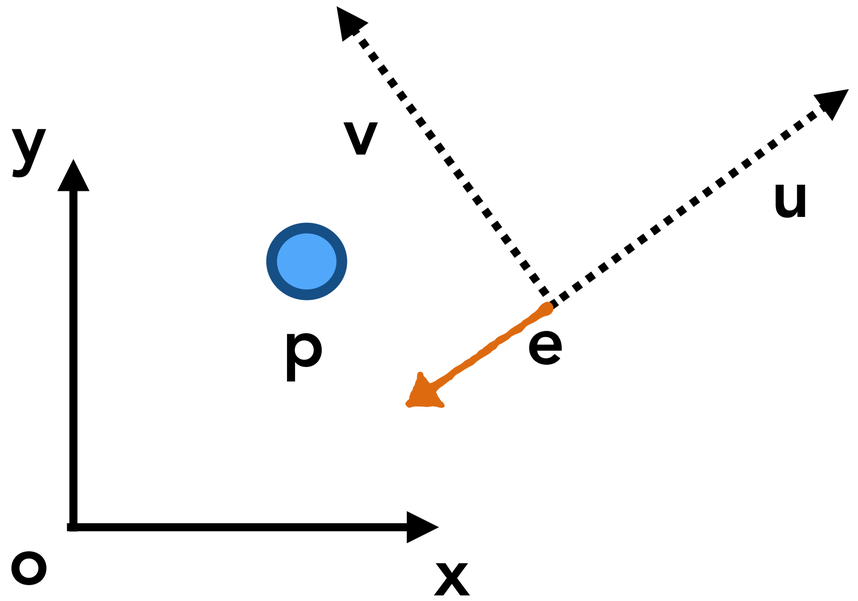
Change of frame

$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

$$\mathbf{p} = \boxed{p'_u \mathbf{u}}$$

Camera frame: origin + basis
in world coordinates

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$



University of Victoria | Computer Science

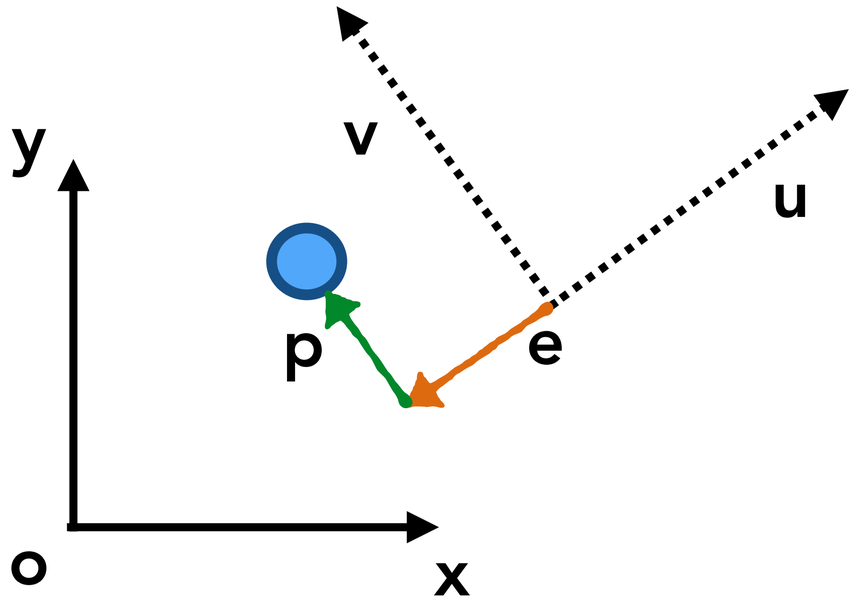
Change of frame

$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

$$\mathbf{p} = p'_u\mathbf{u} + p'_v\mathbf{v}$$

Camera frame: origin + basis
in world coordinates

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$



University of Victoria | Computer Science

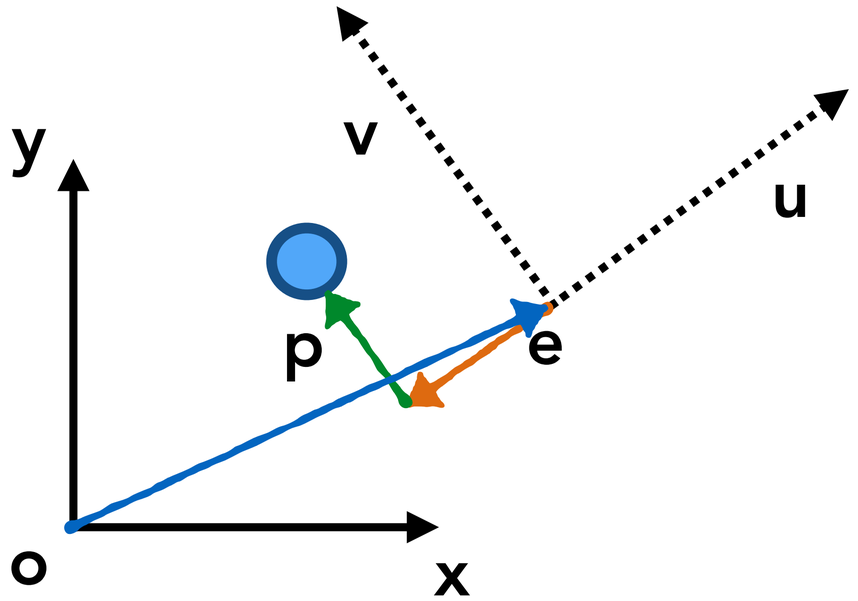
Change of frame

$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

$$\mathbf{p} = \mathbf{e} + p'_u\mathbf{u} + p'_v\mathbf{v}$$

Camera frame: origin + basis
in world coordinates

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$



University of Victoria | Computer Science

Change of frame

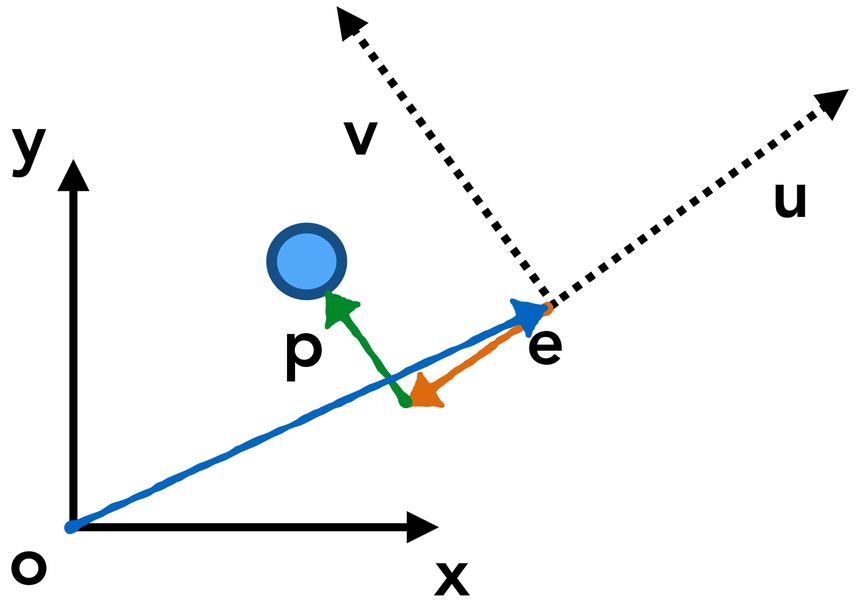
$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

Camera frame: origin + basis

in world coordinates

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$

$$\begin{aligned}\mathbf{p} &= \mathbf{e} + p'_u\mathbf{u} + p'_v\mathbf{v} \\ &= (3,1) - (0.8, 0.6) + (-1.2, 1.6) \\ &= (1,2)\end{aligned}$$



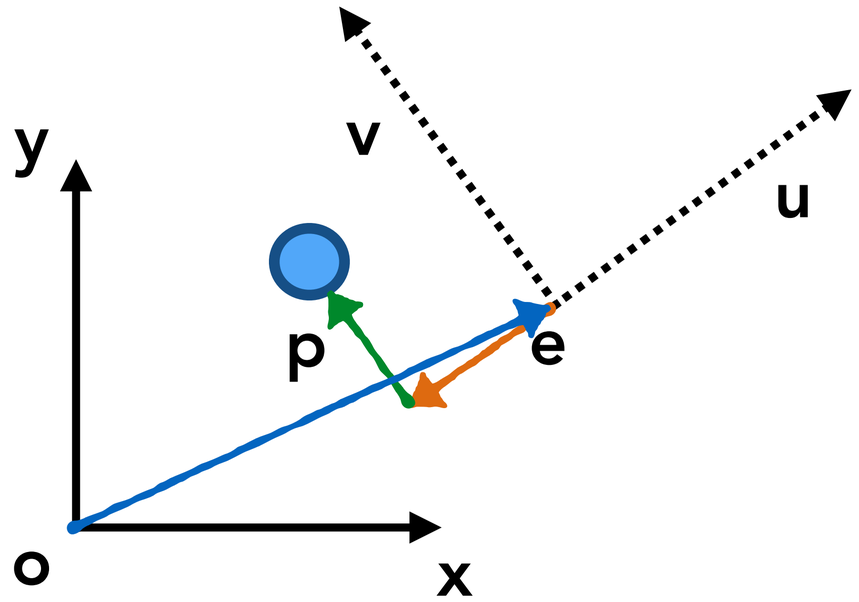
University of Victoria | Computer Science

Change of frame

$$\mathbf{p} = (1,\ 2), \qquad \mathbf{p}' = (-1,\ 2)$$

Camera frame: origin + basis
in world coordinates

$$\mathbf{e} = (3, 1), \mathbf{u} = (0.8,\ 0.6), \mathbf{v} = (-0.6,\ 0.8)$$



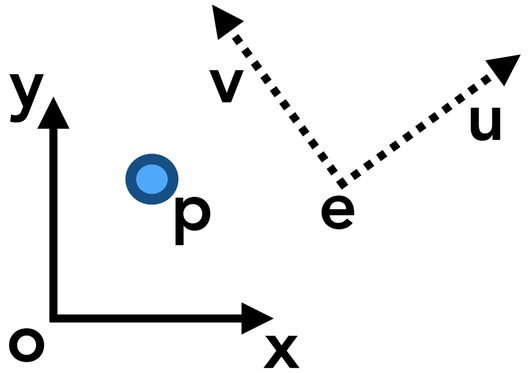
$$\begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{e} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p'_u \\ p'_v \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.8 & -0.6 & 3 \\ 0.6 & 0.8 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 2 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

University of Victoria | Computer Science

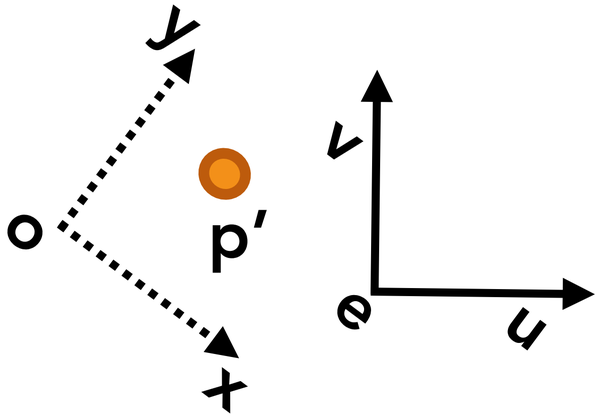
Change of frame



world coordinates

$$\mathbf{p} = \mathbf{F}\,\mathbf{p}'$$





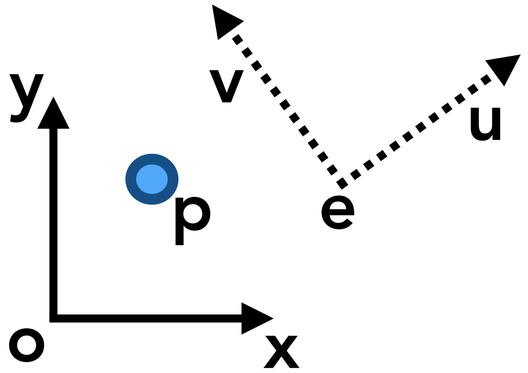
camera coordinates

Transform the world coordicate p to the camera coordinate p'?

$$\mathbf{F}^{-1}\,\mathbf{F} = \mathbf{I}$$

University of Victoria | Computer Science

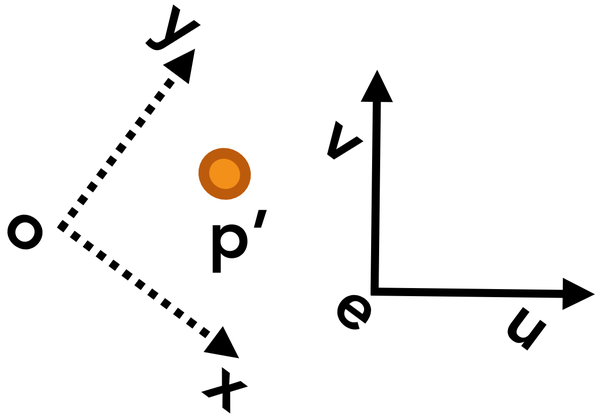
Change of frame



world coordinates

$$\mathbf{p} = \mathbf{F}\,\mathbf{p}'$$

$$\mathbf{F}^{-1}\,\mathbf{p} = \mathbf{p}'$$



camera coordinates

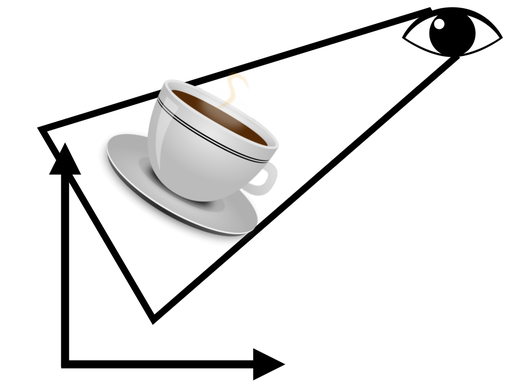
$$\mathbf{F} = \mathbf{T}(\mathbf{e})\,\mathbf{R}$$

$$\mathbf{F}^{-1} = \mathbf{R}^{-1}\,\mathbf{T}(\mathbf{e})^{-1} = \mathbf{R}^{T}\,\mathbf{T}(-\mathbf{e})$$

$$= \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -e_x \\ 0 & 1 & -e_y \\ 0 & 0 & 1 \end{bmatrix}$$

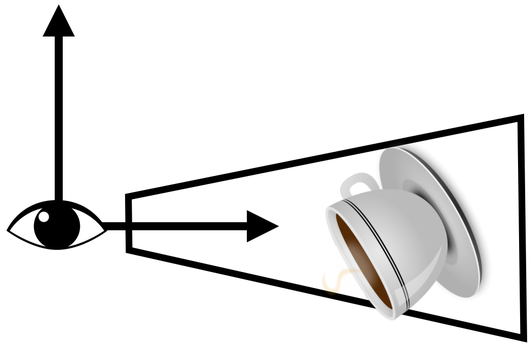
University of Victoria | Computer Science

Camera Transformation

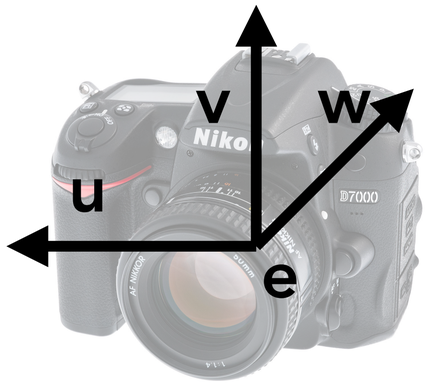


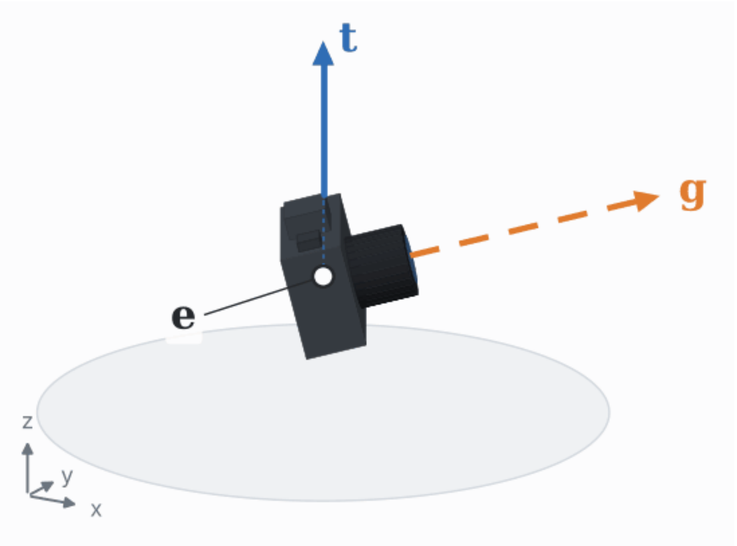
world space

camera

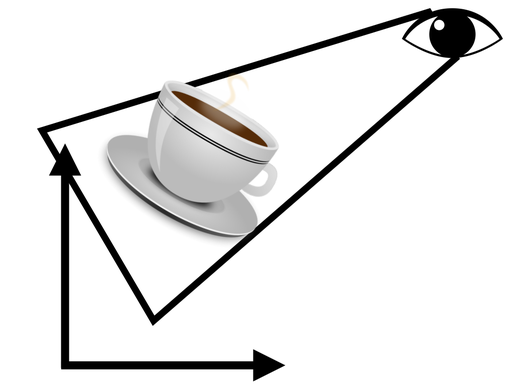


camera space



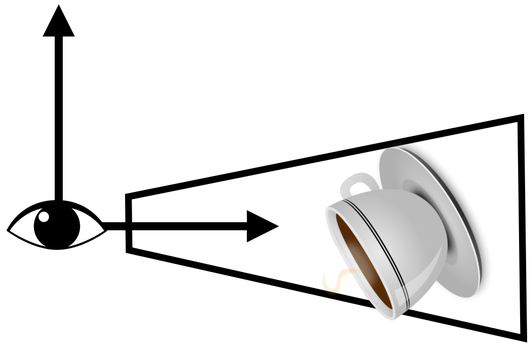


Camera Transformation



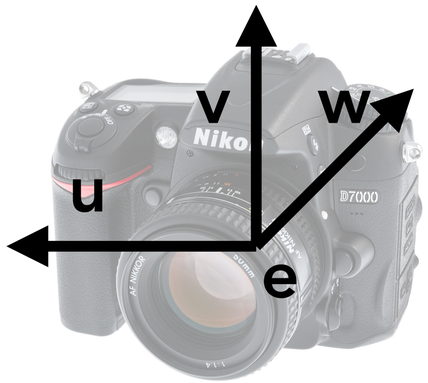
world space

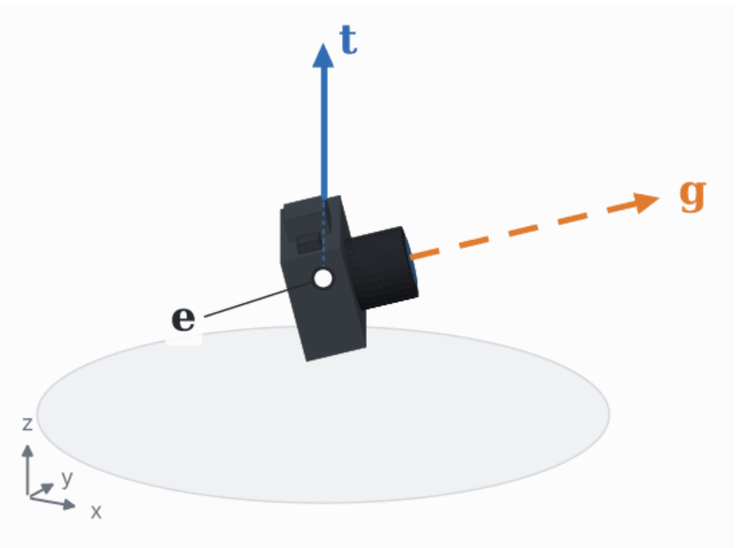
camera



camera space

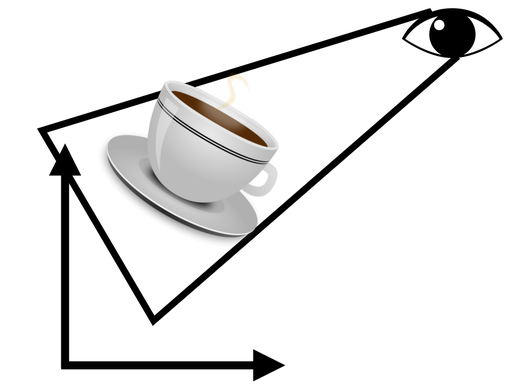
1. Construct the camera reference system given:





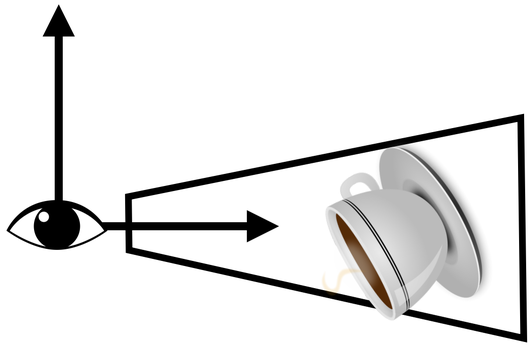
University of Victoria | Computer Science

Camera Transformation



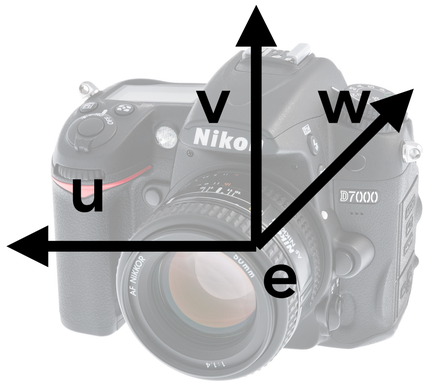
world space

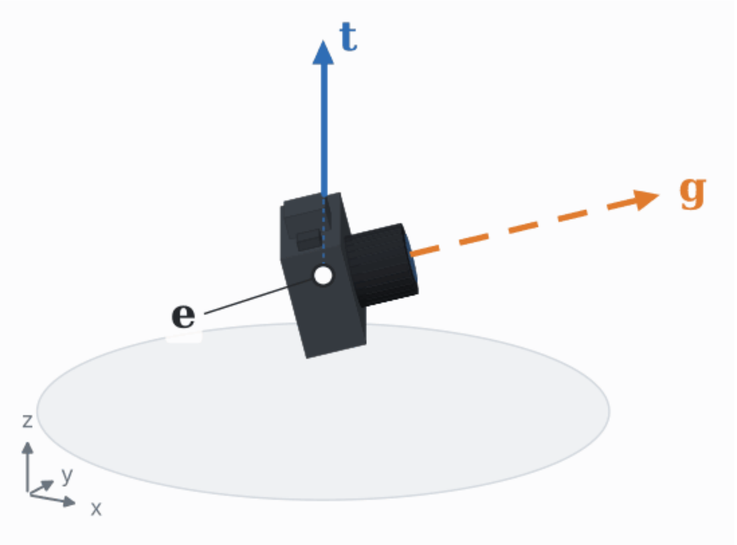
camera



camera space

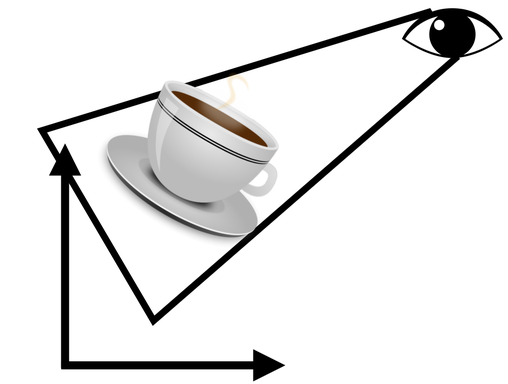
1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**





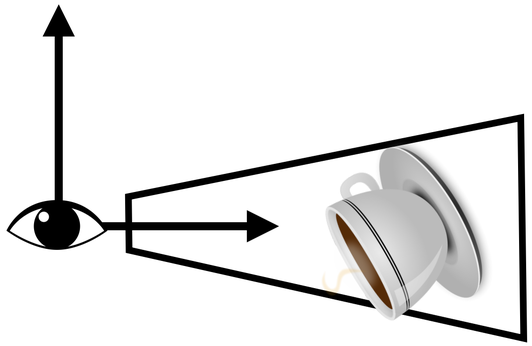
University of Victoria | Computer Science

Camera Transformation



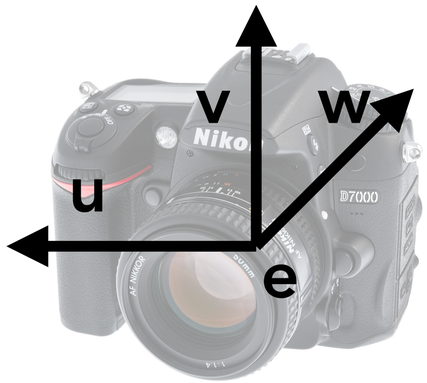
world space

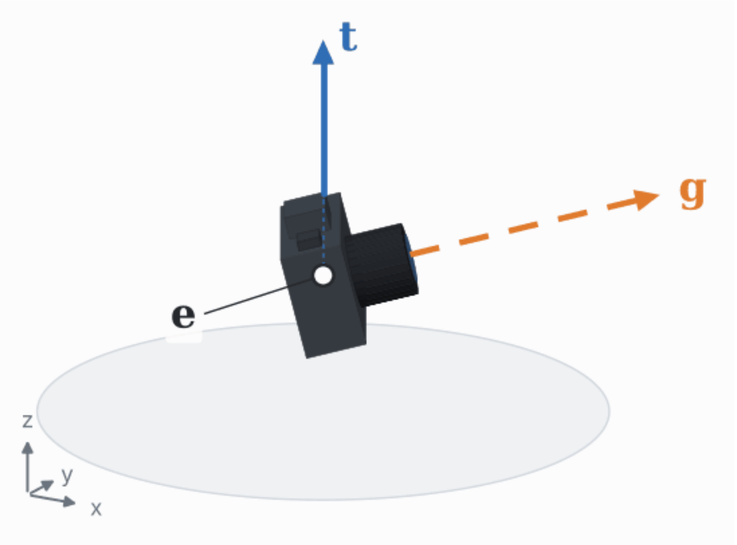
camera



camera space

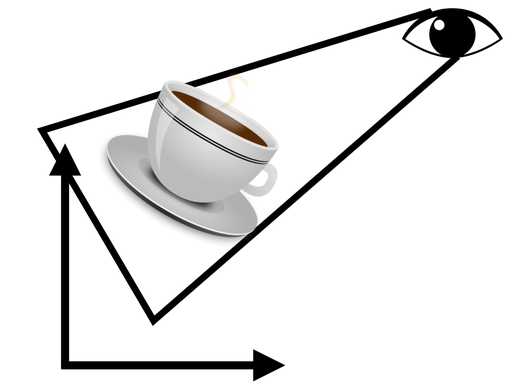
1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**





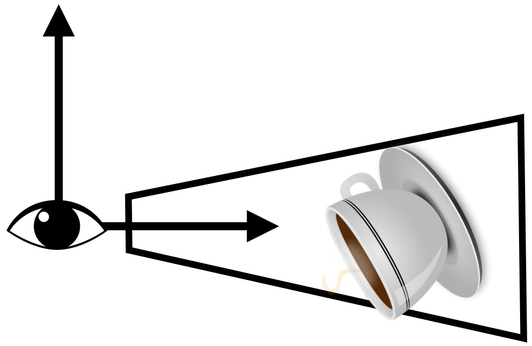
University of Victoria | Computer Science

Camera Transformation



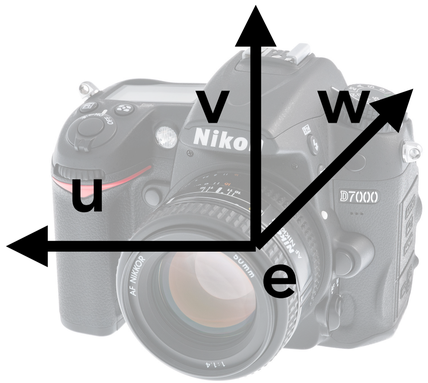
world space

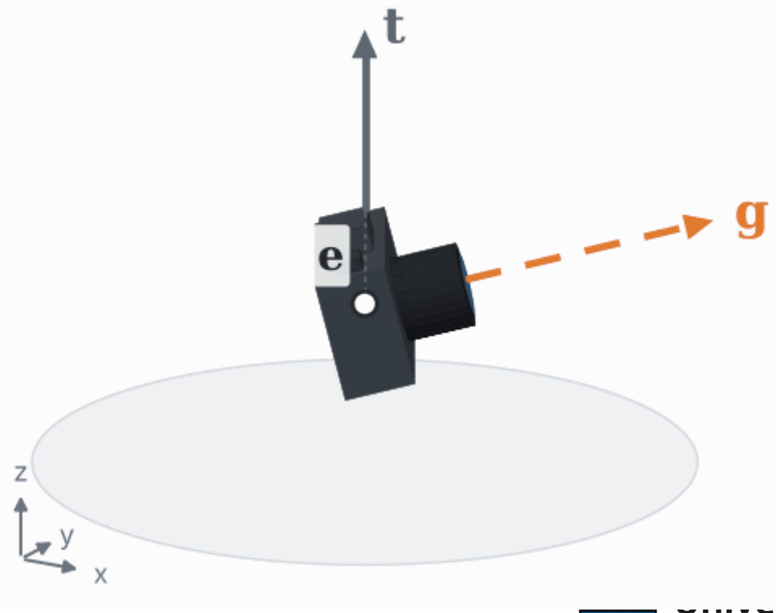
camera



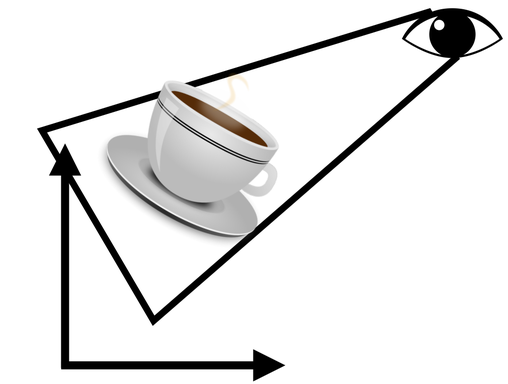
camera space

1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**



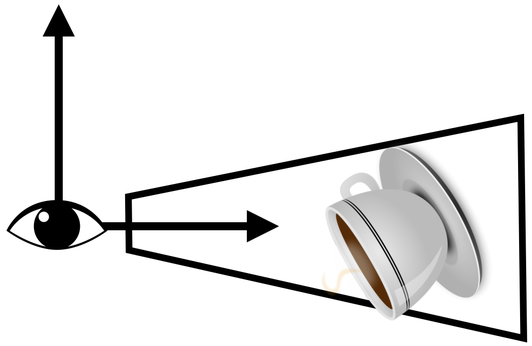


Camera Transformation



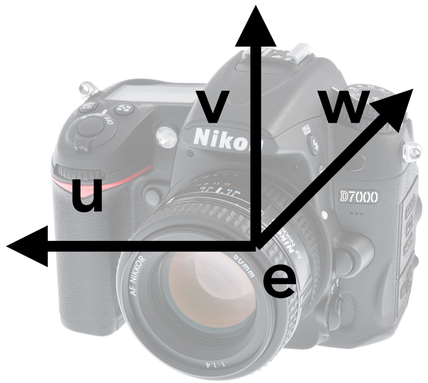
world space

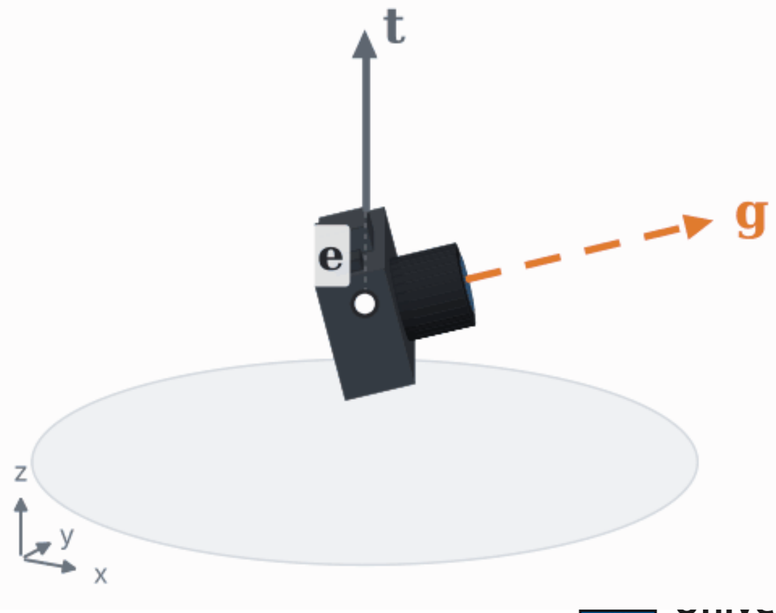
camera



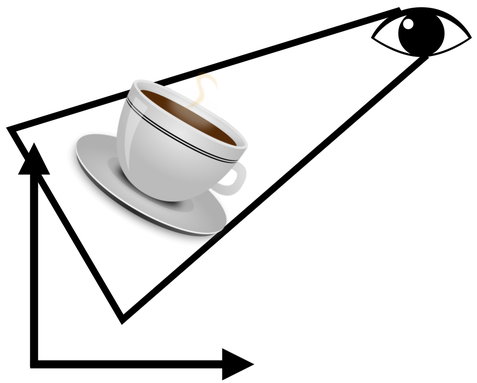
camera space

1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**



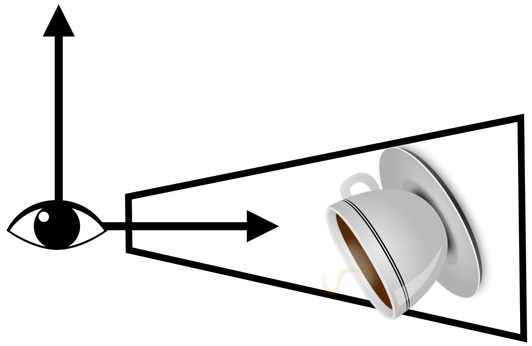


Camera Transformation



world space

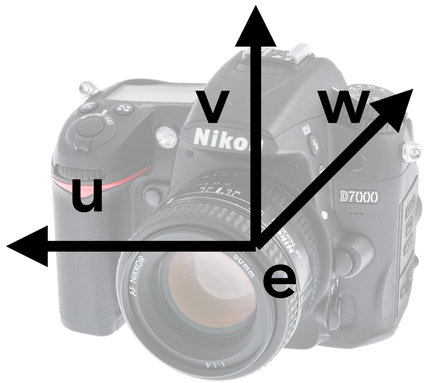
camera

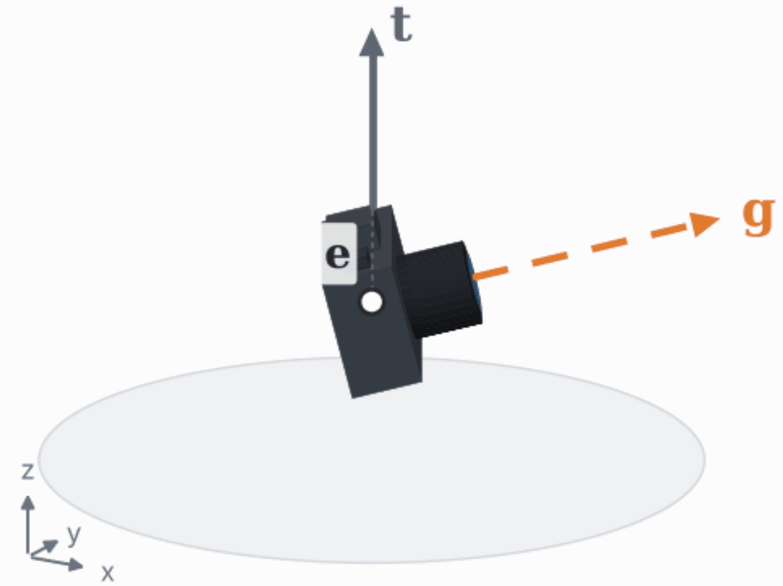


camera space

1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**

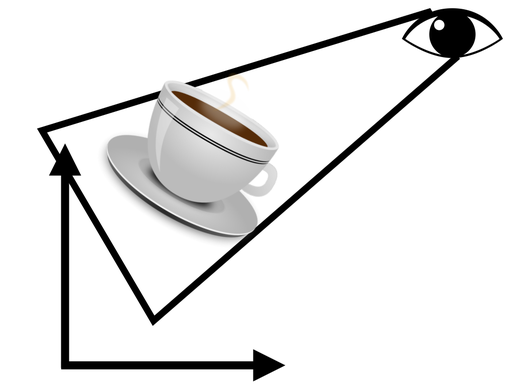
$$\mathbf{w} = -\frac{\mathbf{g}}{||\mathbf{g}||}$$





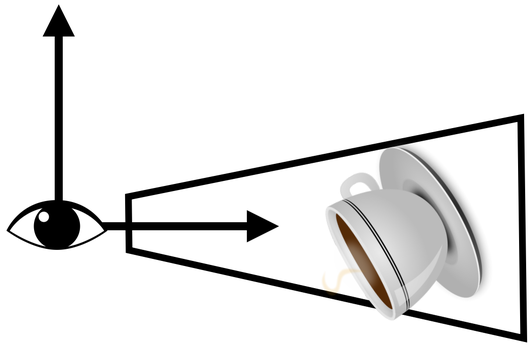
University of Victoria | Computer Science

Camera Transformation



world space

camera

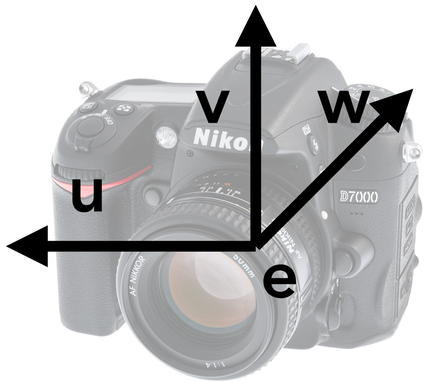


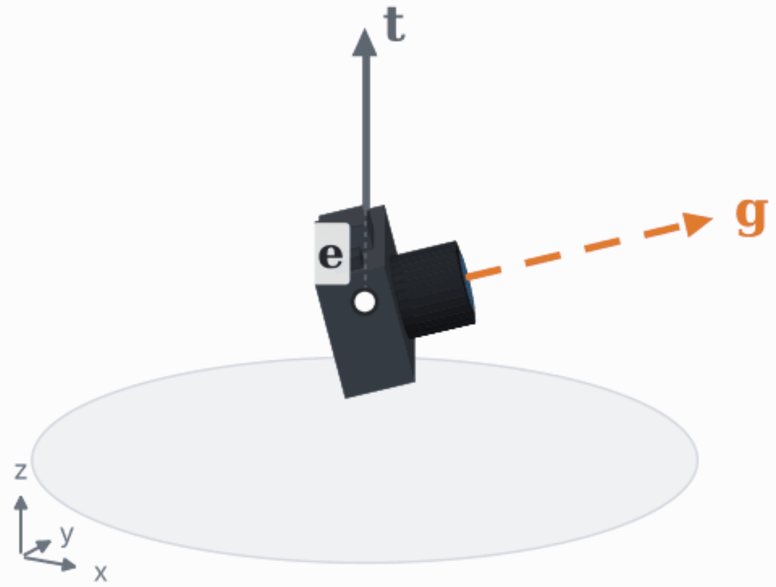
camera space

1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**

$$\mathbf{w} = -\frac{\mathbf{g}}{||\mathbf{g}||}$$

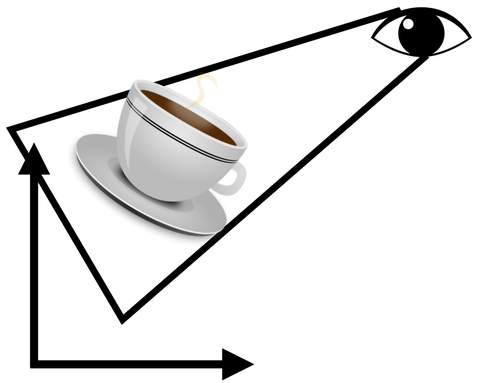
$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{||\mathbf{t} \times \mathbf{w}||}$$





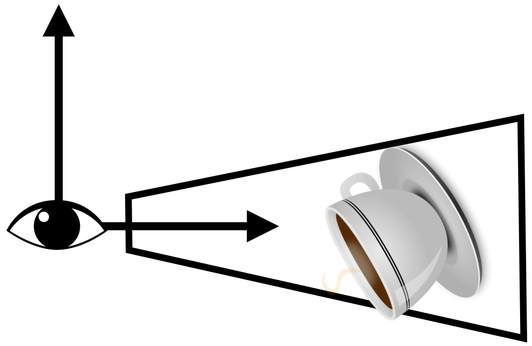
University of Victoria | Computer Science

Camera Transformation



world space

camera



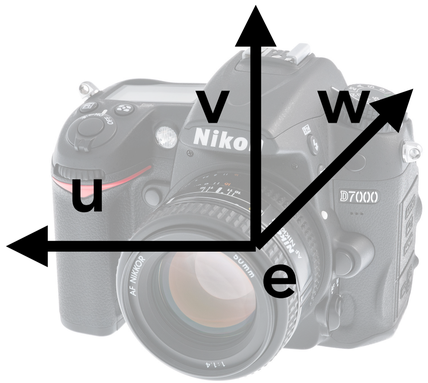
camera space

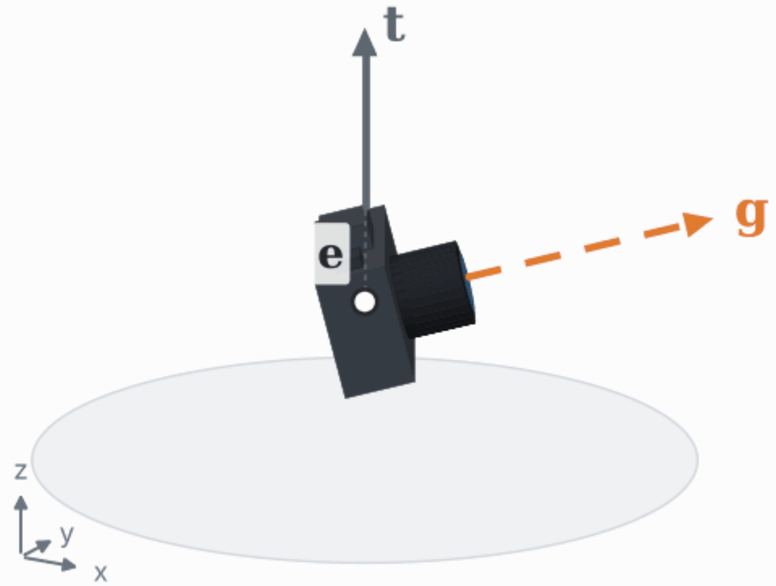
1. Construct the camera reference system given:
    1. The eye position **e**
    2. The gaze direction **g**
    3. The view-up vector **t**

$$\mathbf{w} = -\frac{\mathbf{g}}{||\mathbf{g}||}$$

$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{||\mathbf{t} \times \mathbf{w}||}$$

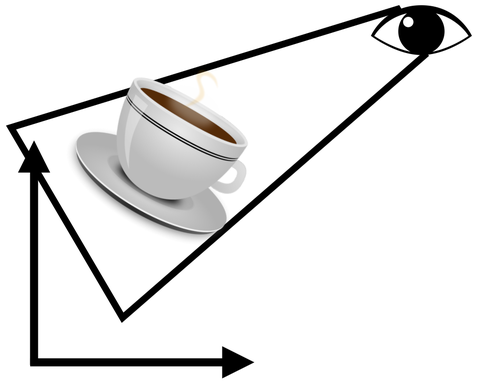
$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$





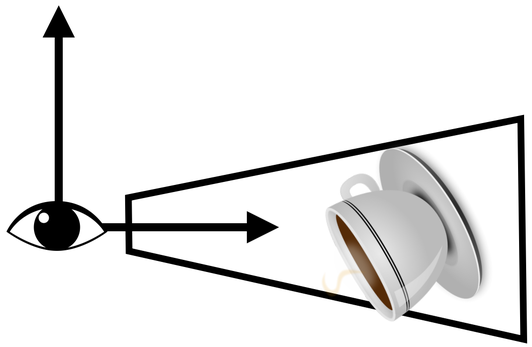
University of Victoria | Computer Science

Camera Transformation



world space

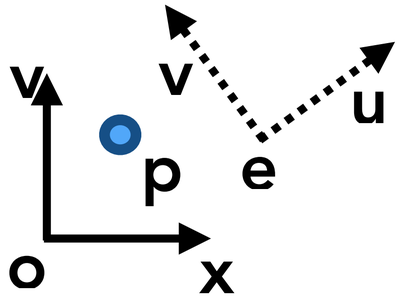
camera



camera space

2. Construct the unique transformations that converts world coordinates into camera coordinates

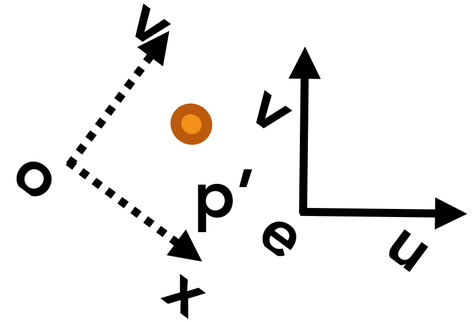
$$\mathbf{M}_{cam} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1}$$



world coordinates

$$\mathbf{F}^{-1}\,\mathbf{p} = \mathbf{p}'$$

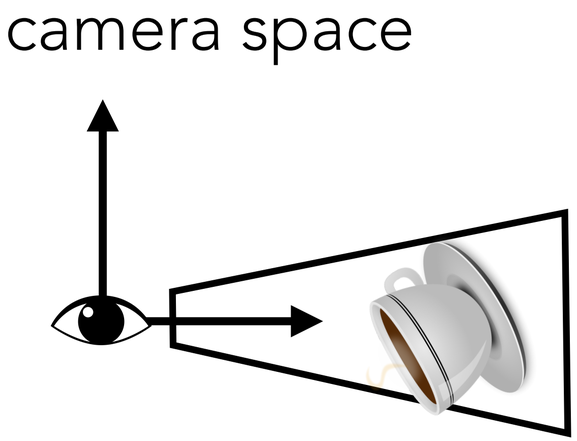
$$\mathbf{M}_{\mathrm{cam}}\,\mathbf{p} = \mathbf{p}'$$



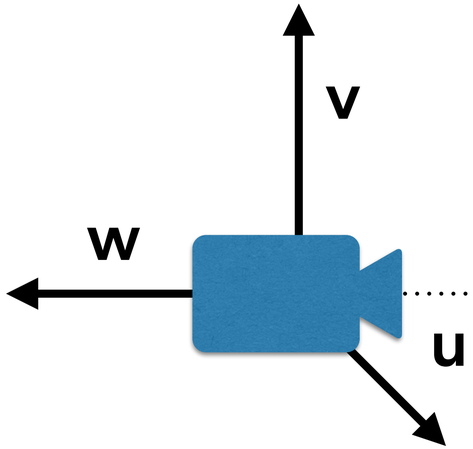
camera coordinates

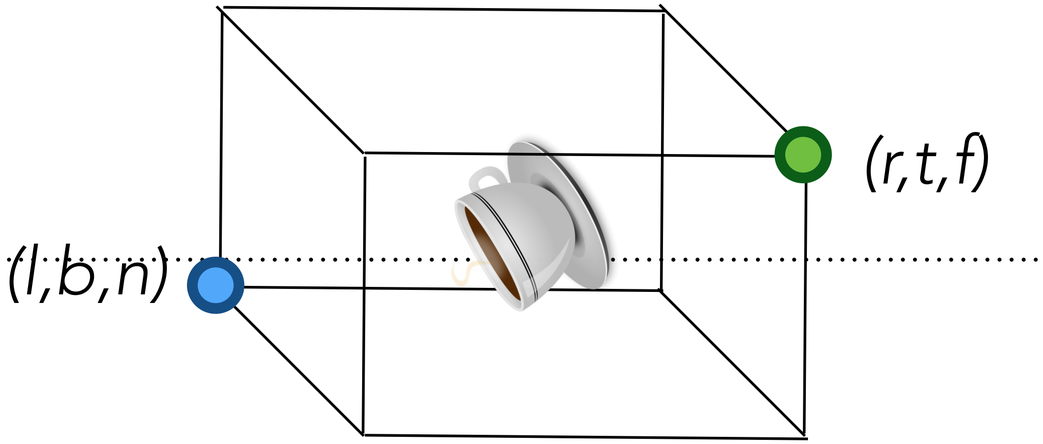
University of Victoria | Computer Science

Orthographic Projection



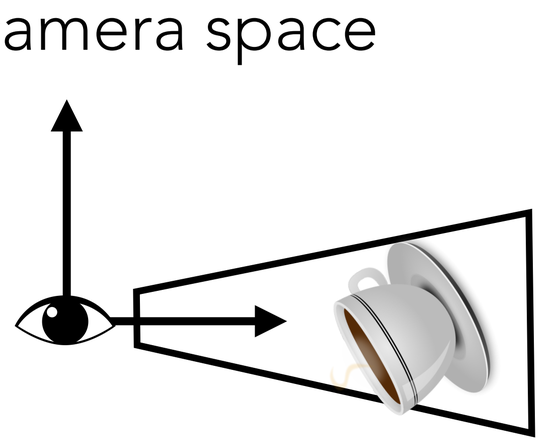
- Squeeze everything the camera can see into:
- Canonical view volume is the cube $[-1,1]^3$
- The camera sees only the view volume bounded by six planes:



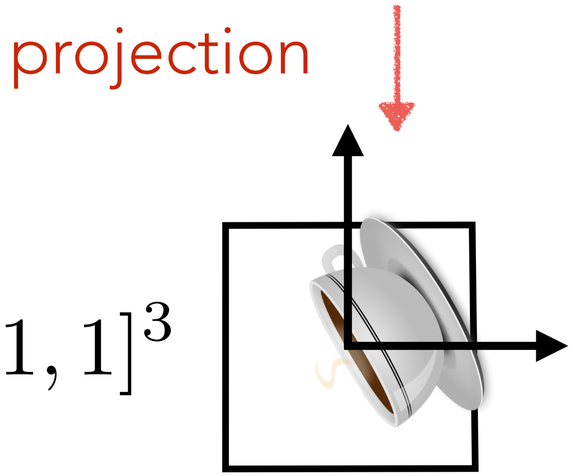


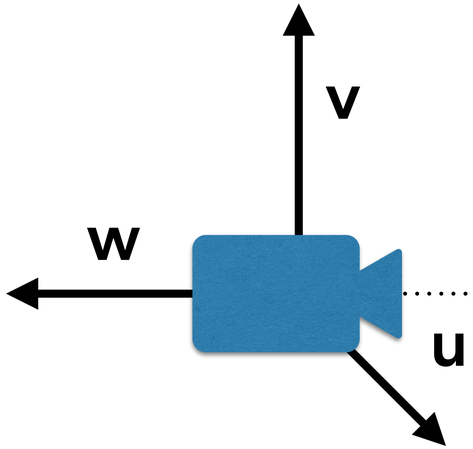
University of Victoria | Computer Science

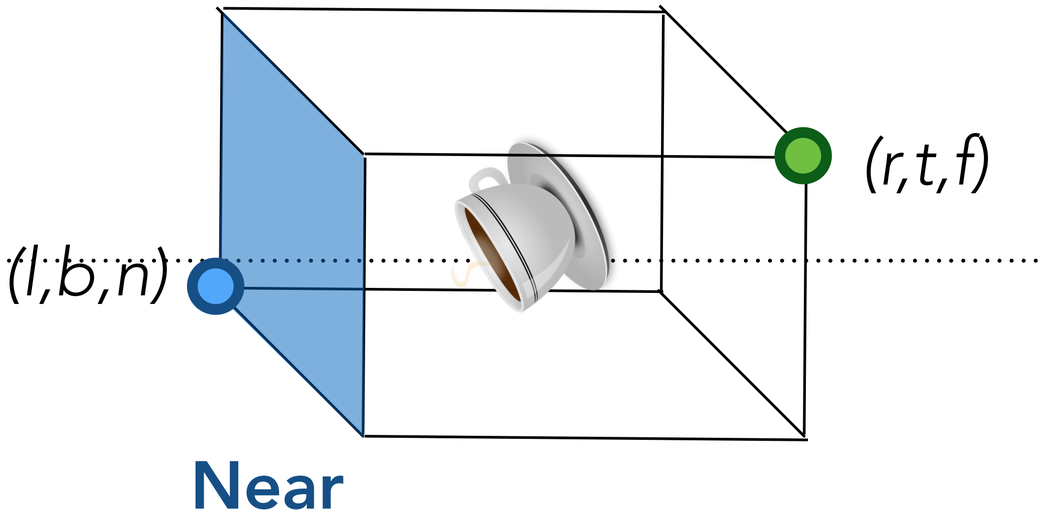
Orthographic Projection



- Squeeze everything the camera can see into:
- Canonical view volume is the cube $[-1,1]^3$
- The camera sees only the view volume bounded by six planes:

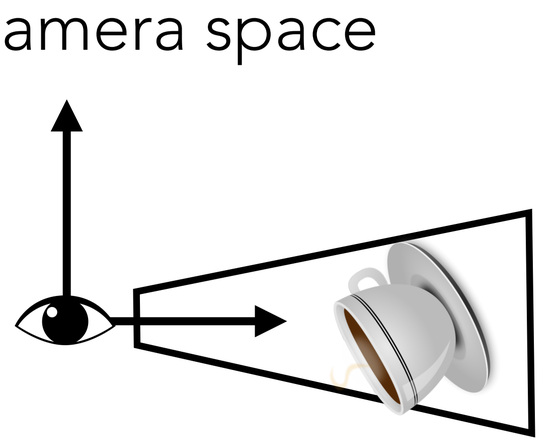




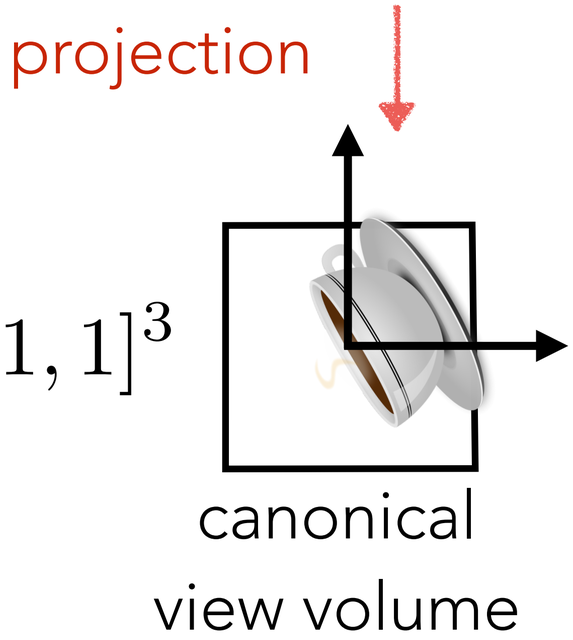


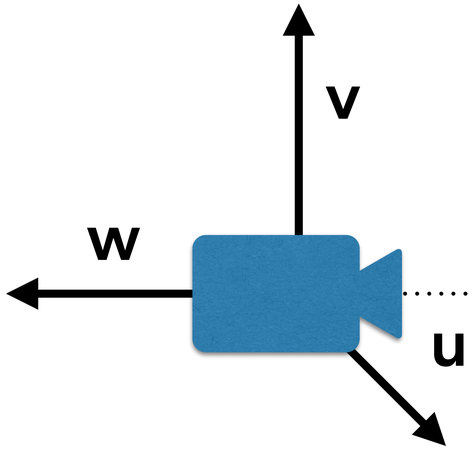
University of Victoria | Computer Science

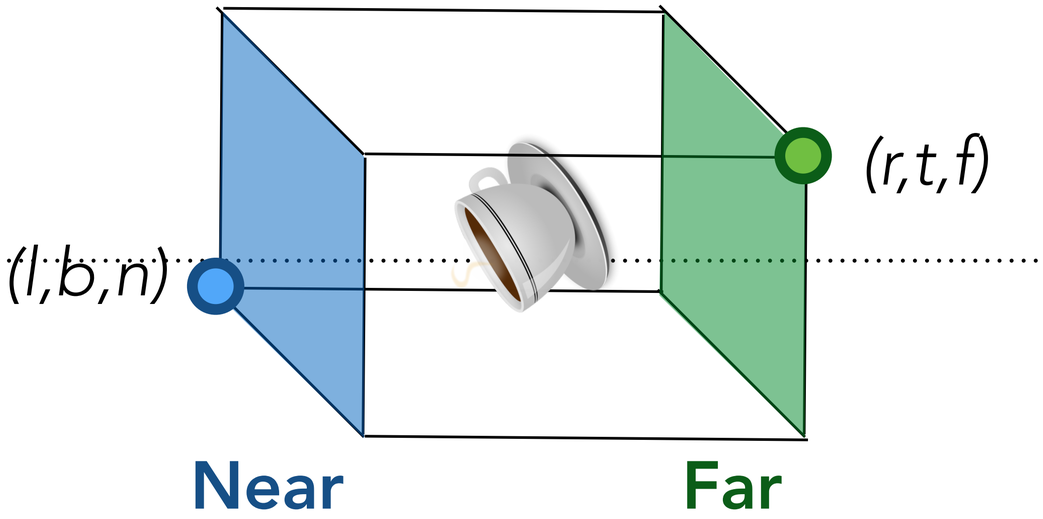
Orthographic Projection



- Squeeze everything the camera can see into:
- Canonical view volume is the cube $[-1,1]^3$
- The camera sees only the view volume bounded by six planes:

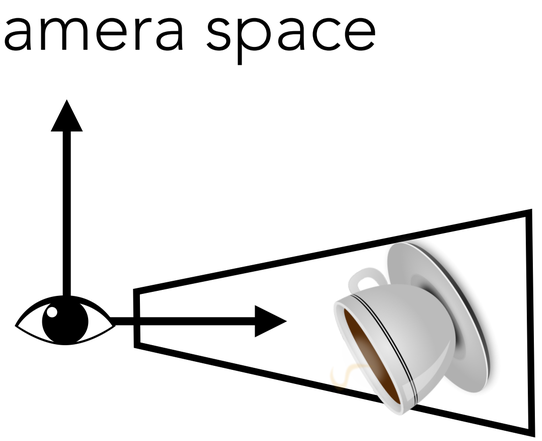




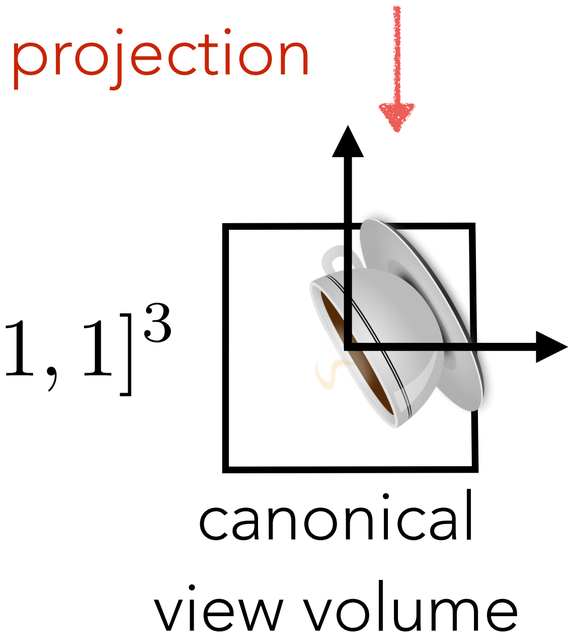


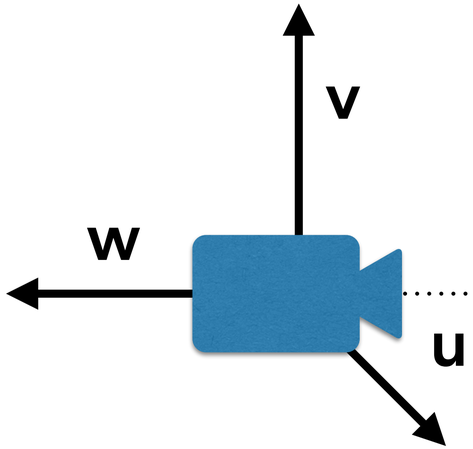
University of Victoria | Computer Science

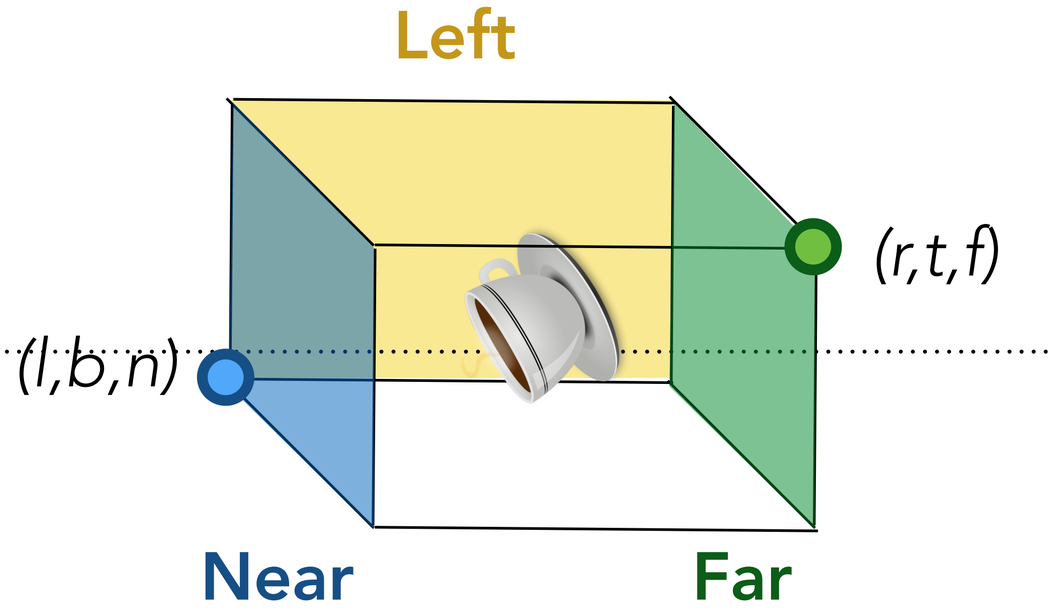
Orthographic Projection



- Squeeze everything the camera can see into:
- Canonical view volume is the cube $[-1,1]^3$
- The camera sees only the view volume bounded by six planes:

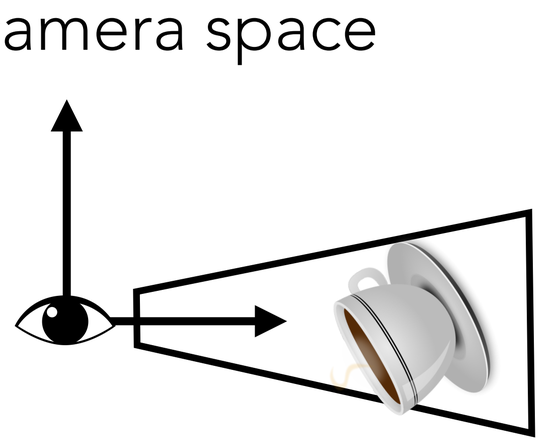




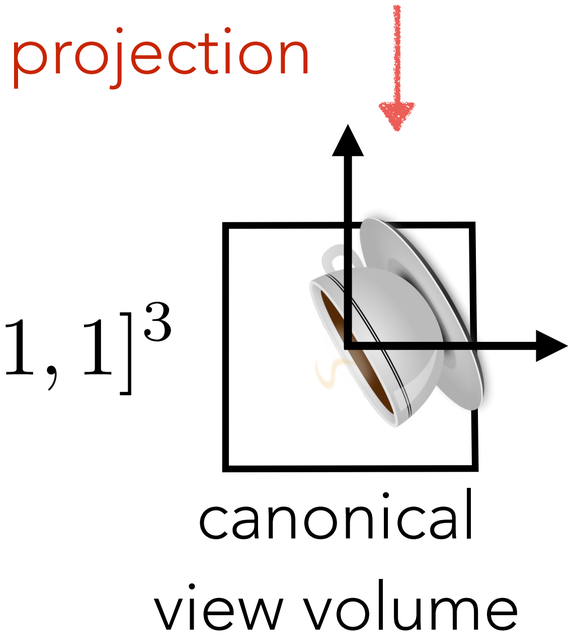


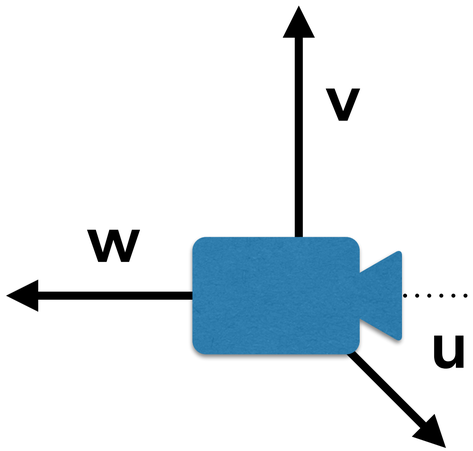
University of Victoria | Computer Science

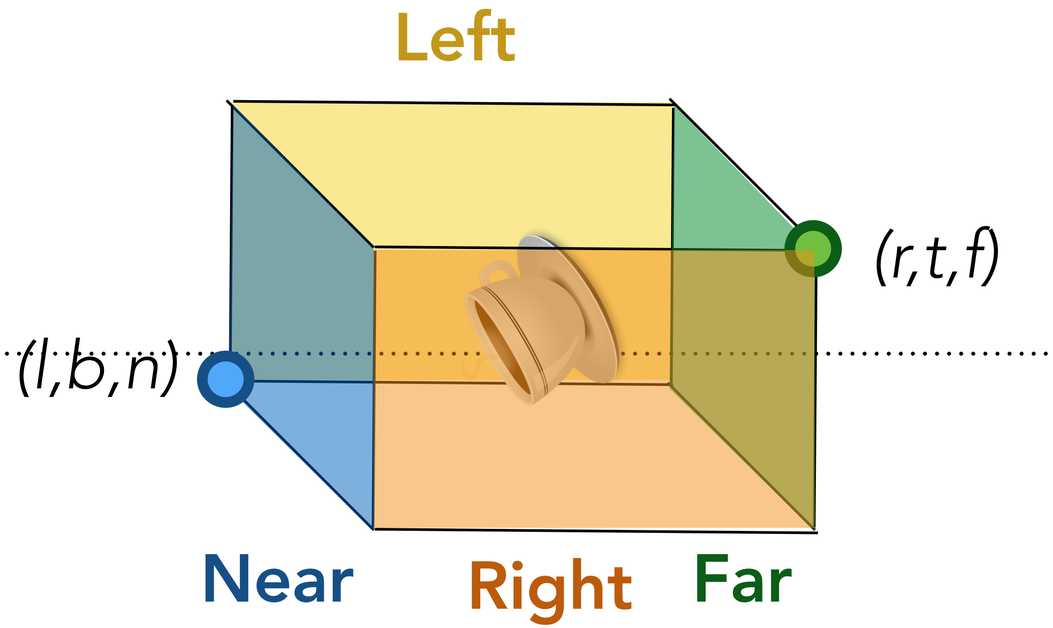
Orthographic Projection



- Squeeze everything the camera can see into:
- Canonical view volume is the cube $[-1,1]^3$
- The camera sees only the view volume bounded by six planes:

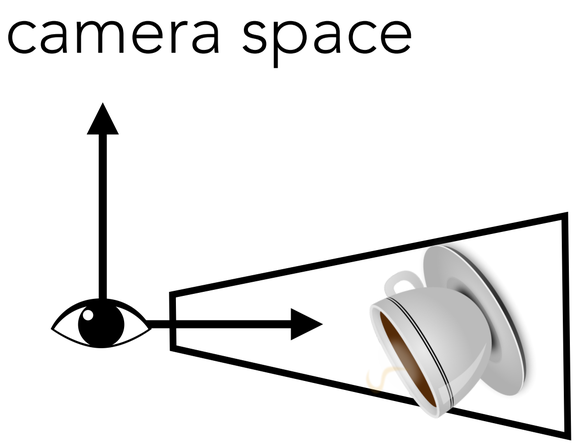


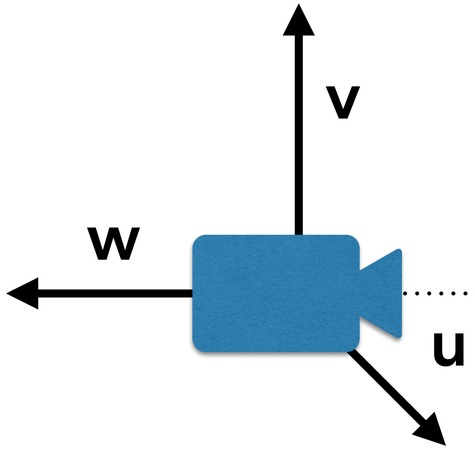


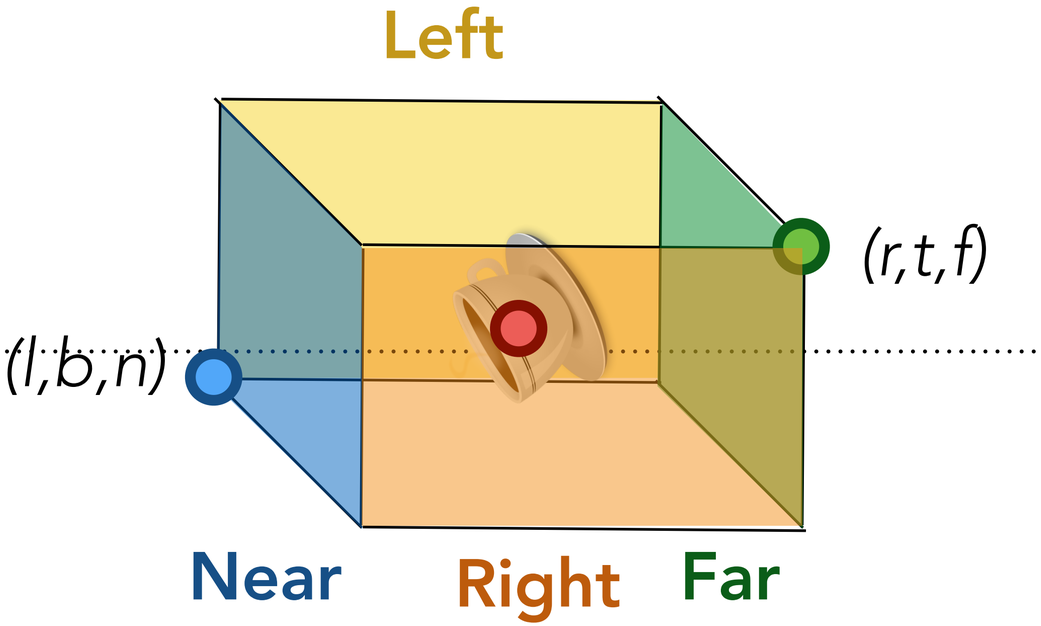


University of Victoria | Computer Science

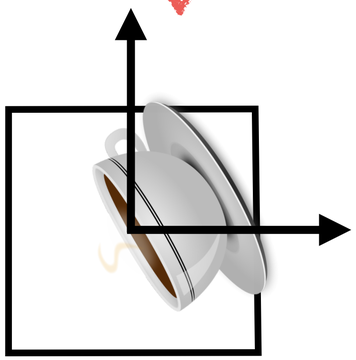
Orthographic Projection







projection



$[-1,1]^3$

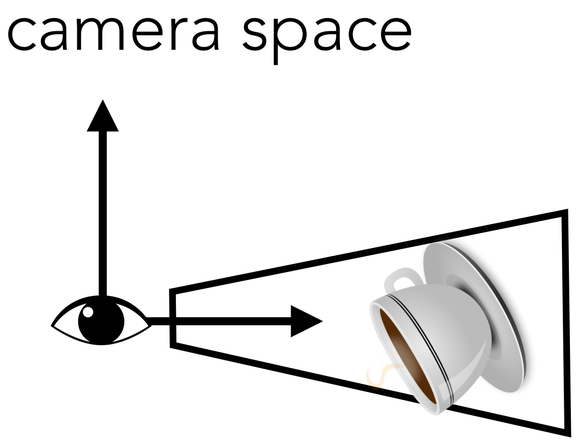
canonical
view volume

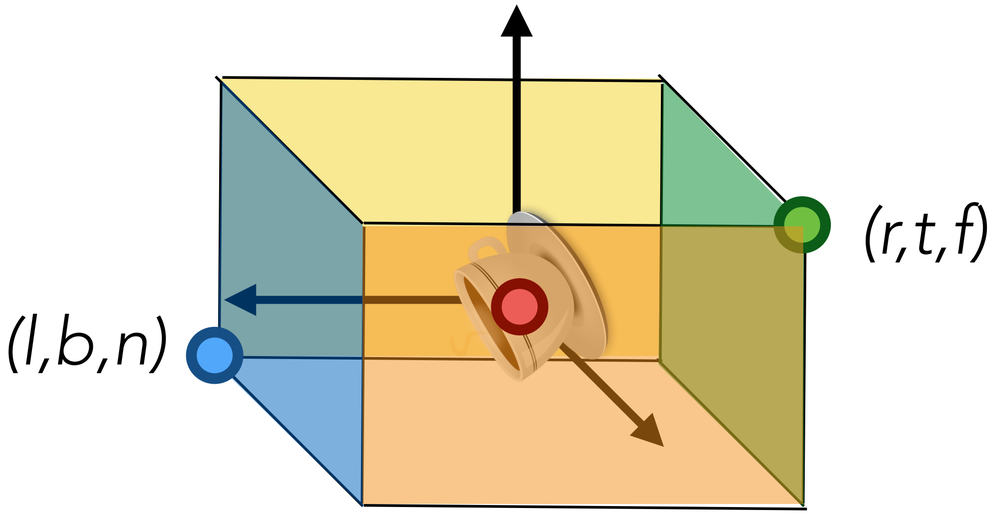
Center:

$$\mathbf{c} = \left(\frac{l+r}{2}, \frac{b+t}{2}, \frac{n+f}{2} \right)$$

University of Victoria | Computer Science

Orthographic Projection





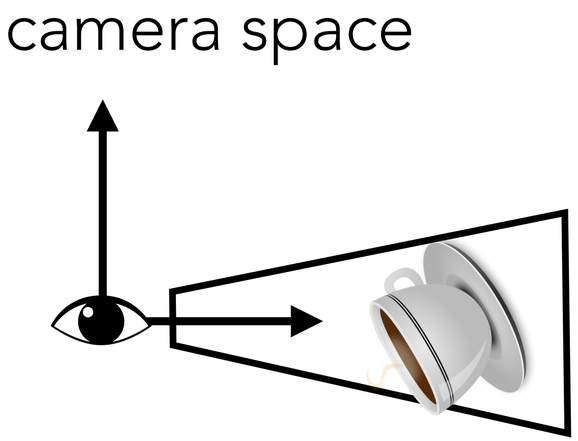
Center:

$$\mathbf{c} = \left(\frac{l+r}{2}, \frac{b+t}{2}, \frac{n+f}{2} \right)$$

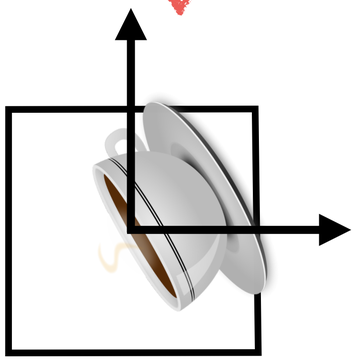
Translate **-c** to put the center at the origin

University of Victoria | Computer Science

Orthographic Projection

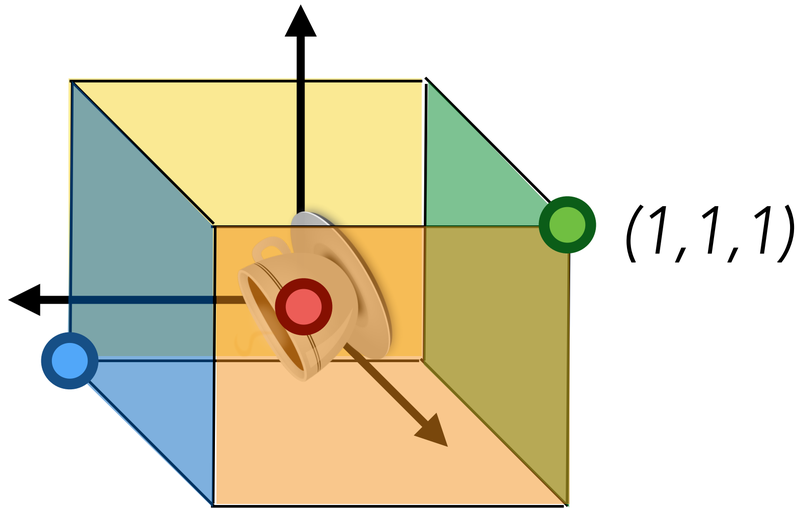


projection



$[-1,1]^3$

canonical
view volume

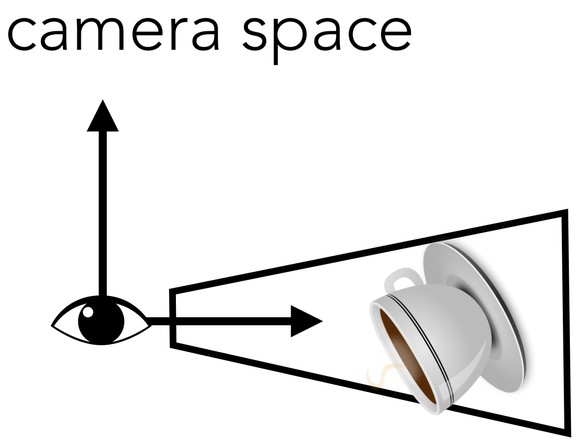


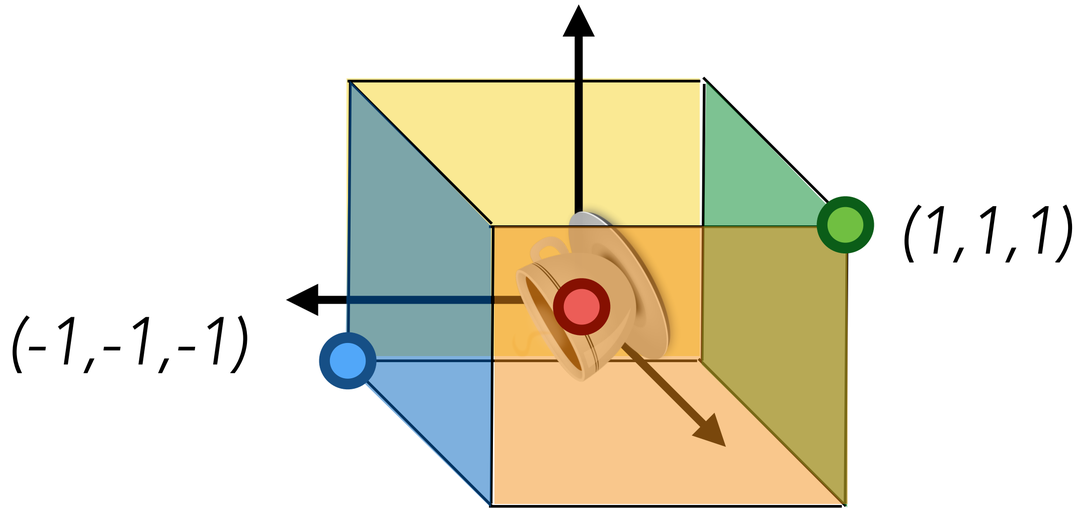
Scale the view volume to the canonical view cube:

$$s_x = \frac{2}{r-l}, s_y = \frac{2}{t-b}, s_z = \frac{2}{n-f}$$

University of Victoria | Computer Science

Orthographic Projection



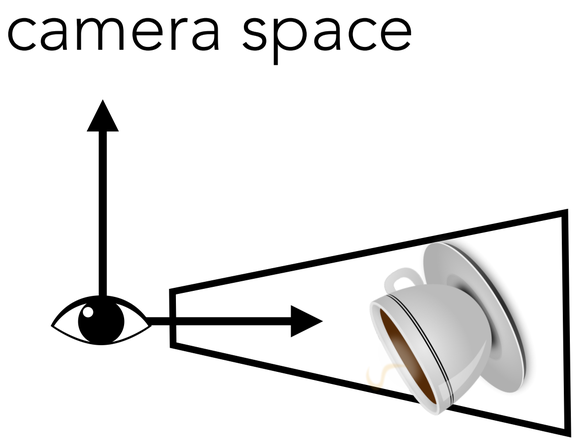


Translate then scale:

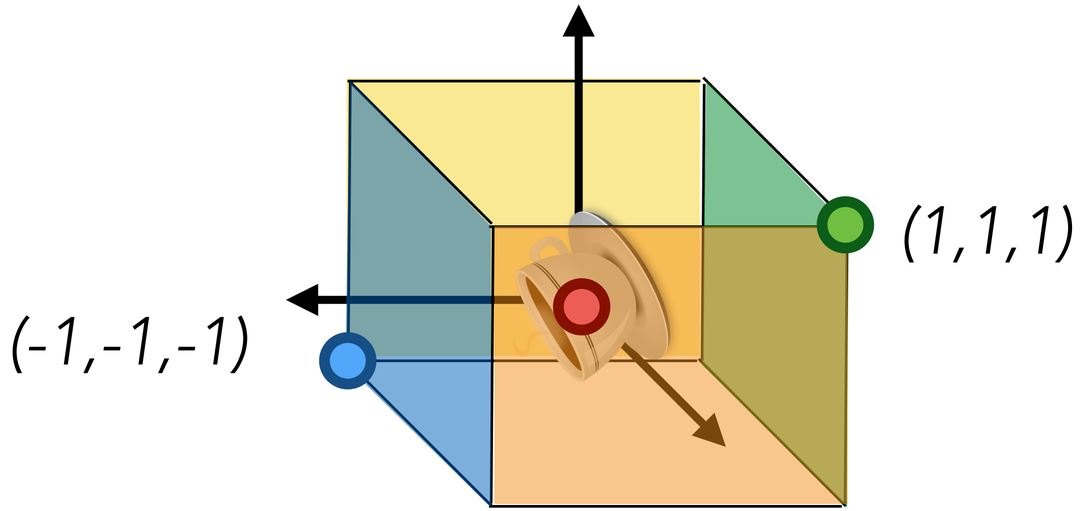
$$\mathbf{T}(-\mathbf{c}) = \begin{bmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{S} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{n-f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

University of Victoria | Computer Science

Orthographic Projection



$[-1,1]^3$



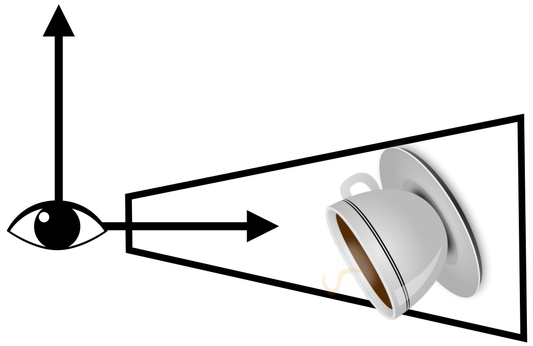
Translate then scale: $\mathbf{M}_{orth} = \mathbf{S}\,\mathbf{T}(-\mathbf{c})$

$$\mathbf{S} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{n-f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{T}(-\mathbf{c}) = \begin{bmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

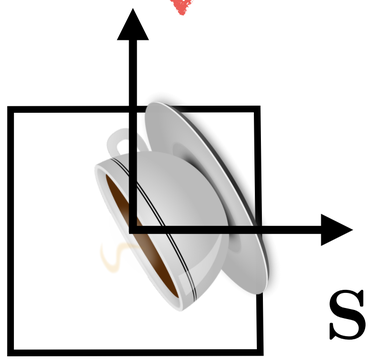
University of Victoria | Computer Science

Orthographic Projection

camera space

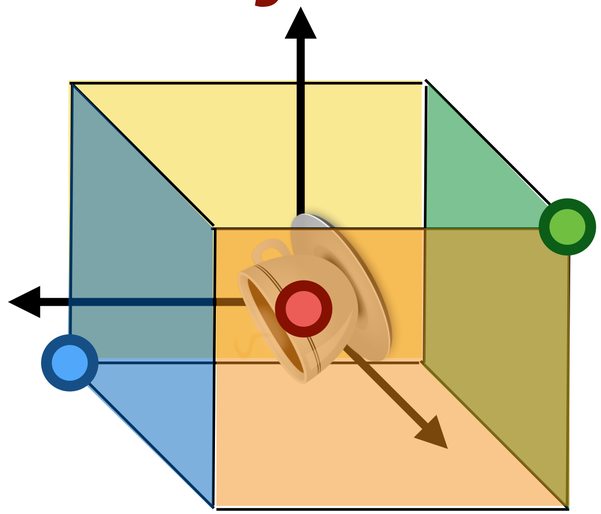


projection



$[-1,1]^3$

canonical view volume



Translate then scale: $\mathbf{M}_{orth} = \mathbf{S}\,\mathbf{T}(-\mathbf{c})$

$$\mathbf{S}\,\mathbf{T}(-\mathbf{c}) = \begin{bmatrix} s_x & 0 & 0 & -s_x c_x \\ 0 & s_y & 0 & -s_y c_y \\ 0 & 0 & s_z & -s_z c_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

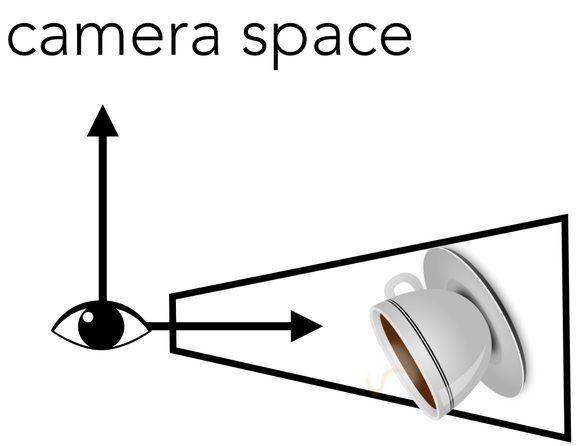
$$-s_x c_x = -\frac{2}{r-l} \cdot \frac{l+r}{2} = -\frac{r+l}{r-l}$$

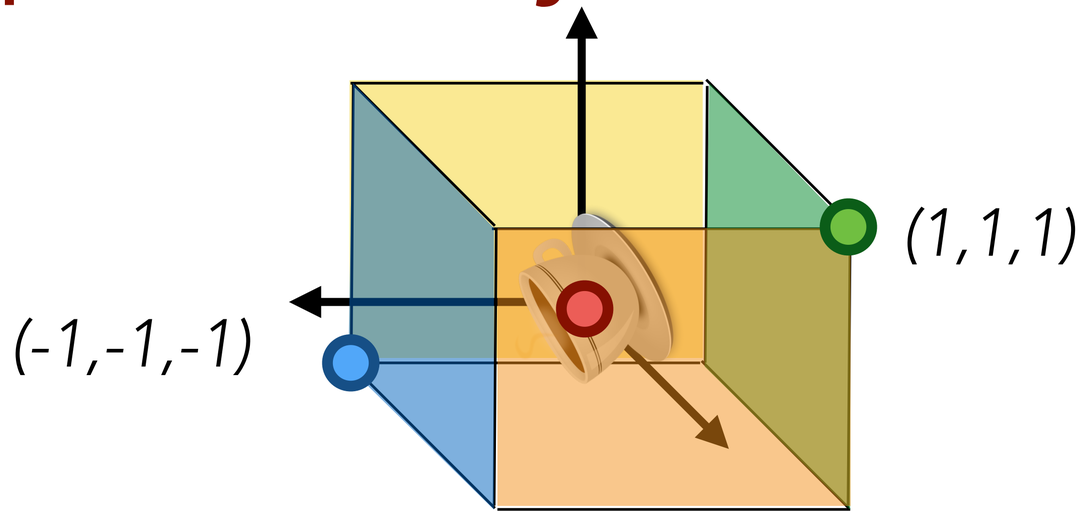
$$-s_y c_y = -\frac{2}{t-b} \cdot \frac{b+t}{2} = -\frac{t+b}{t-b}$$

$$-s_z c_z = -\frac{2}{n-f} \cdot \frac{n+f}{2} = -\frac{n+f}{n-f}$$

University of Victoria | Computer Science

Orthographic Projection



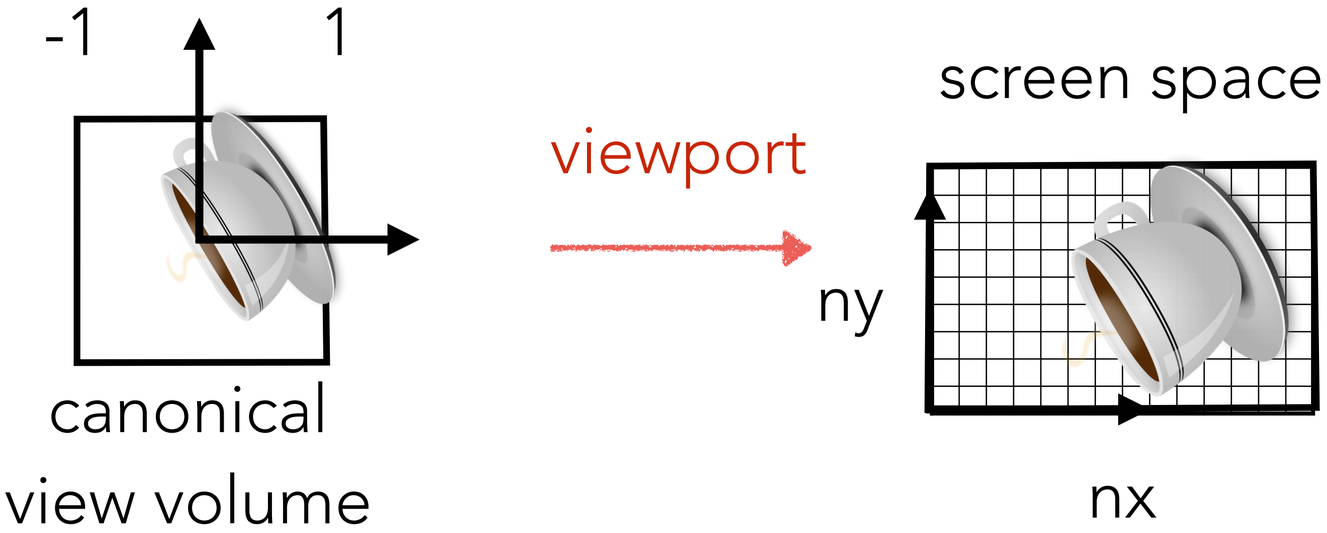


Translate then scale: $\mathbf{M}_{orth} = \mathbf{S}\,\mathbf{T}(-\mathbf{c})$

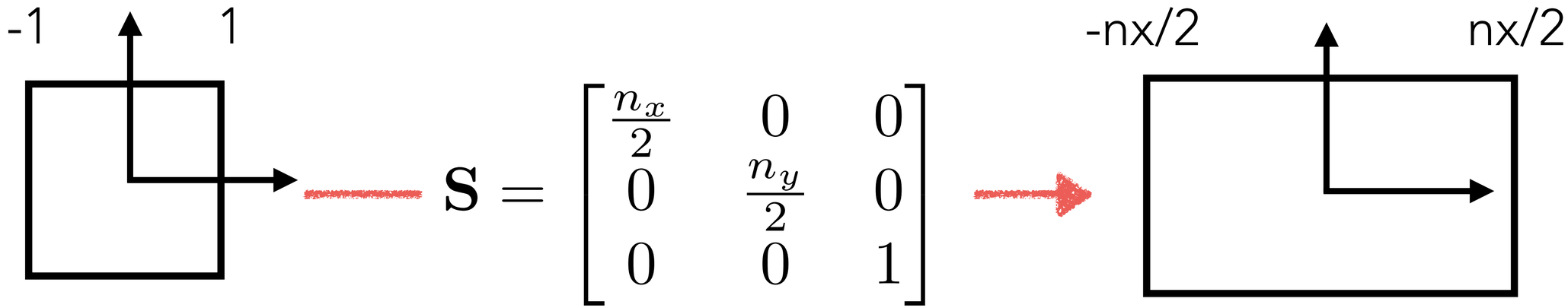
$$\mathbf{M}_{orth} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

University of Victoria | Computer Science

Viewport transformation

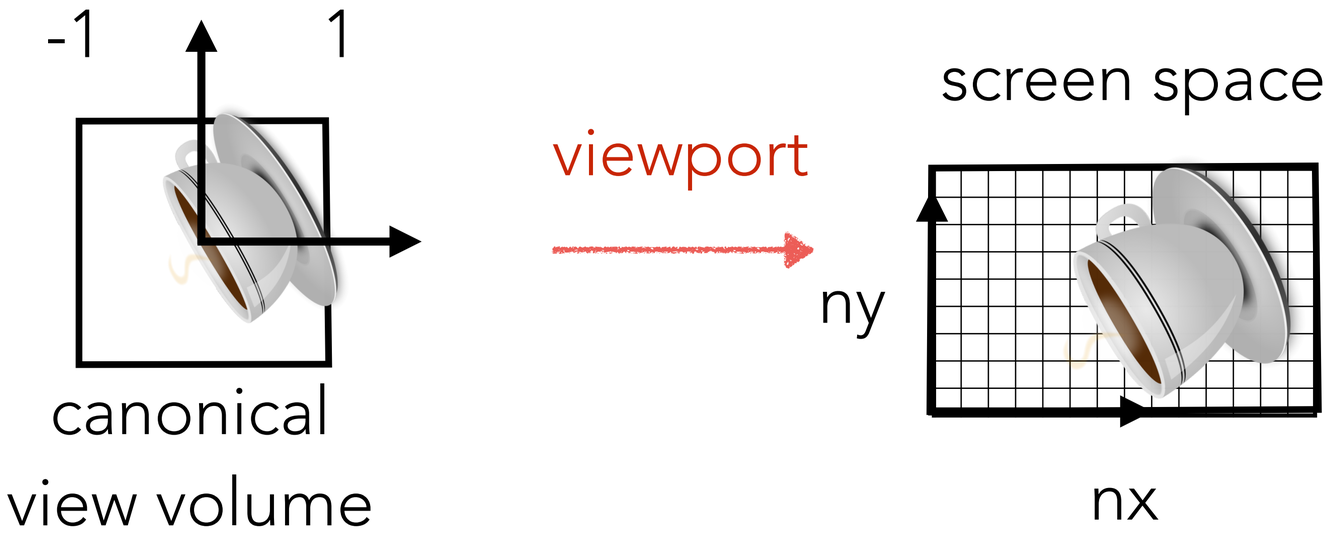


Scale then translate:

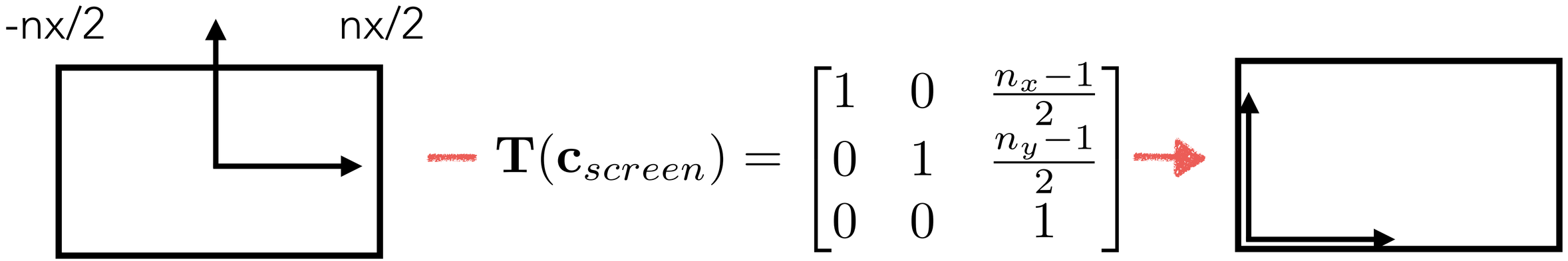


University of Victoria | Computer Science

Viewport transformation



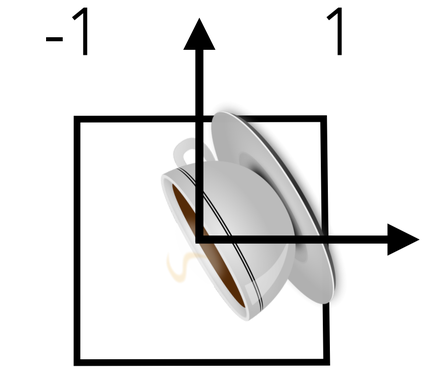
Scale then translate:



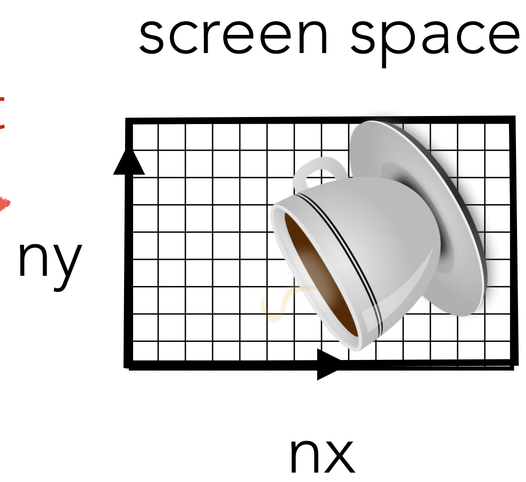
$$\mathbf{T}(\mathbf{c}_{screen}) = \begin{bmatrix} 1 & 0 & \frac{n_x-1}{2} \\ 0 & 1 & \frac{n_y-1}{2} \\ 0 & 0 & 1 \end{bmatrix}$$

University of Victoria | Computer Science

Viewport transformation

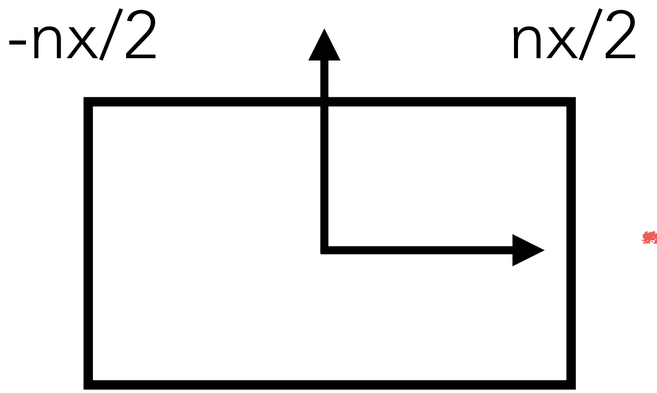


viewport

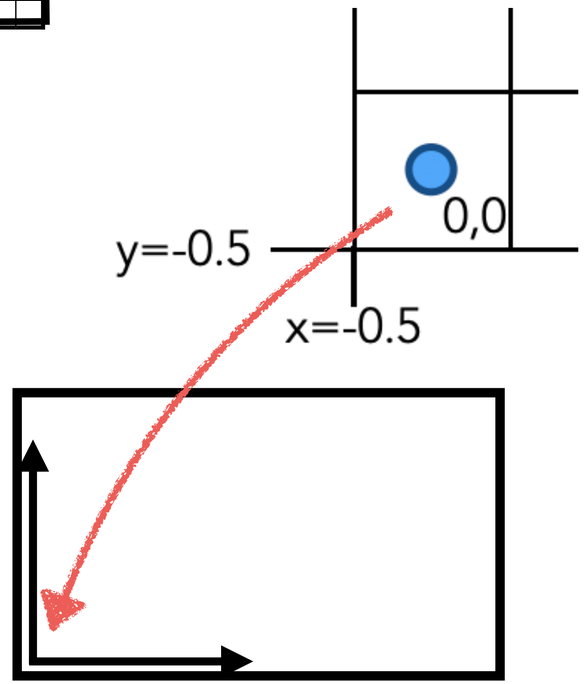


Reminder: origin is at left down pixel location

Scale then translate:

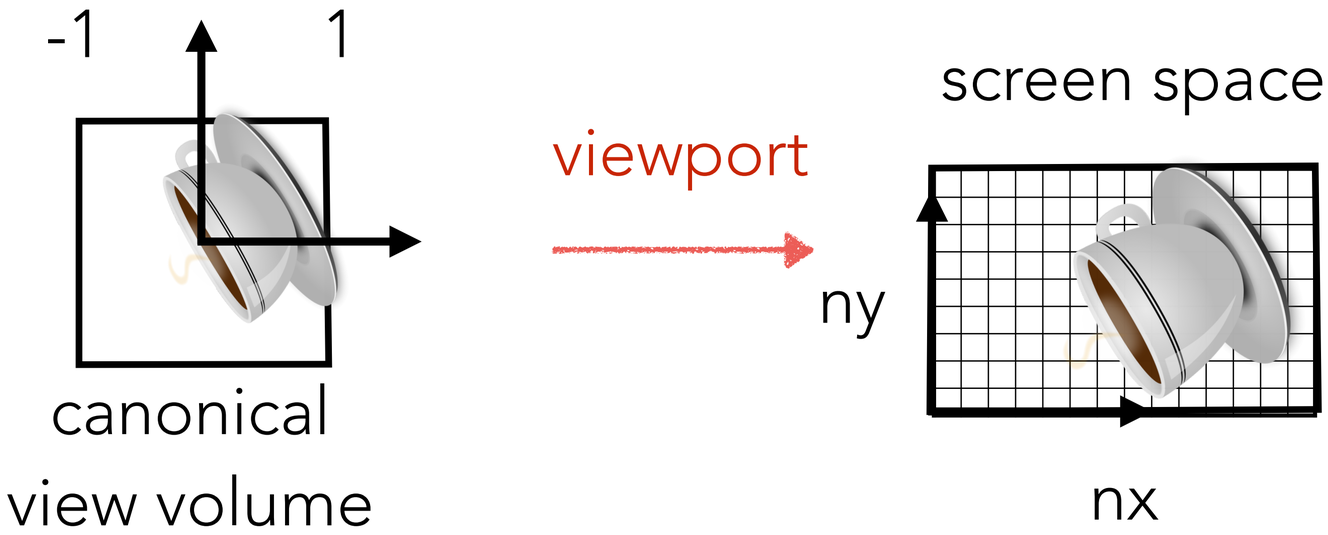


$$\mathbf{T}(\mathbf{c}_{screen}) = \begin{bmatrix} 1 & 0 & \frac{n_x - 1}{2} \\ 0 & 1 & \frac{n_y - 1}{2} \\ 0 & 0 & 1 \end{bmatrix}$$



University of Victoria | Computer Science

Viewport transformation

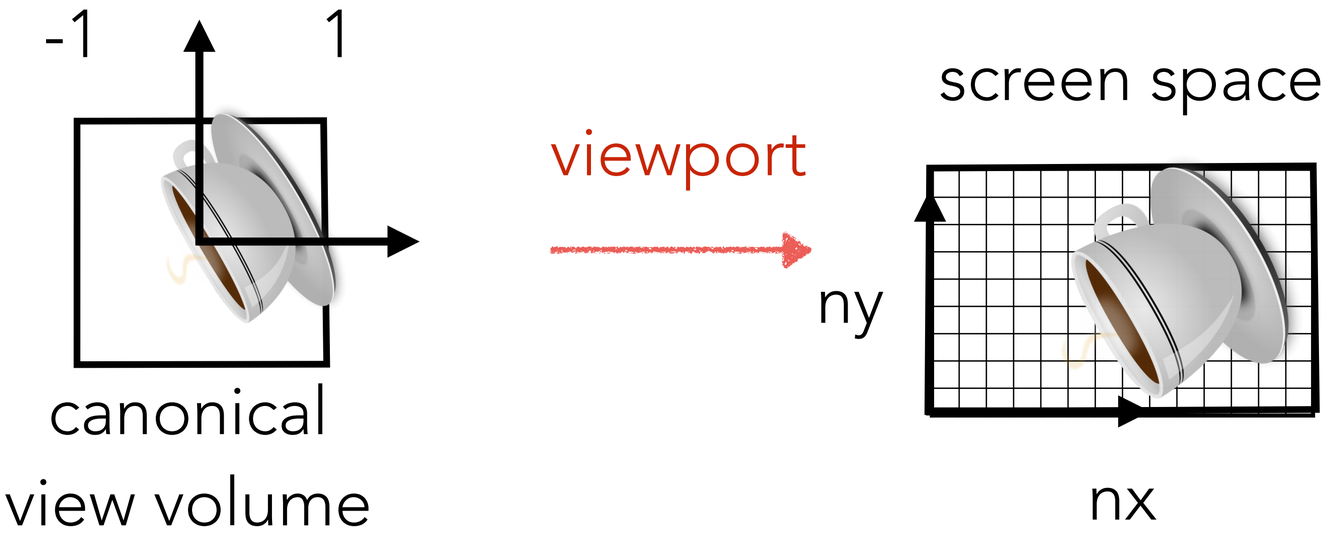


Scale then translate:

$$\mathbf{M}_{vp} = \mathbf{T}(\mathbf{c}_{screen})\,\mathbf{S} = \begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & c_x \\ 0 & s_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} nx/2 & 0 & \frac{n_x-1}{2} \\ 0 & ny/2 & \frac{n_y-1}{2} \\ 0 & 0 & 1 \end{bmatrix}$$

University of Victoria | Computer Science

Viewport transformation



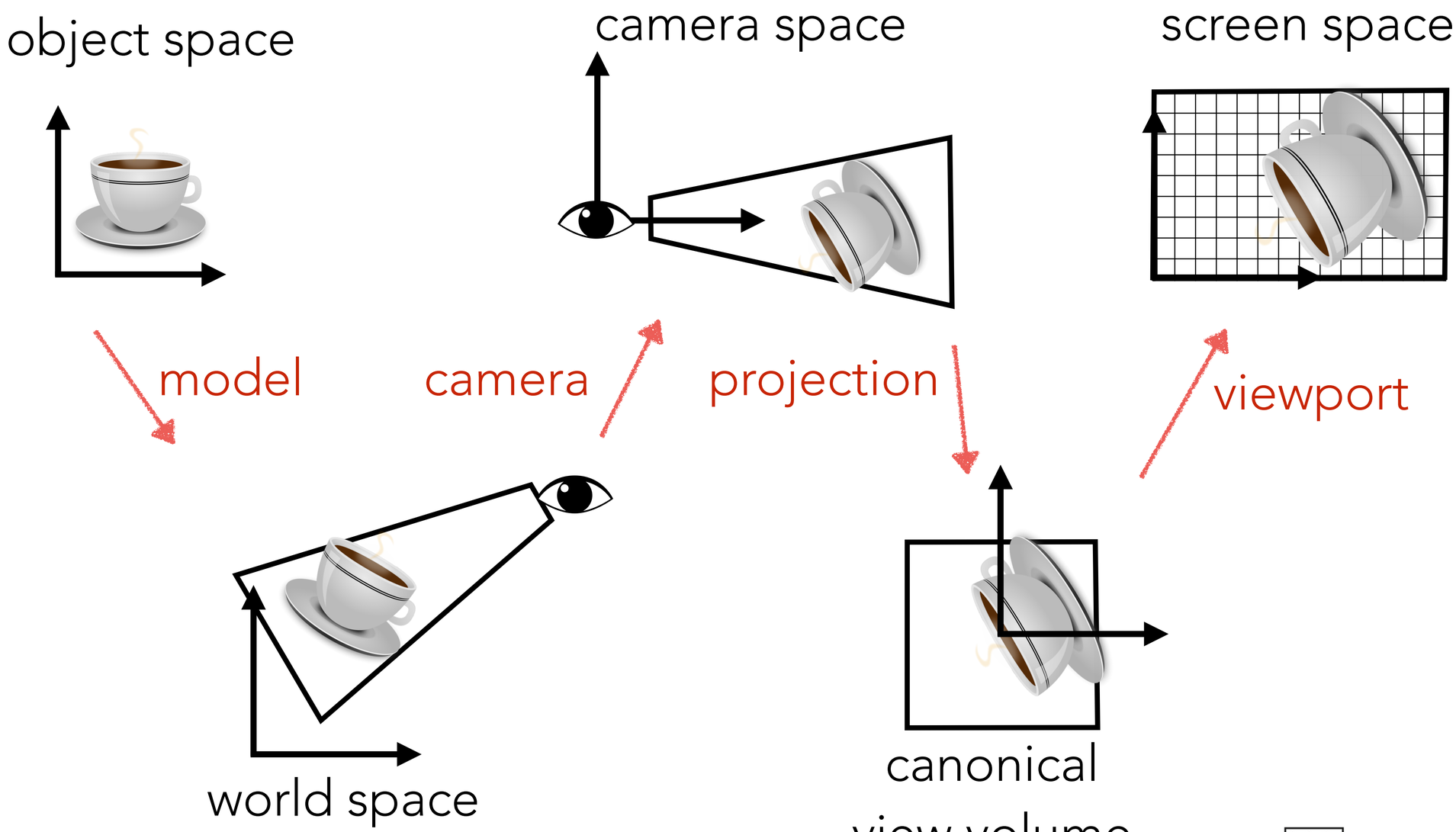
For 3D:

$$\begin{bmatrix} x_{screen} \\ y_{screen} \\ z_{canonical} \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{n_x}{2} & 0 & 0 & \frac{n_x-1}{2} \\ 0 & \frac{n_y}{2} & 0 & \frac{n_y-1}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{canonical} \\ y_{canonical} \\ z_{canonical} \\ 1 \end{bmatrix}$$

***z* is kept as it is for the depth test**

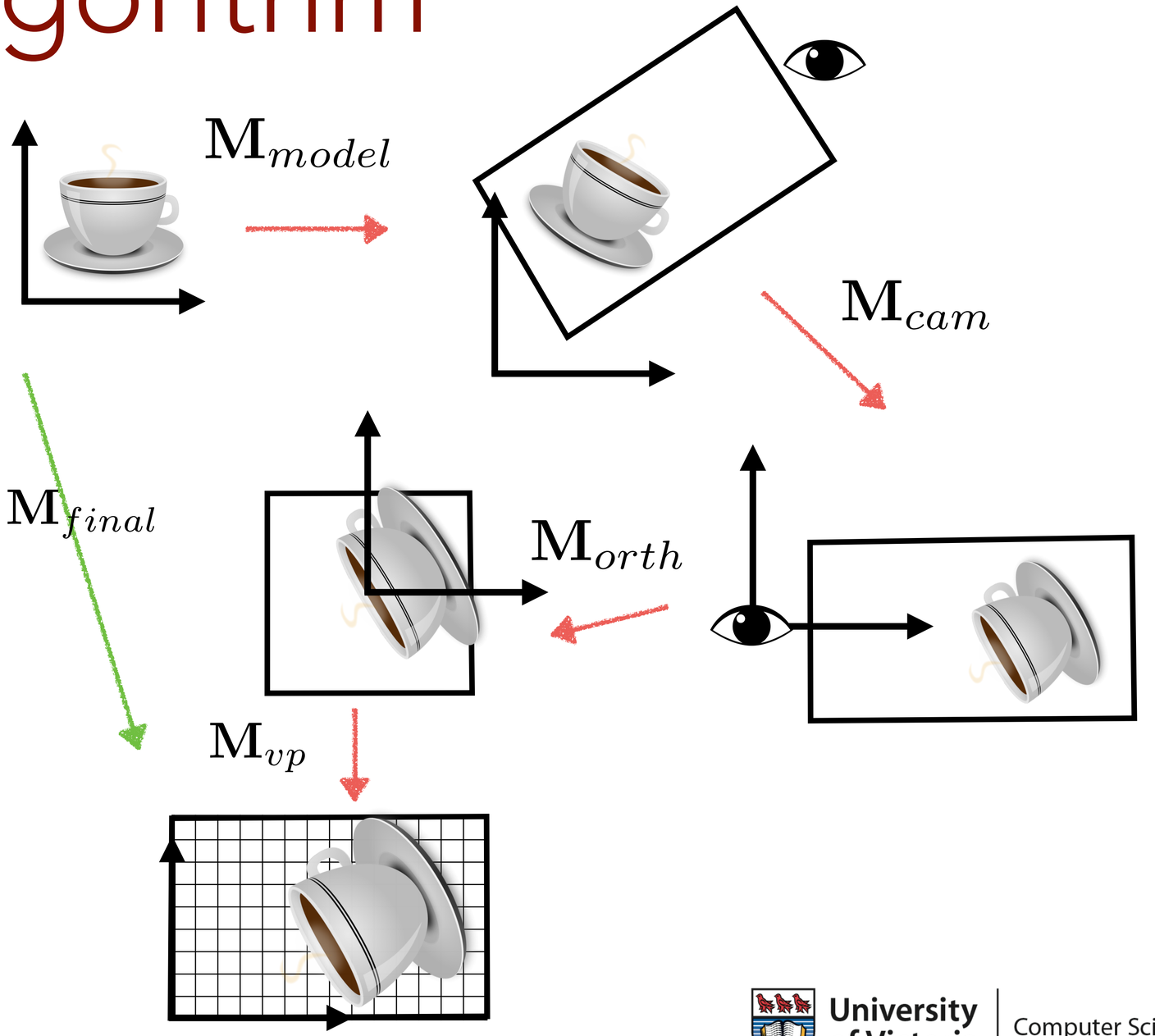
University of Victoria | Computer Science

Viewing Transformation



Algorithm

- Construct Viewport Matrix $\mathbf{M}_{vp}$
- Construct Projection Matrix $\mathbf{M}_{orth}$
- Construct Camera Matrix $\mathbf{M}_{cam}$
- $\mathbf{M} = \mathbf{M}_{vp}\mathbf{M}_{orth}\mathbf{M}_{cam}$
- For each model
  - Construct Model Matrix $\mathbf{M}_{model}$
  - $\mathbf{M}_{final} = \mathbf{M}\mathbf{M}_{model}$
  - For every point **p** in each primitive of the model
    - $\mathbf{p}_{final} = \mathbf{M}_{final}\mathbf{p}$
  - Rasterize the model



University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

Batch transform n points: $\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} \quad (2 \times n)$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

Batch transform n points: $\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} \quad (2 \times n)$

Applying transformations one by one: $\mathbf{L}_3\big(\mathbf{L}_2(\mathbf{L}_1\mathbf{X})\big) : \quad 4n + 4n + 4n = 12n$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

Batch transform n points: $\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} \quad (2 \times n)$

Applying transformations one by one: $\mathbf{L}_3\big(\mathbf{L}_2(\mathbf{L}_1\mathbf{X})\big)$: $\quad 4n + 4n + 4n = 12n$

Combine transformations first: $\underbrace{(\mathbf{L}_3\mathbf{L}_2\mathbf{L}_1)}_{\mathbf{M}:\ 8+8\ =\ 16}\ \mathbf{X}$: $\quad 16 + 4n$

University of Victoria | Computer Science

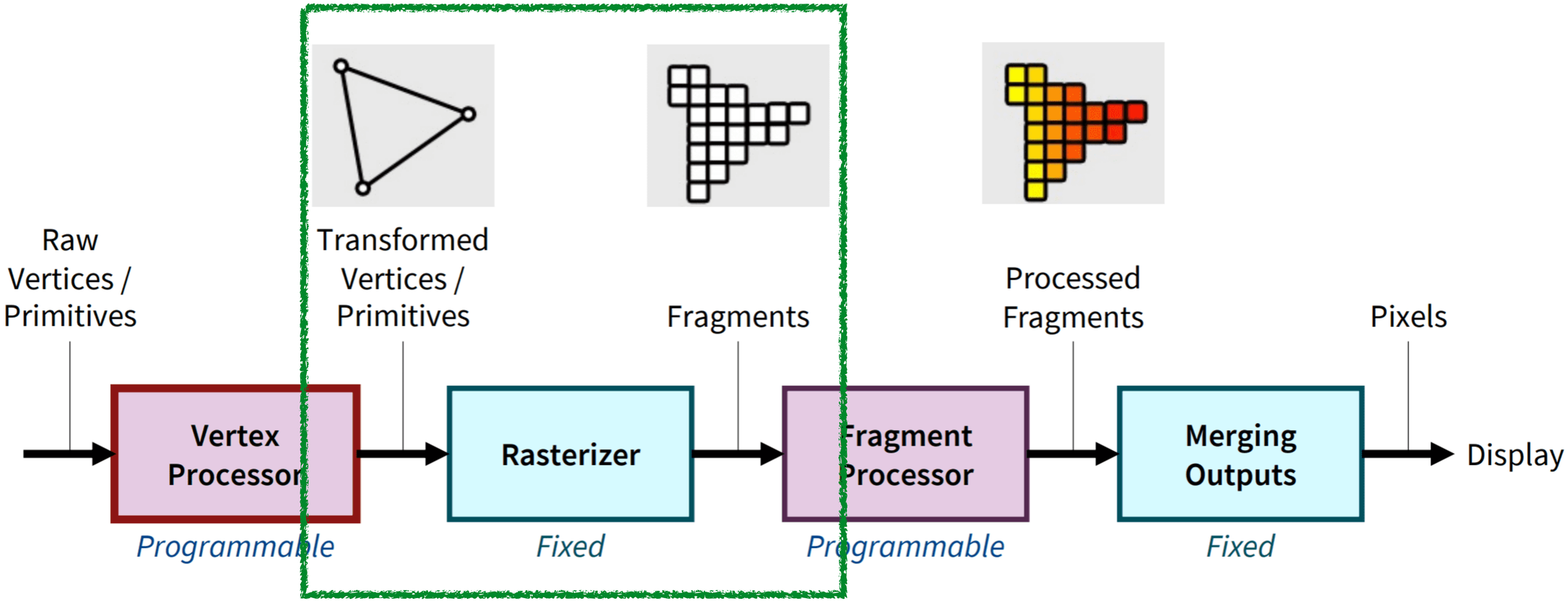
References

Fundamentals of Computer Graphics, Fourth Edition
4th Edition **by Steve Marschner, Peter Shirley**

Chapter 7

Rasterization

Ray tracing → Rasterization



Youngdo Lee; https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

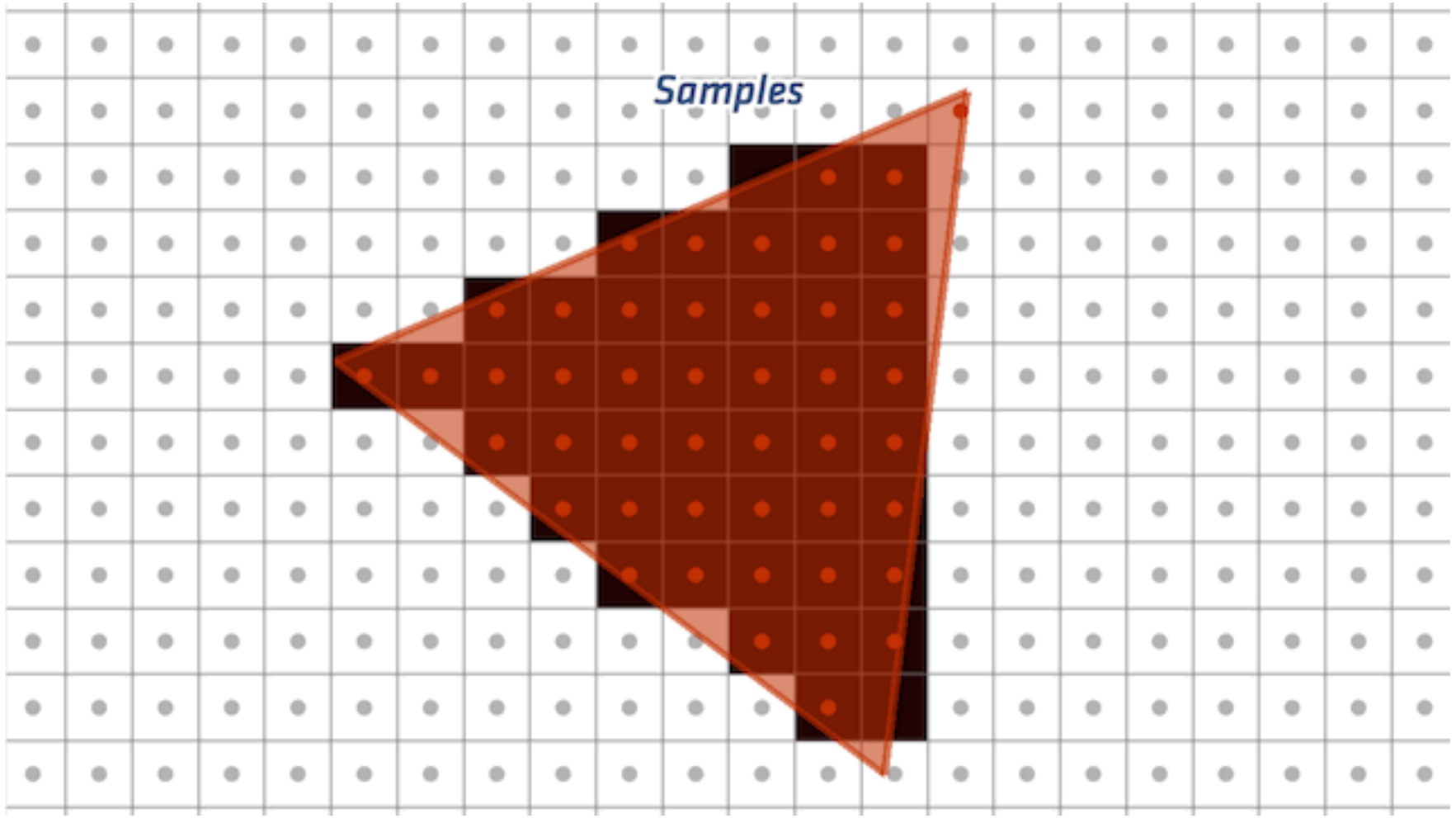


Image Copyright: Andrea Tagliasacchi

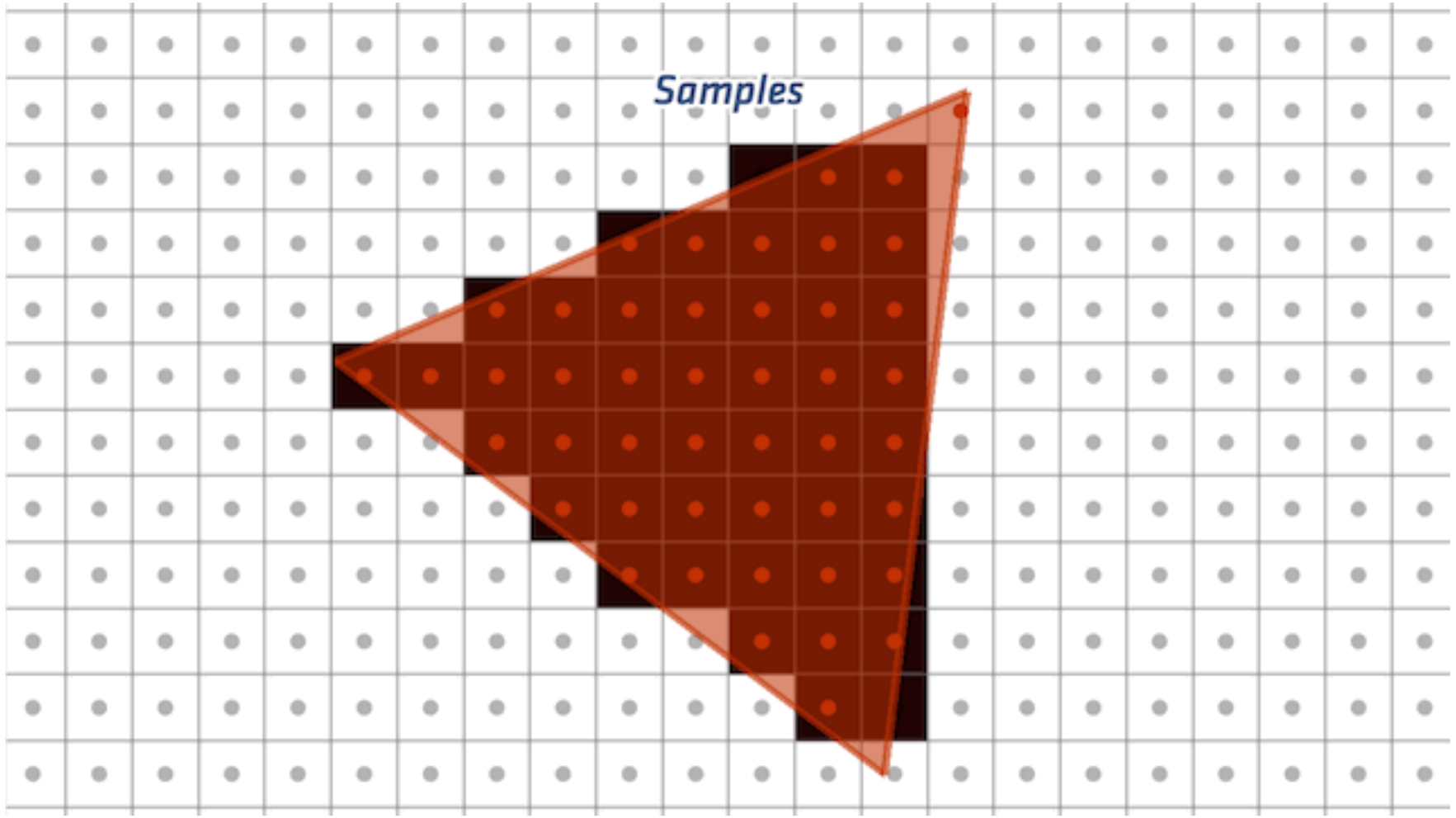
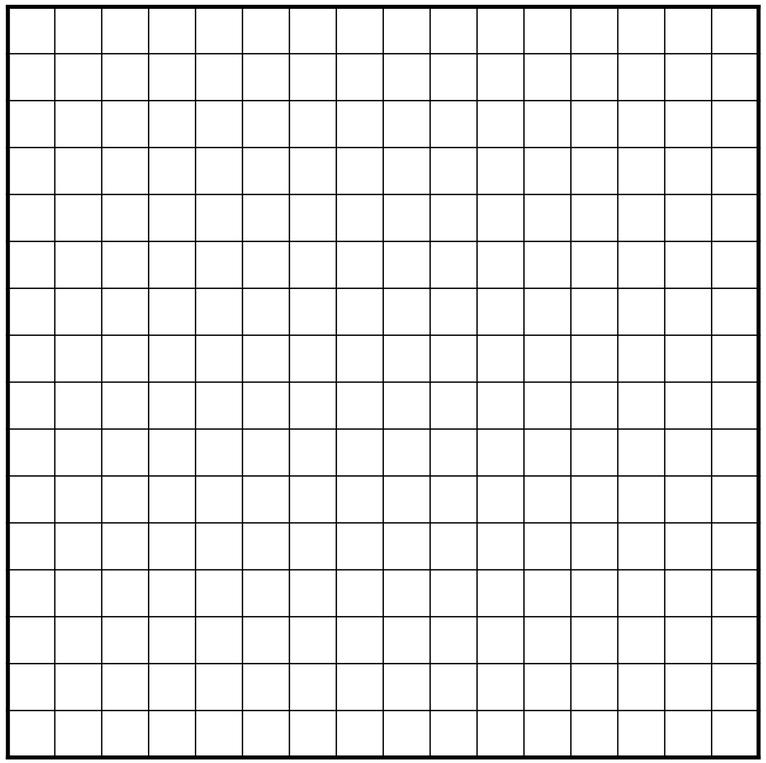


Image Copyright: Andrea Tagliasacchi

2D Canvas



pixel grid

Image Copyright: Andrea Tagliasacchi

2D Canvas

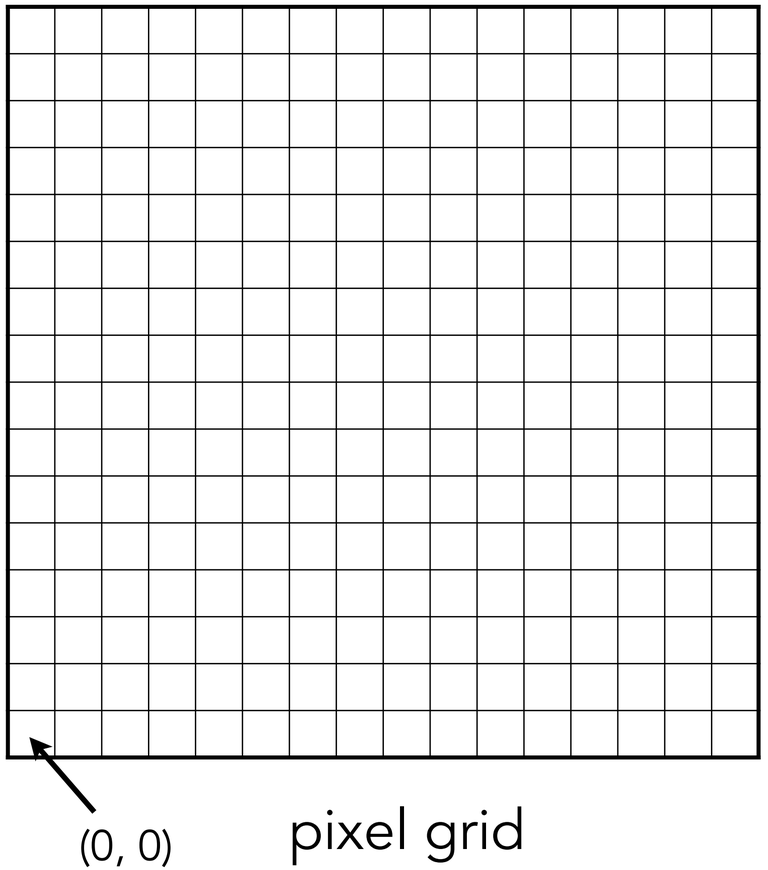


Image Copyright: Andrea Tagliasacchi

2D Canvas

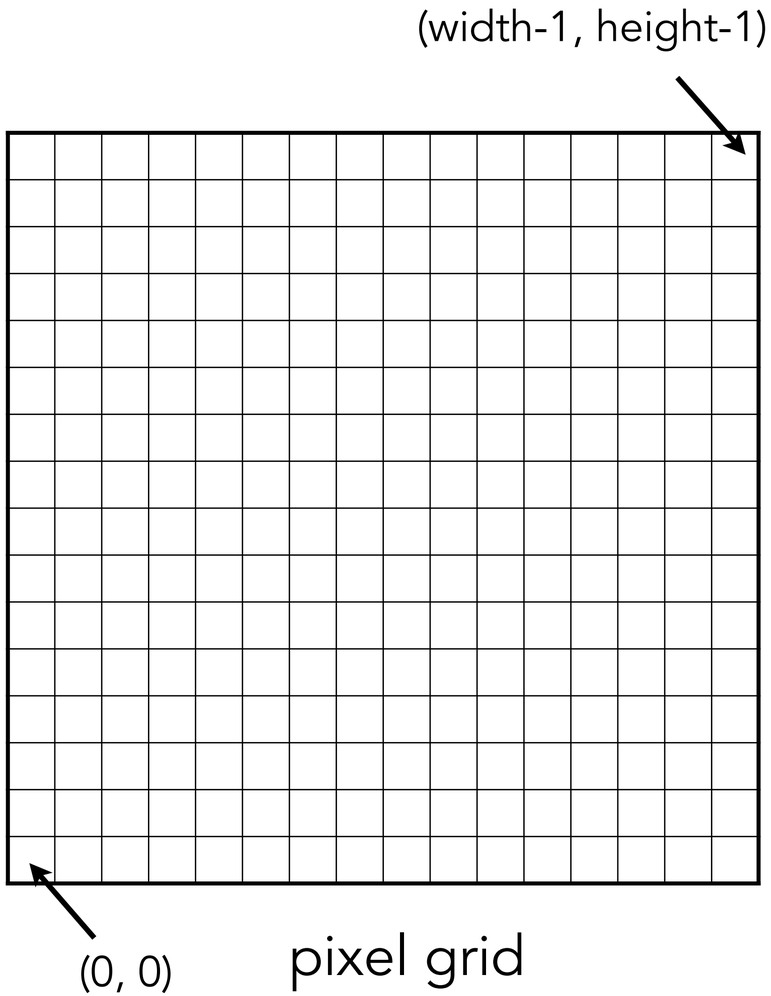


Image Copyright: Andrea Tagliasacchi

2D Canvas

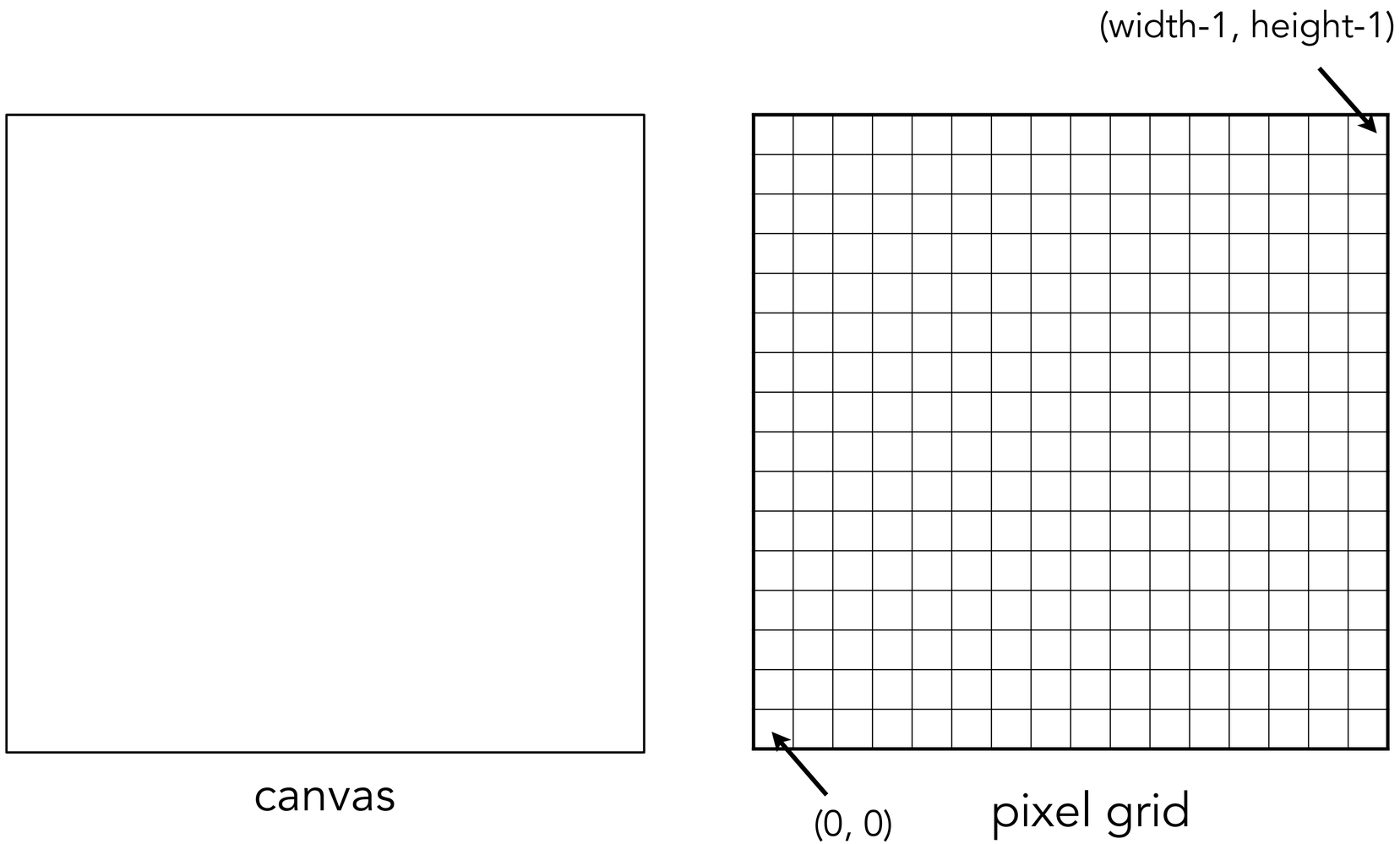


Image Copyright: Andrea Tagliasacchi

2D Canvas

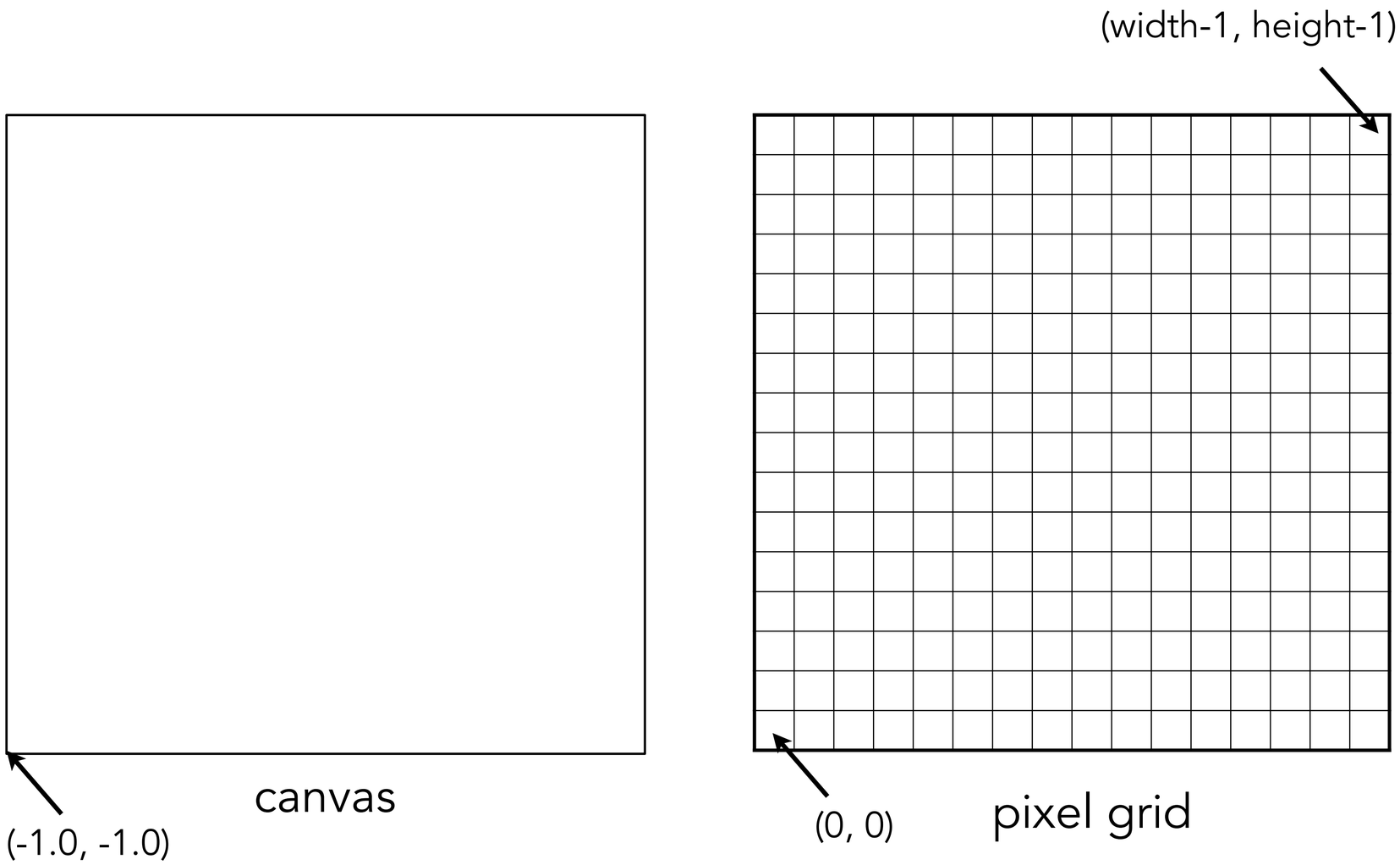


Image Copyright: Andrea Tagliasacchi

2D Canvas

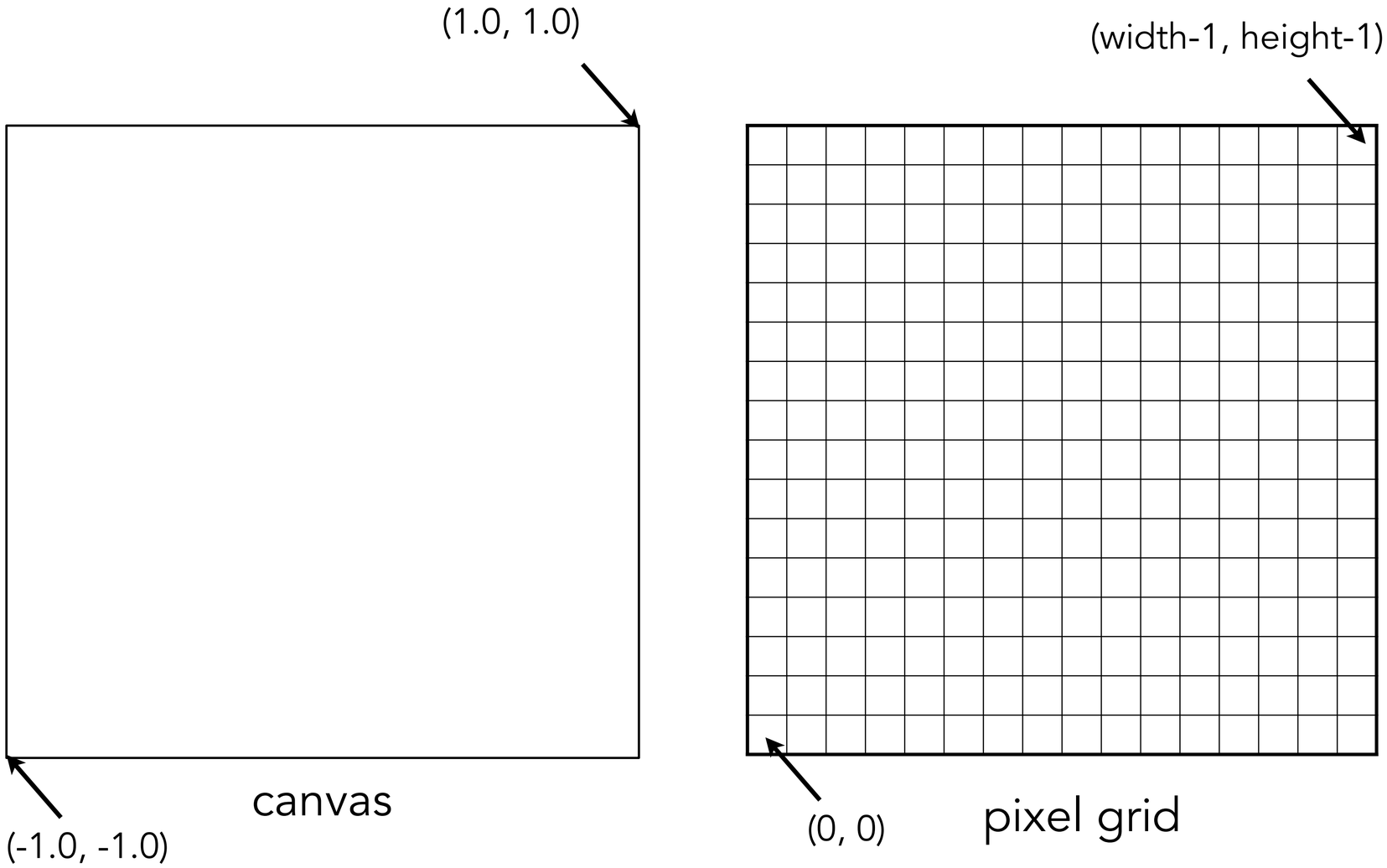


Image Copyright: Andrea Tagliasacchi

2D Canvas

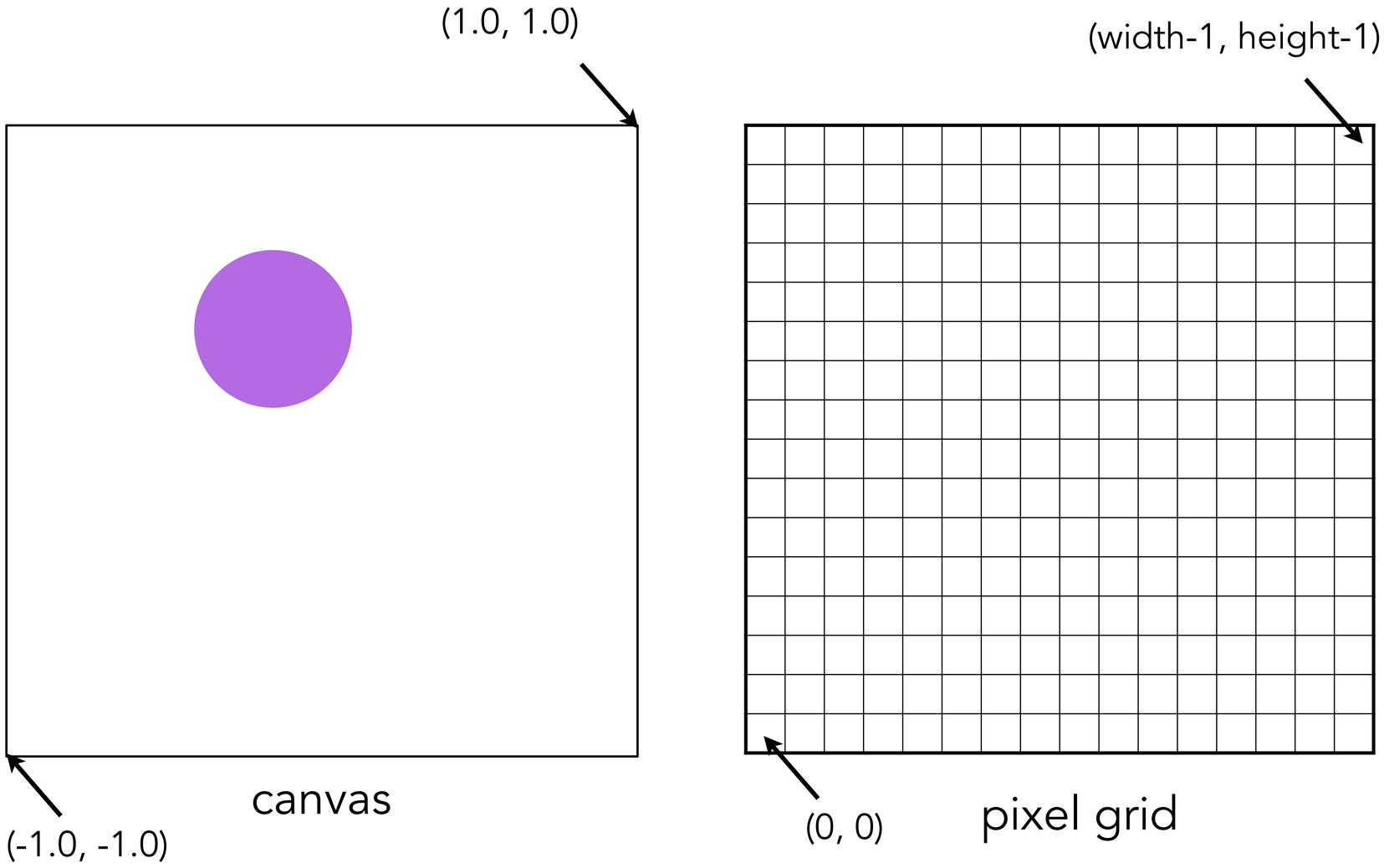


Image Copyright: Andrea Tagliasacchi

2D Canvas

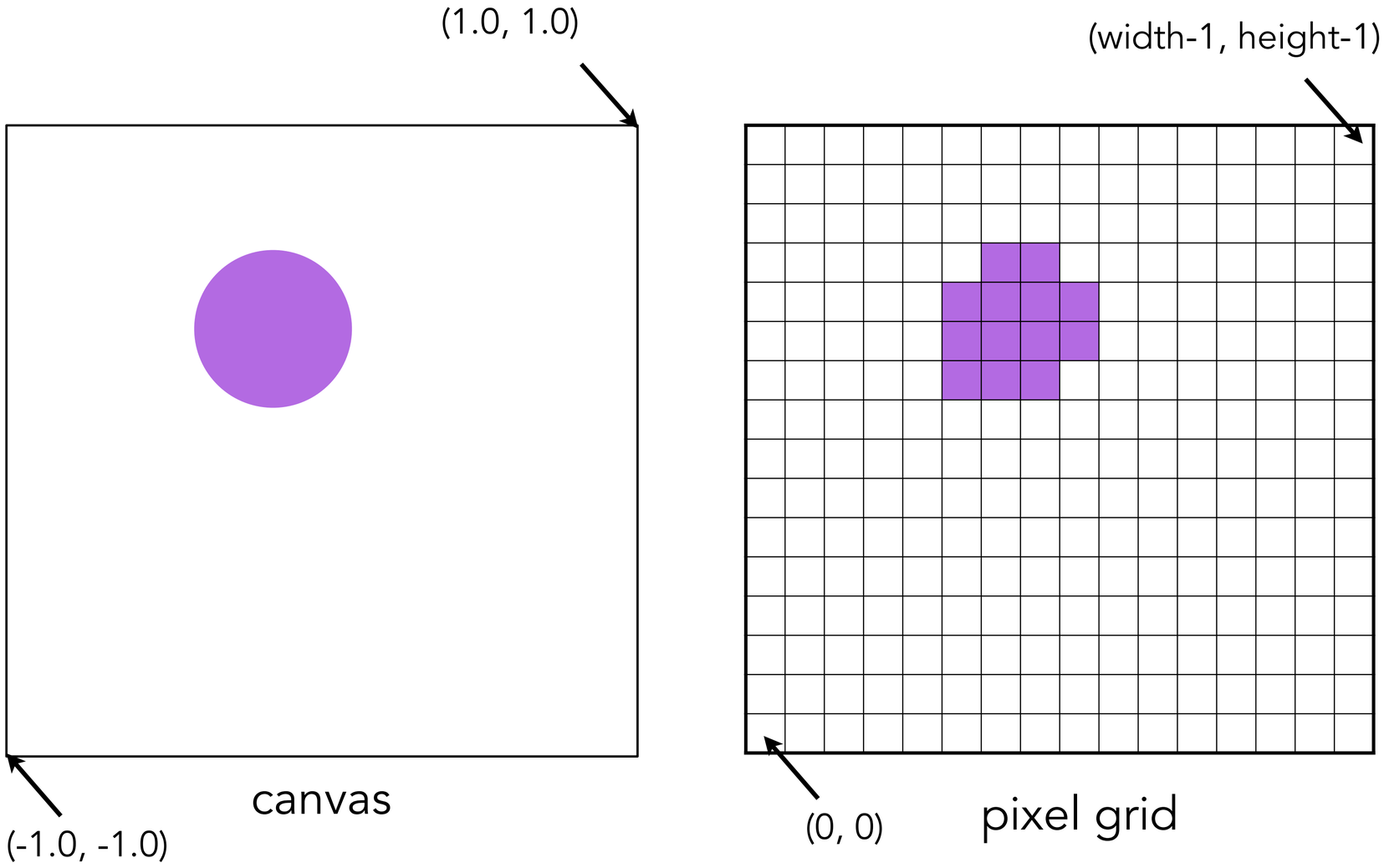


Image Copyright: Andrea Tagliasacchi

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve
  - $F(x,y) > 0 \rightarrow$ outside curve

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve
  - $F(x,y) > 0 \rightarrow$ outside curve
- Example: Circle with center (c_x, c_y) and radius r

University of Victoria | Computer Science

Implicit Geometry Representation

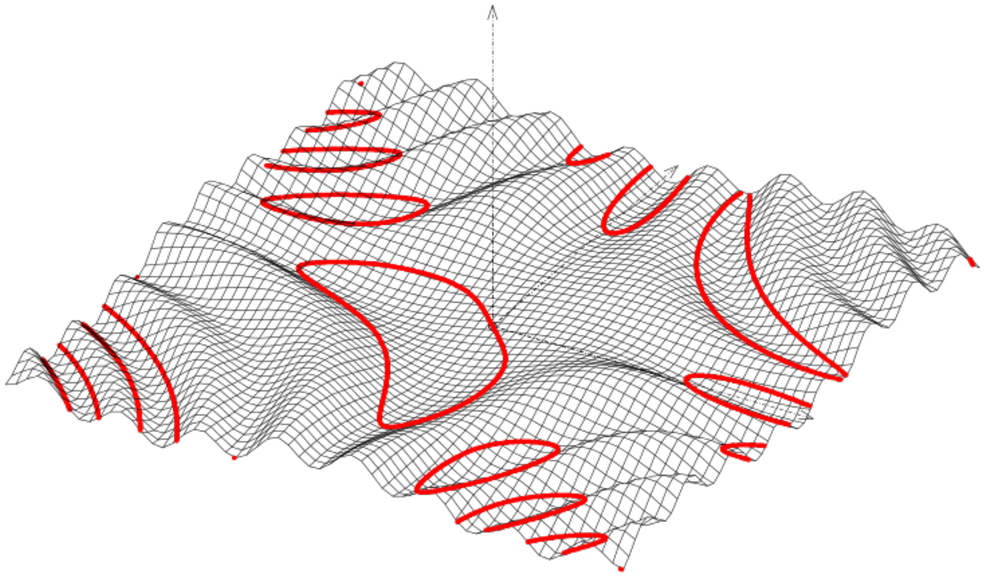
- Define a curve as zero set of 2D implicit function
  - *F(x,y) = 0* → on curve
  - *F(x,y) < 0* → inside curve
  - *F(x,y) > 0* → outside curve
- Example: Circle with center (c_x, c_y) and radius r

$$F(x, y) = (x - c_x)^2 + (y - c_y)^2 - r^2$$

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve
  - $F(x,y) > 0 \rightarrow$ outside curve



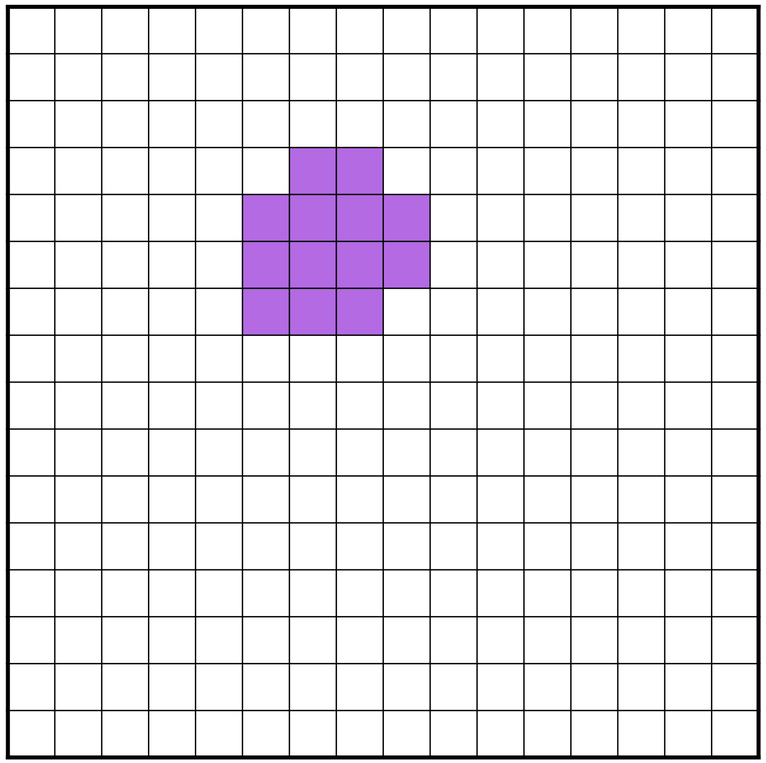
By Ag2gaeh - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=45004240

Implicit Rasterization

for all pixels (i,j)

(x,y) = map_to_canvas (i,j)

if F(x,y) < 0



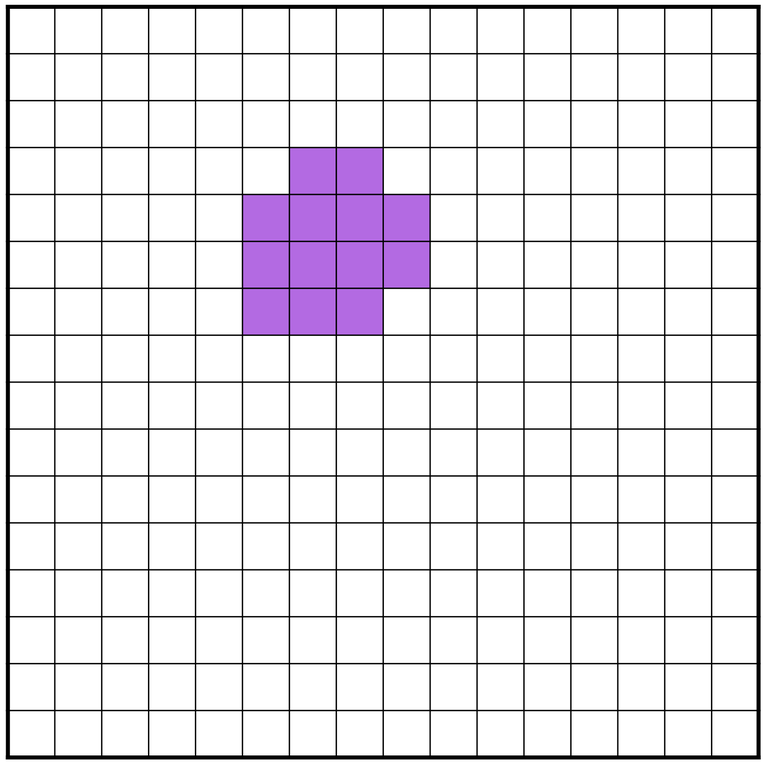
Implicit Rasterization

```
for all pixels (i,j)

(x,y) = map_to_canvas (i,j)

 if F(x,y) < 0

       set_pixel (i,j, color)
```

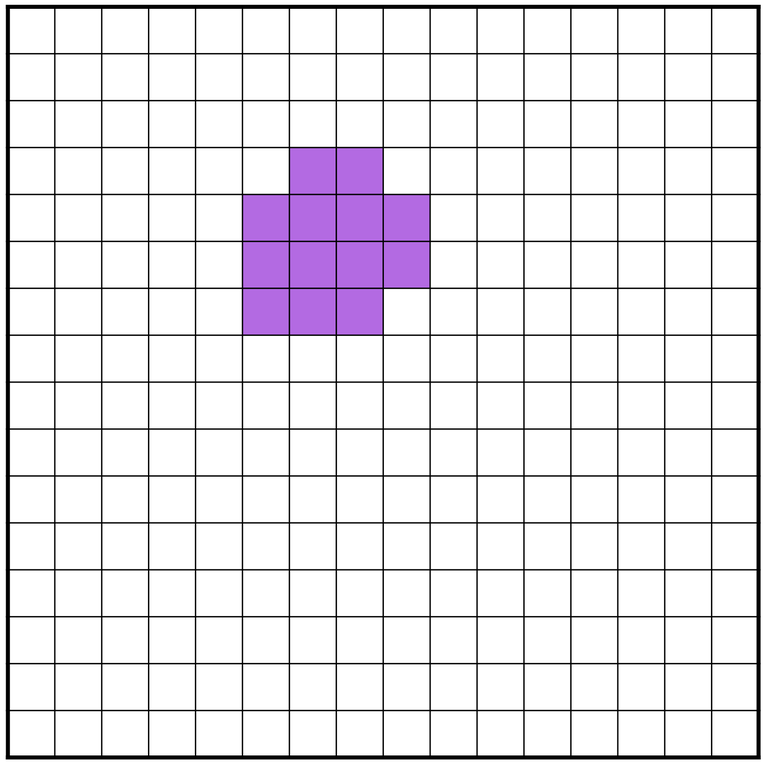


Implicit Rasterization

for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

        for x in xmin .. xmax:



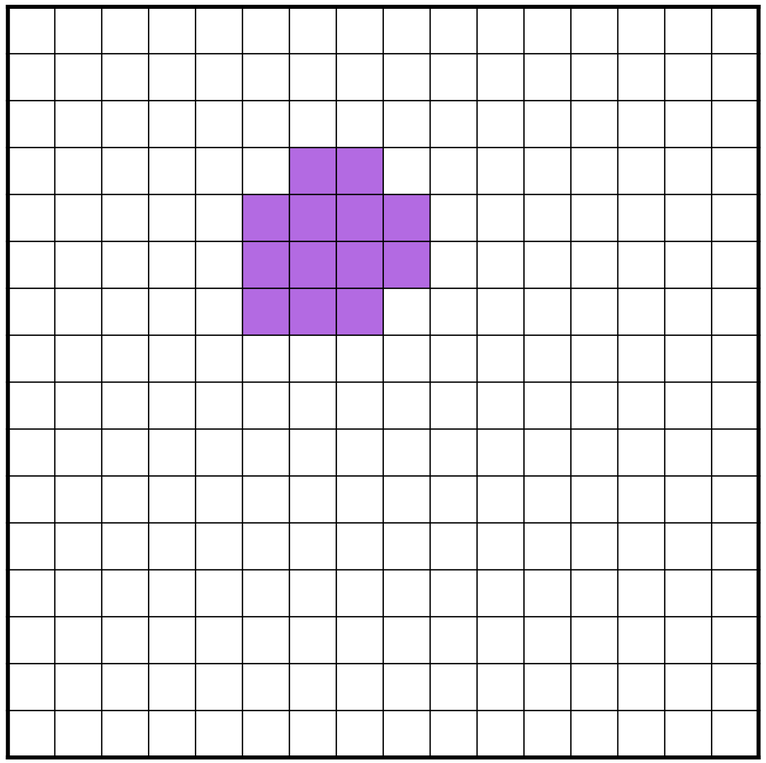
Implicit Rasterization

for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

        for x in xmin .. xmax:

            p = (x, y)



University of Victoria | Computer Science

Implicit Rasterization

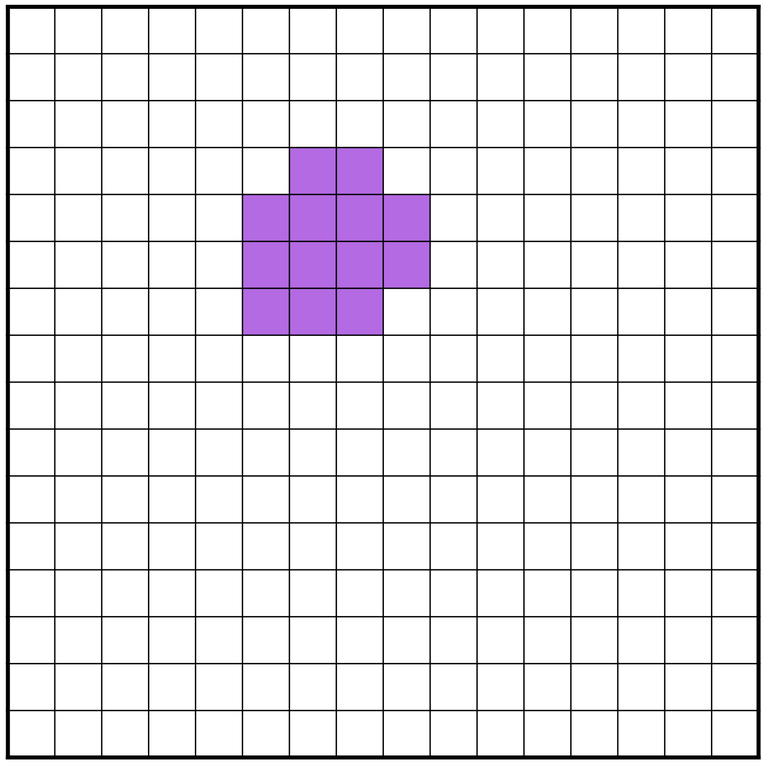
for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

        for x in xmin .. xmax:

            p = (x, y)

            if F(x,y) < 0



University of Victoria | Computer Science

Implicit Rasterization

```
for each triangle (v0, v1, v2) in screen space:

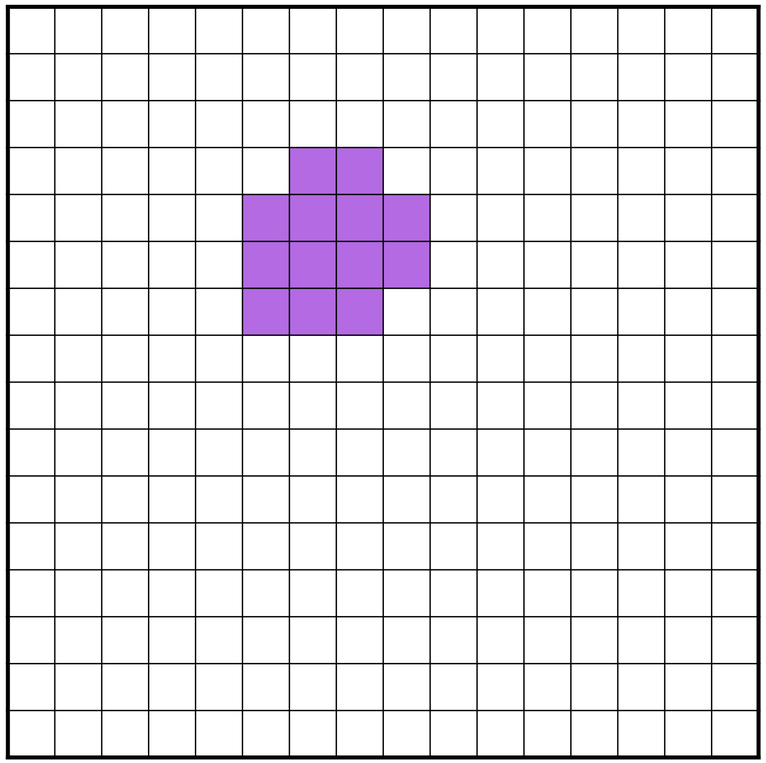
    for y in ymin .. ymax:

        for x in xmin .. xmax:

            p = (x, y)

            if F(x,y) < 0

                set_pixel (i,j, color)
```



University of Victoria | Computer Science

Implicit Rasterization

```
for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

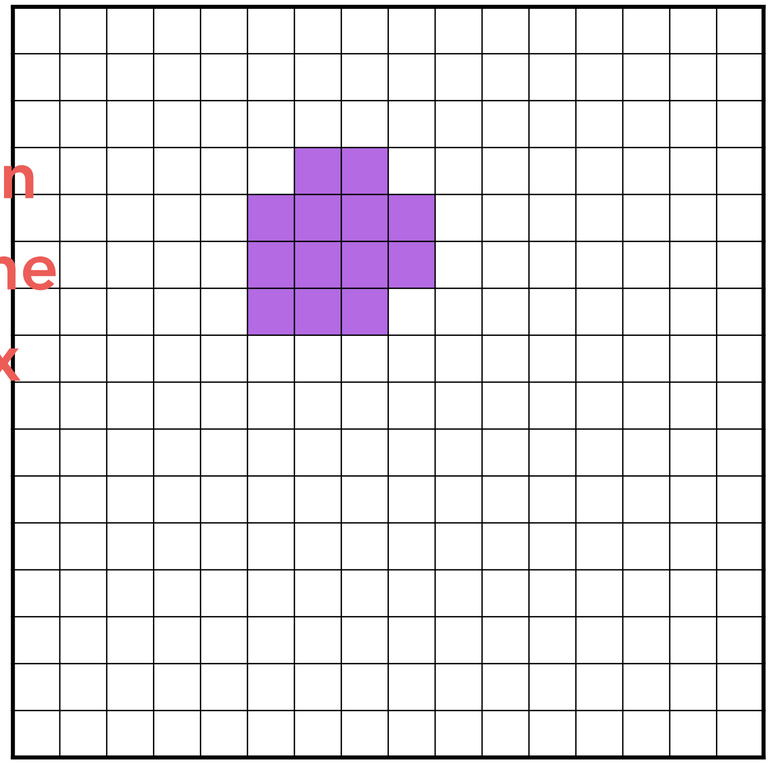
        for x in xmin .. xmax:

            p = (x, y)

            if F(x,y) < 0

                set_pixel (i,j, color)
```

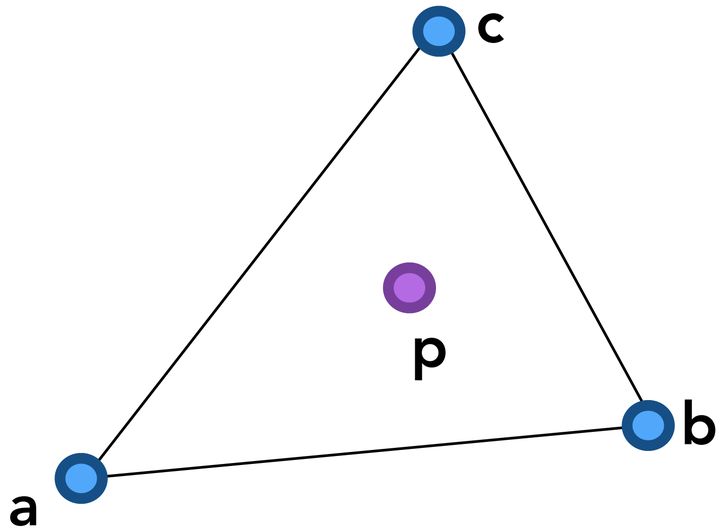
Only iterate on pixels within the bounding box



University of Victoria | Computer Science

Barycentric Interpolation

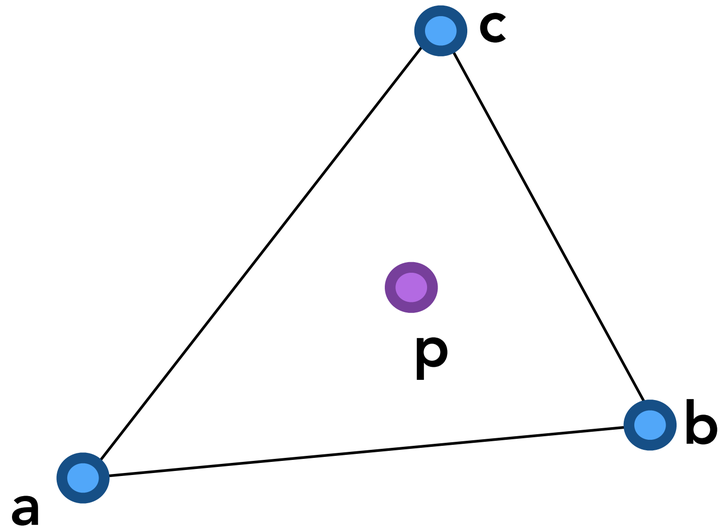
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$



University of Victoria | Computer Science

Barycentric Interpolation

- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**



$$\begin{bmatrix} \mathbf{a}_x & \mathbf{b}_x & \mathbf{c}_x \\ \mathbf{a}_y & \mathbf{b}_y & \mathbf{c}_y \\ 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = \begin{bmatrix} \mathbf{p}_x \\ \mathbf{p}_y \\ 1 \end{bmatrix}$$

University of Victoria | Computer Science

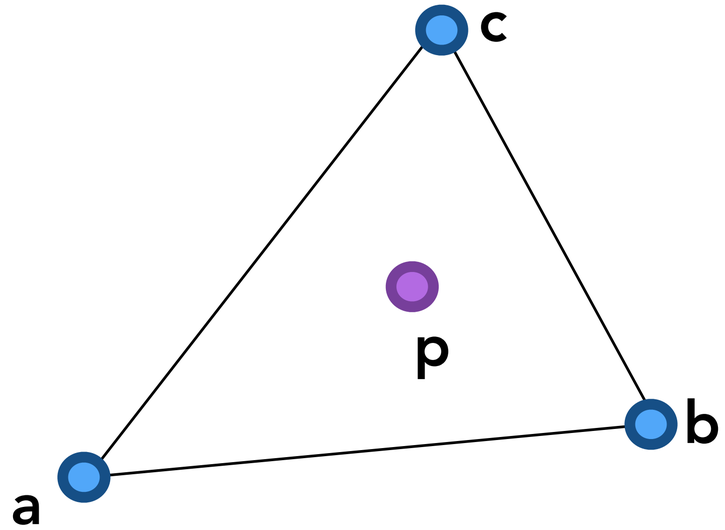
Barycentric Interpolation

- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas

$$\alpha(\mathbf{p}) = \frac{\text{area}(\mathbf{p}, \mathbf{b}, \mathbf{c})}{\text{area}(\mathbf{a}, \mathbf{b}, \mathbf{c})}$$

$$\beta(\mathbf{p}) = \frac{\text{area}(\mathbf{p}, \mathbf{c}, \mathbf{a})}{\text{area}(\mathbf{a}, \mathbf{b}, \mathbf{c})}$$

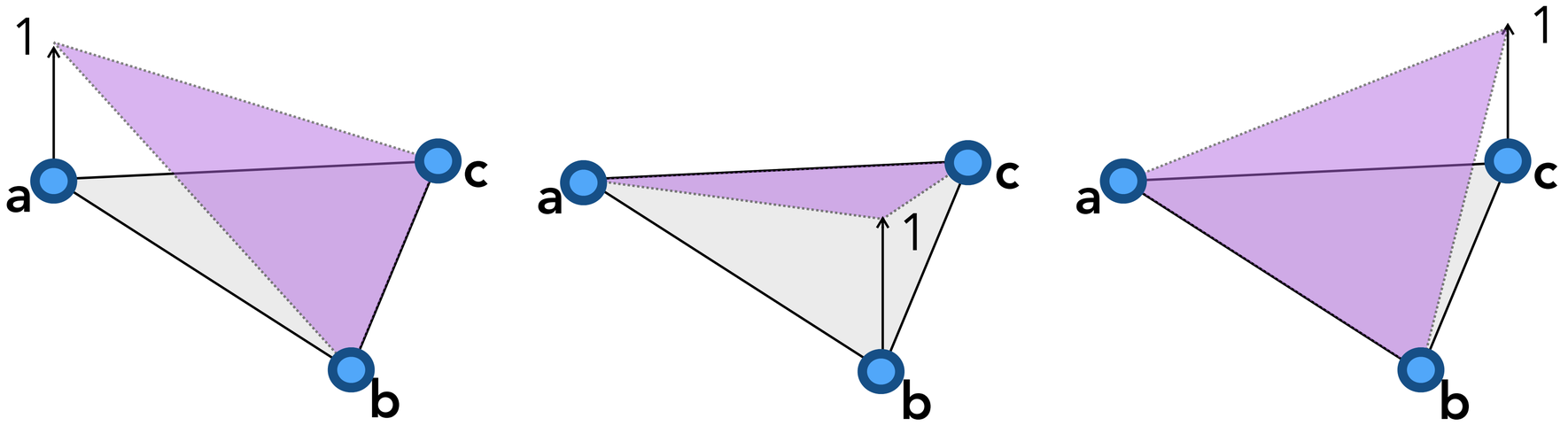
$$\gamma(\mathbf{p}) = \frac{\text{area}(\mathbf{p}, \mathbf{a}, \mathbf{b})}{\text{area}(\mathbf{a}, \mathbf{b}, \mathbf{c})}$$



University of Victoria | Computer Science

Barycentric Interpolation

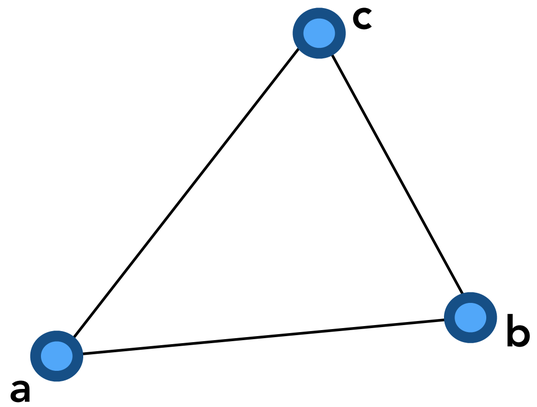
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions



University of Victoria | Computer Science

Barycentric Interpolation

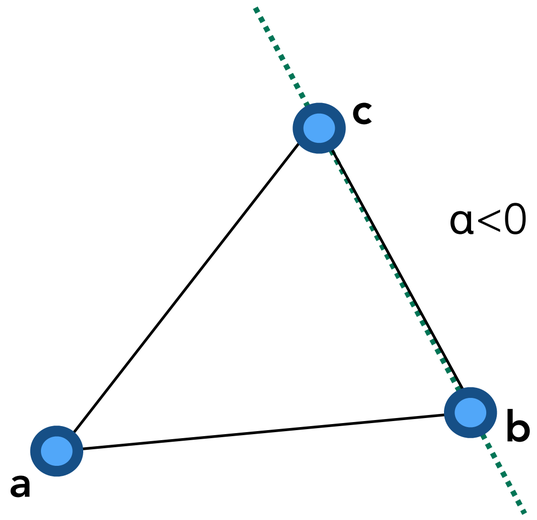
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

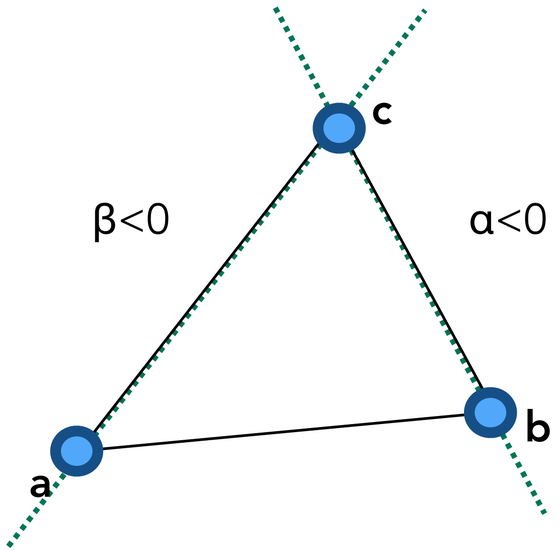
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

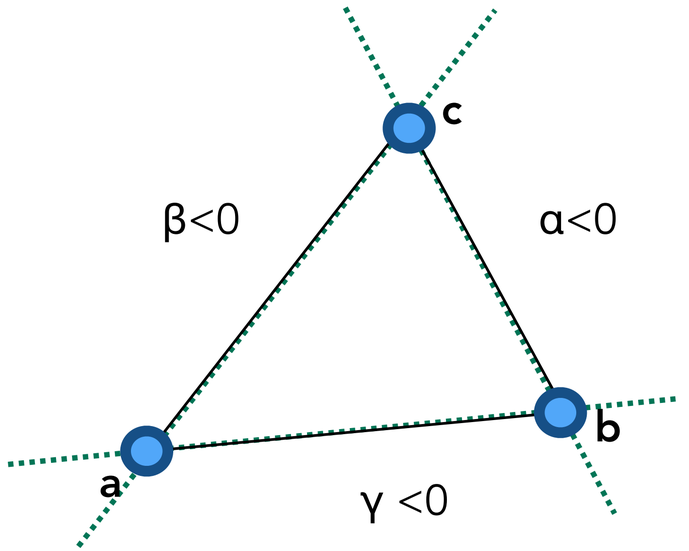
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

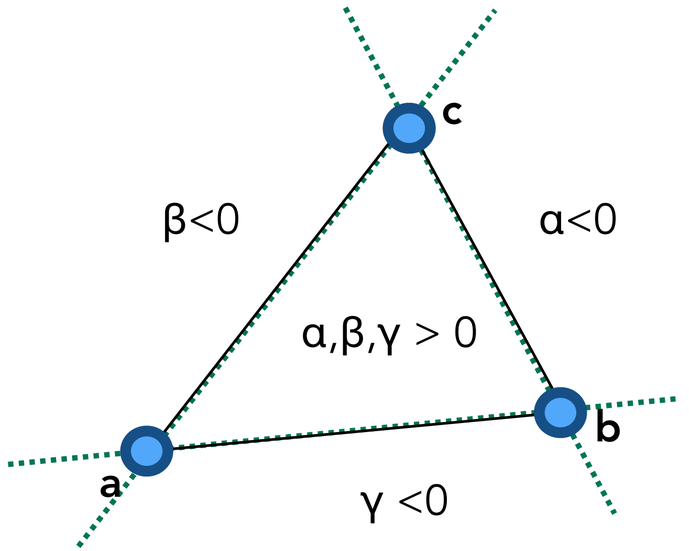
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

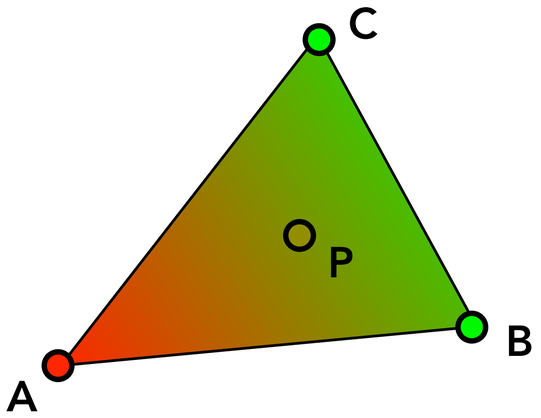
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

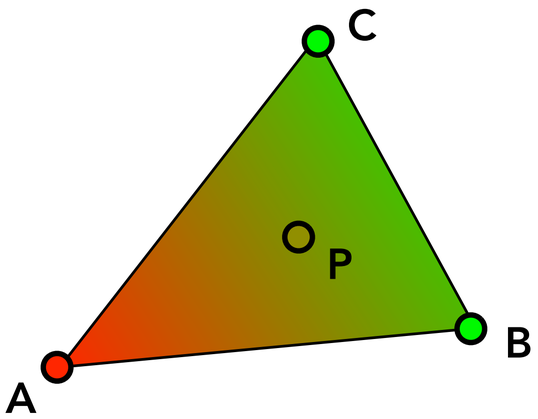
- Barycentric coordinates:
  - **p** = α**a** + β**b** + γ**c** with α + β + γ = 1
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - α(**p**), β(**p**), γ(**p**) are linear functions
  - Gives inside/outside information
  - Use barycentric coordinates to interpolate vertex normals (or other data, e.g. colors)



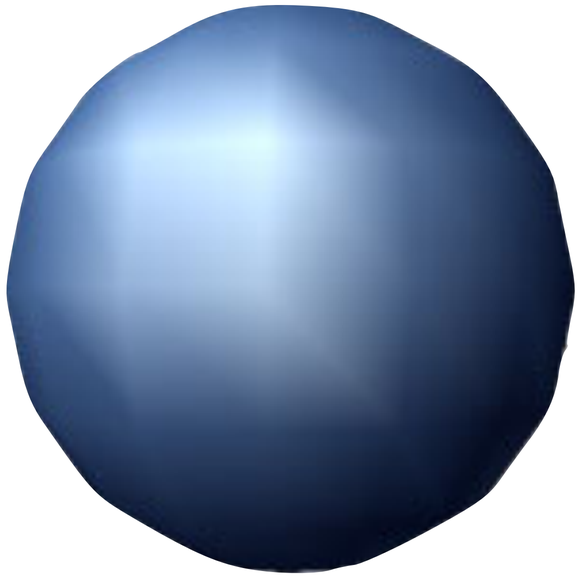
$$\mathbf{n}(\mathbf{P}) \quad = \quad \alpha \cdot \mathbf{n}(\mathbf{A}) + \beta \cdot \mathbf{n}(\mathbf{B}) + \gamma \cdot \mathbf{n}(\mathbf{C})$$

University of Victoria | Computer Science

Color Interpolation

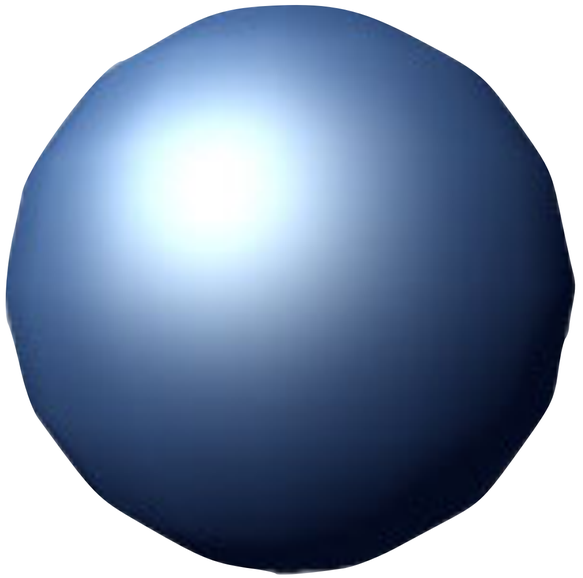


Per-vertex



Evaluate color on vertices, then interpolates it

Per-pixel



Interpolates positions and normals, then evaluate color on each pixel

http://www.cgchannel.com/2010/11/cg-science-for-artists-part-2-the-real-time-rendering-pipeline/

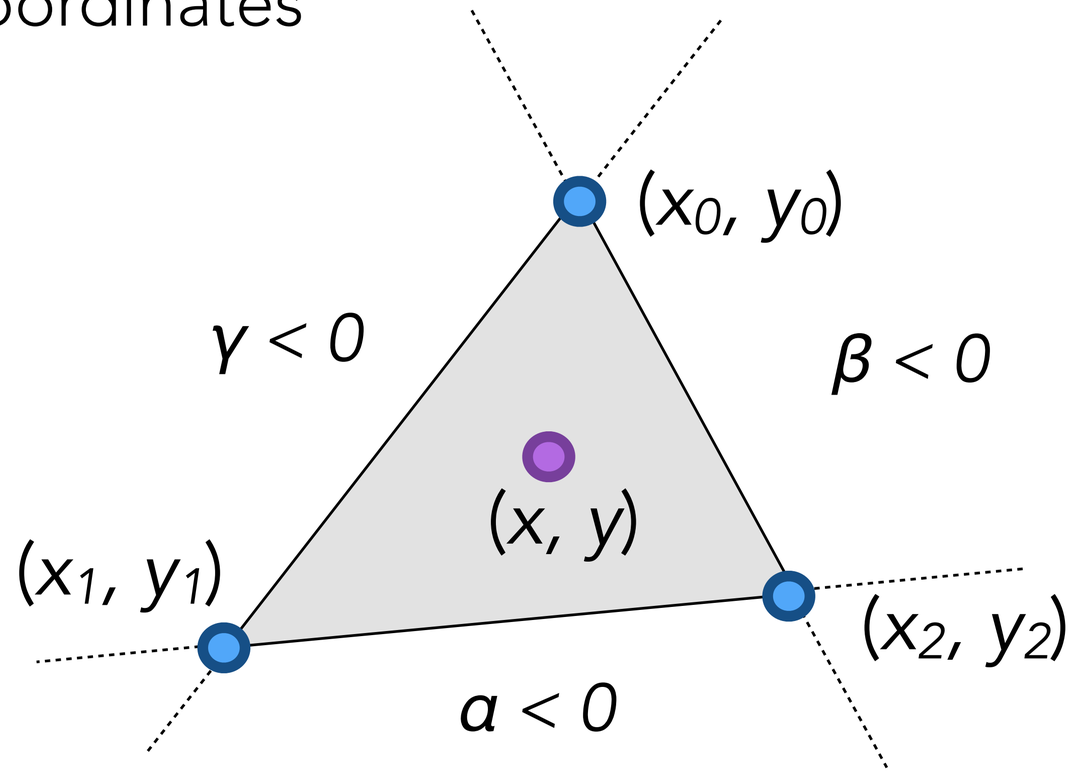
Triangle Rasterization

- Each triangle is represented as three 2D points (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

$$x = \alpha \cdot x_0 + \beta \cdot x_1 + \gamma \cdot x_2$$

$$y = \alpha \cdot y_0 + \beta \cdot y_1 + \gamma \cdot y_2$$

$$\alpha + \beta + \gamma = 1$$



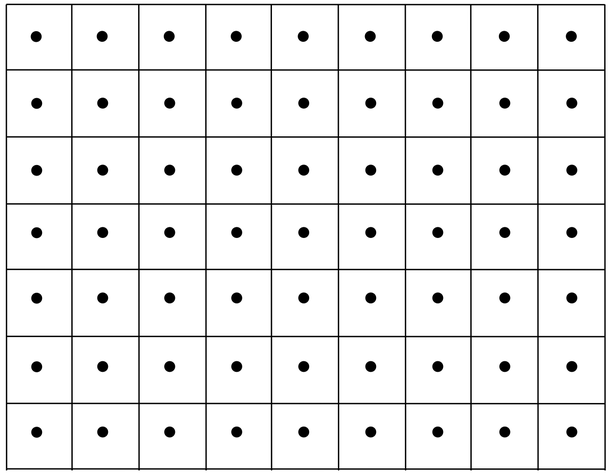
University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points

  (x_0, y_0), (x_1, y_1), (x_2, y_2)

- Rasterization using barycentric coordinates



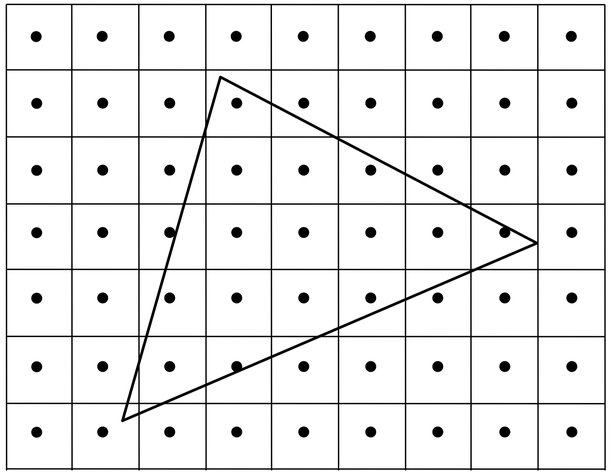
University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points

  (x_0, y_0), (x_1, y_1), (x_2, y_2)

- Rasterization using barycentric coordinates

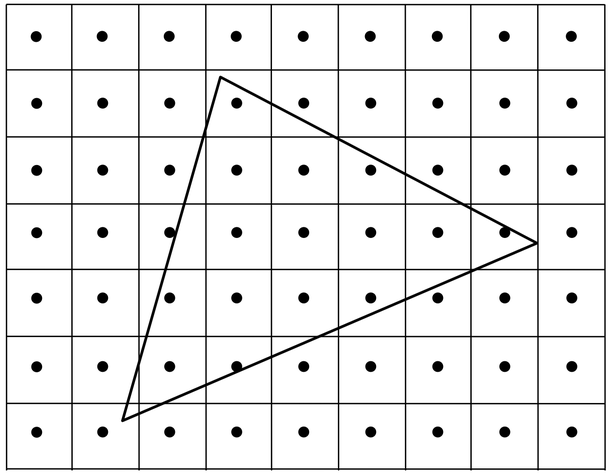


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

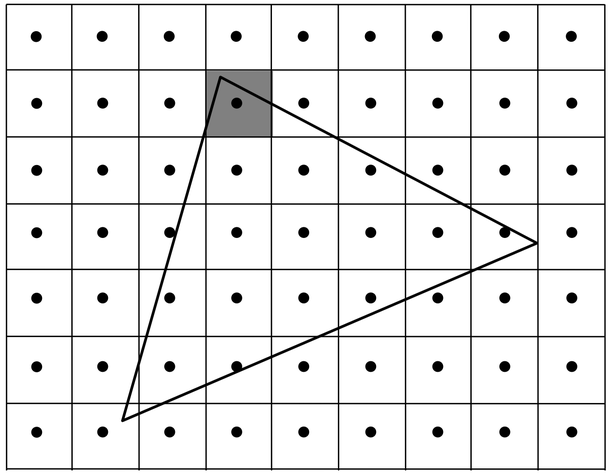


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

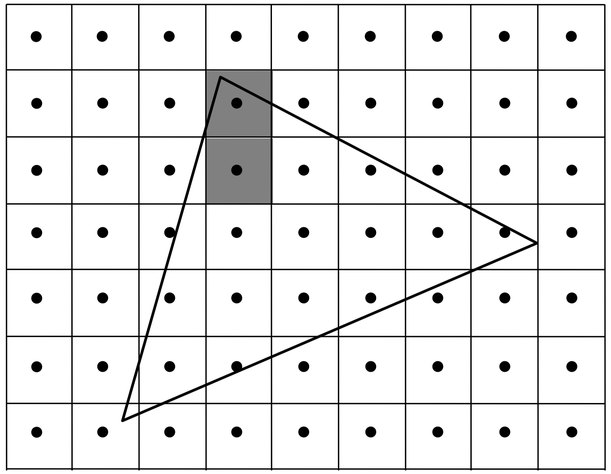


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

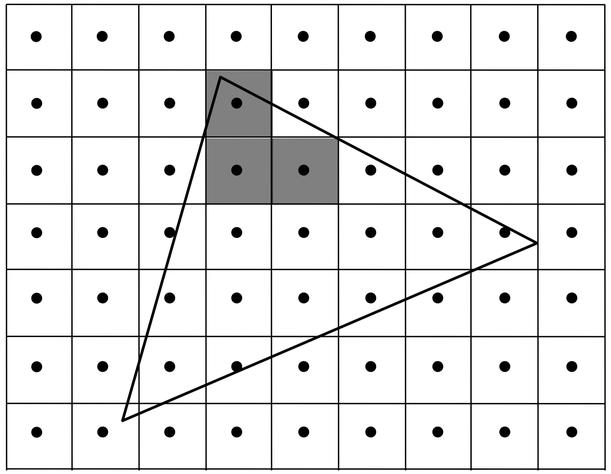


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

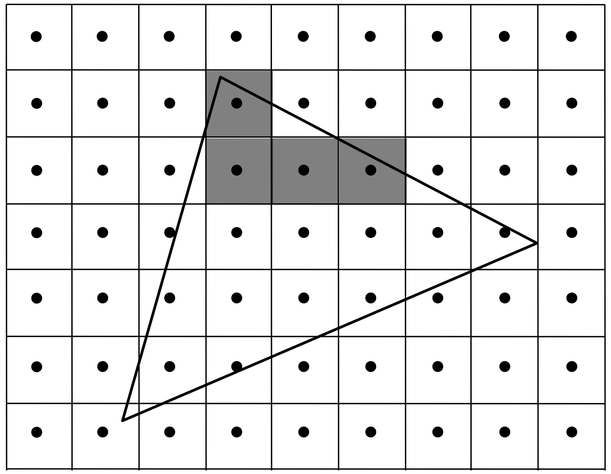


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

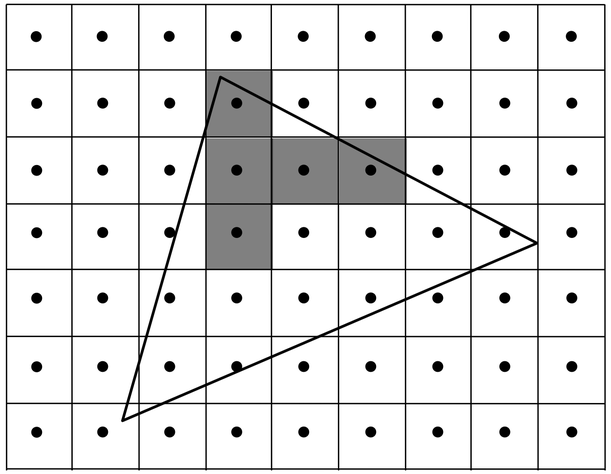


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

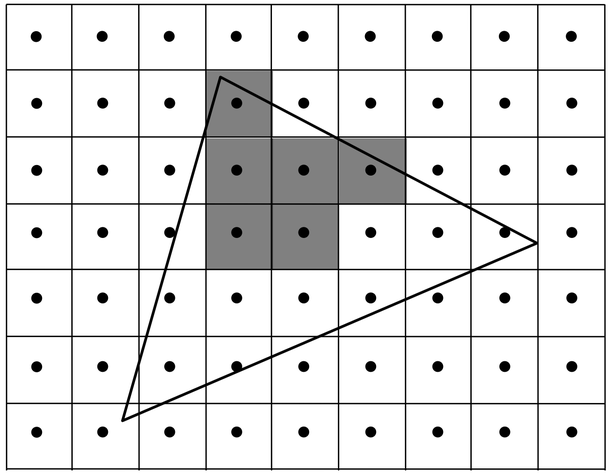


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

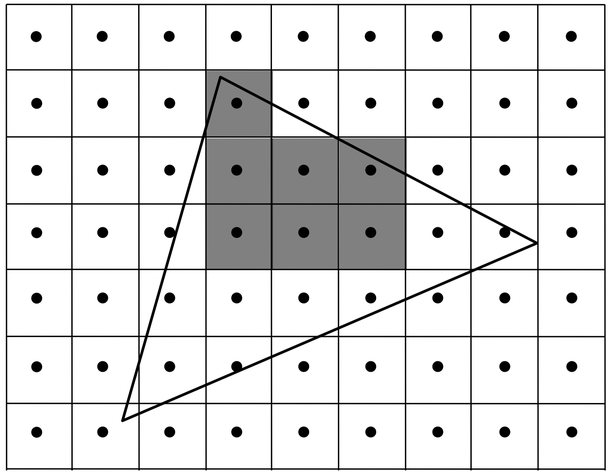


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

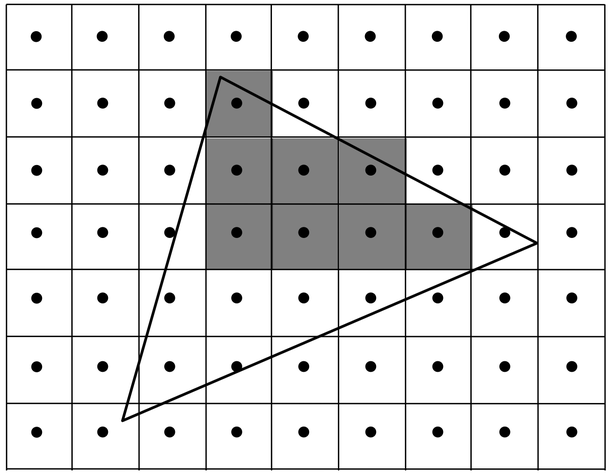


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

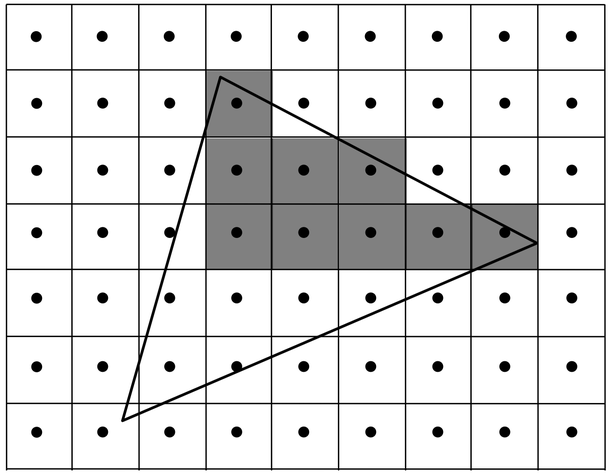


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

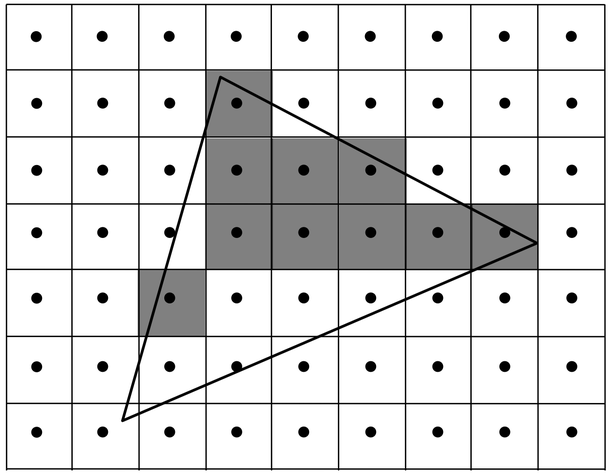


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

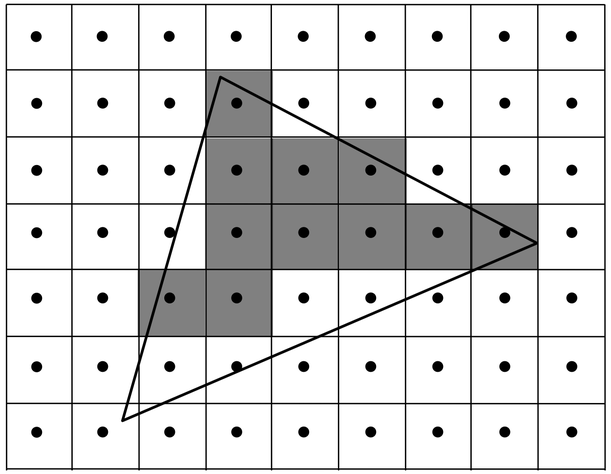


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

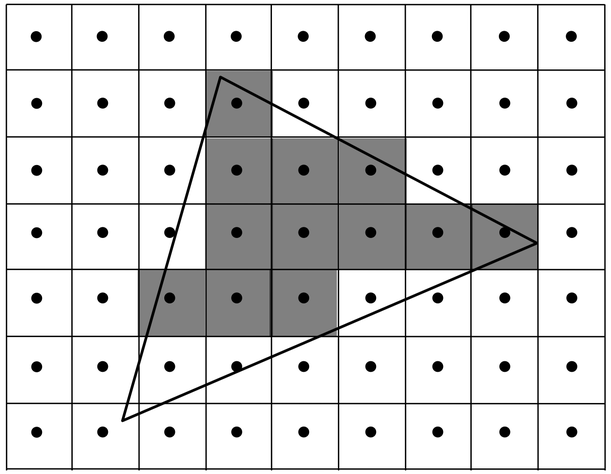


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

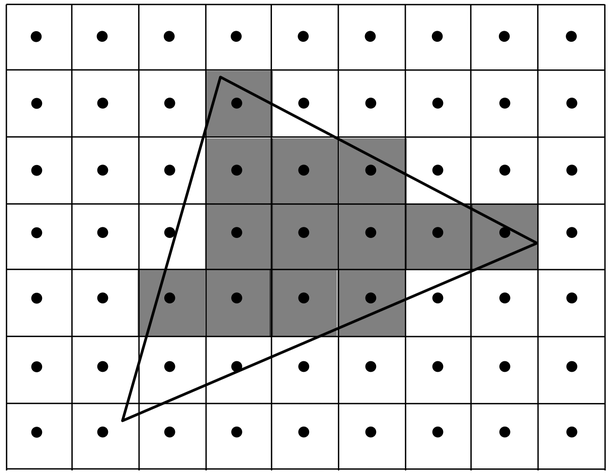


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

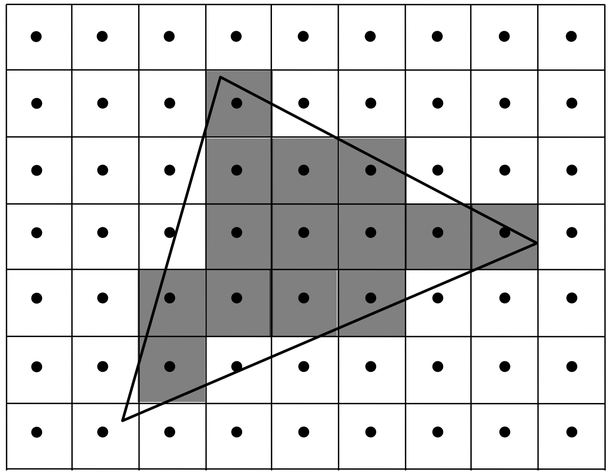


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

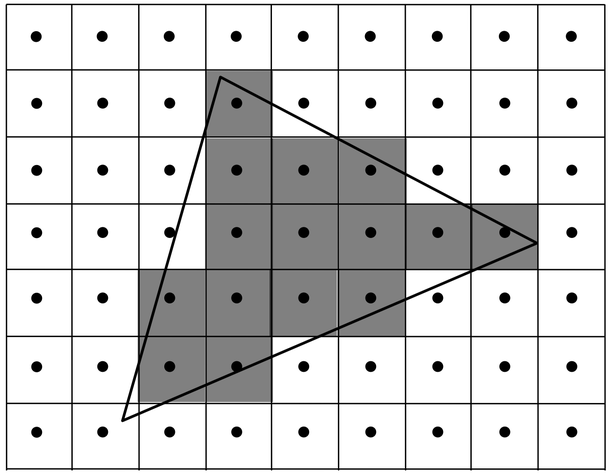


University of Victoria | Computer Science

Triangle Rasterization

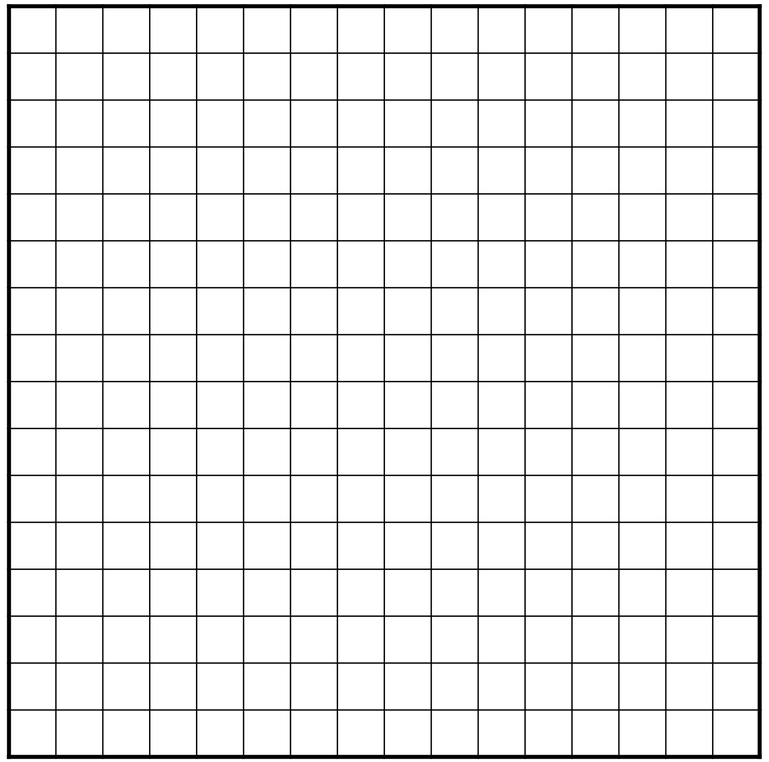
- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

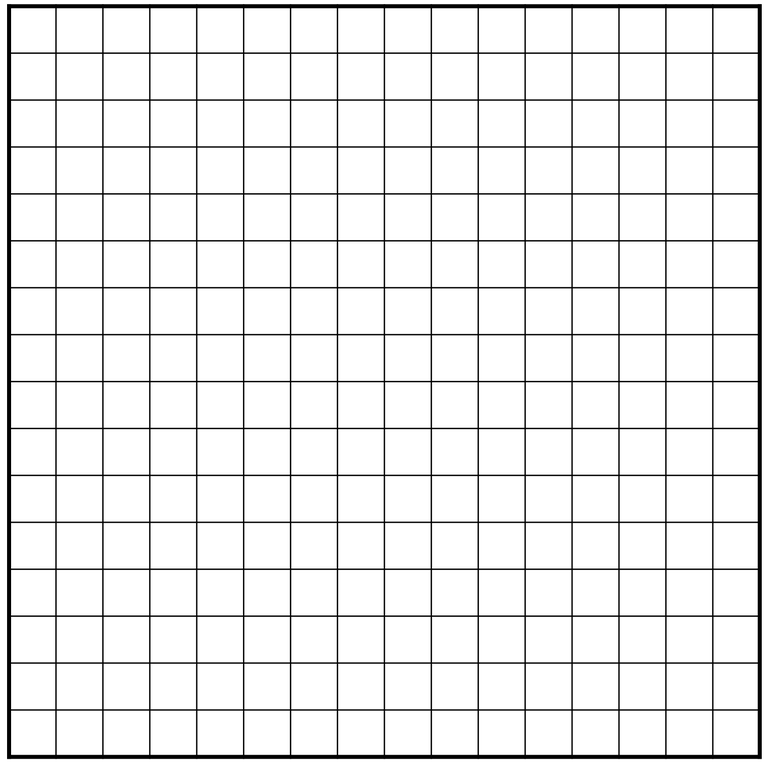
```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```



University of Victoria | Computer Science

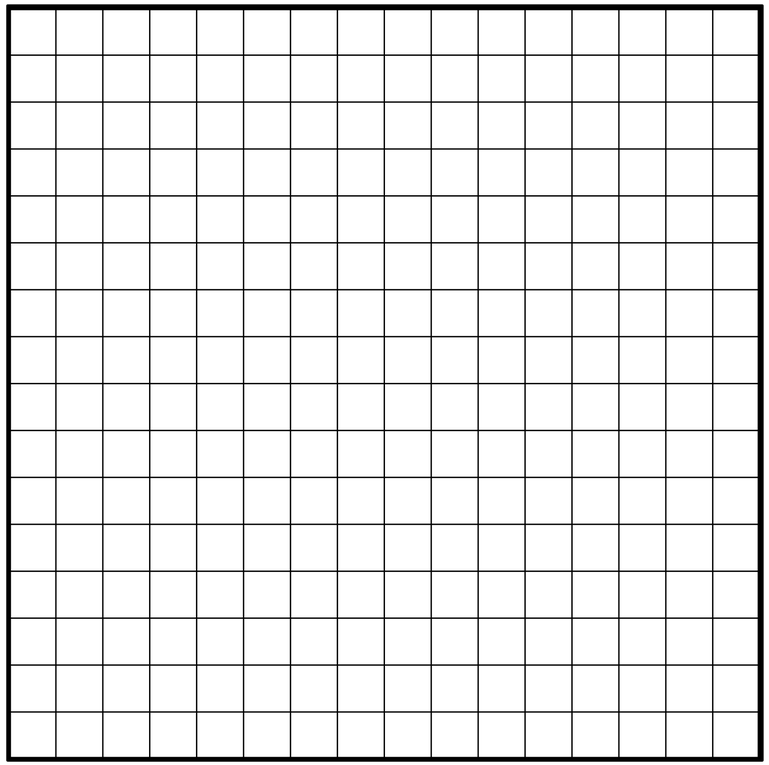
Clipping

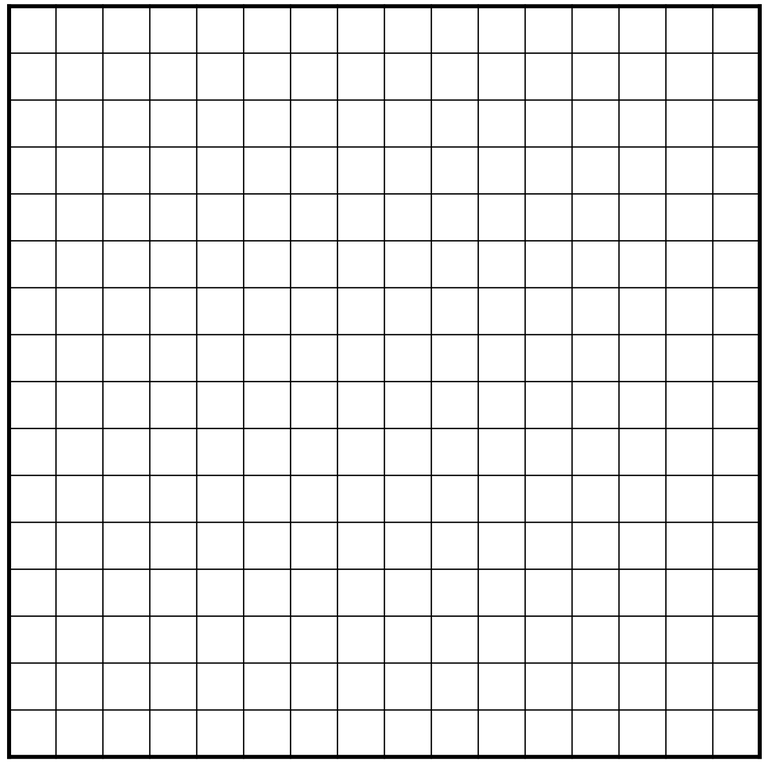




- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

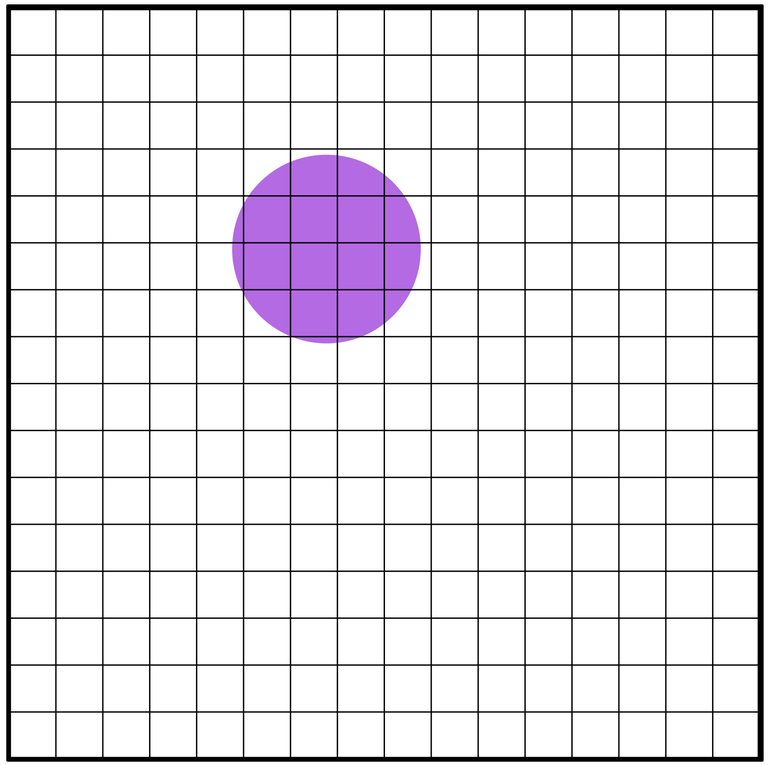
Clipping

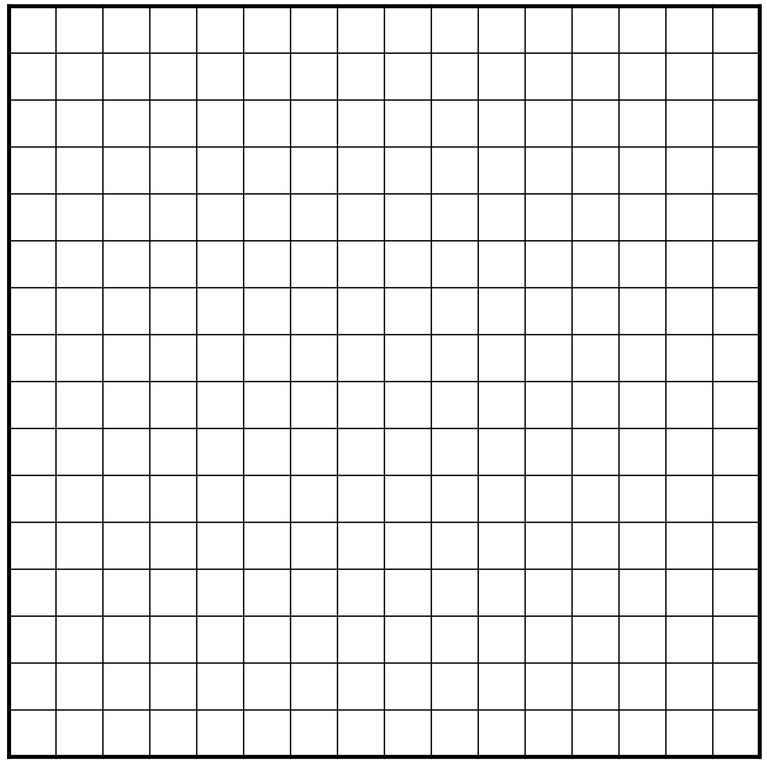




- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

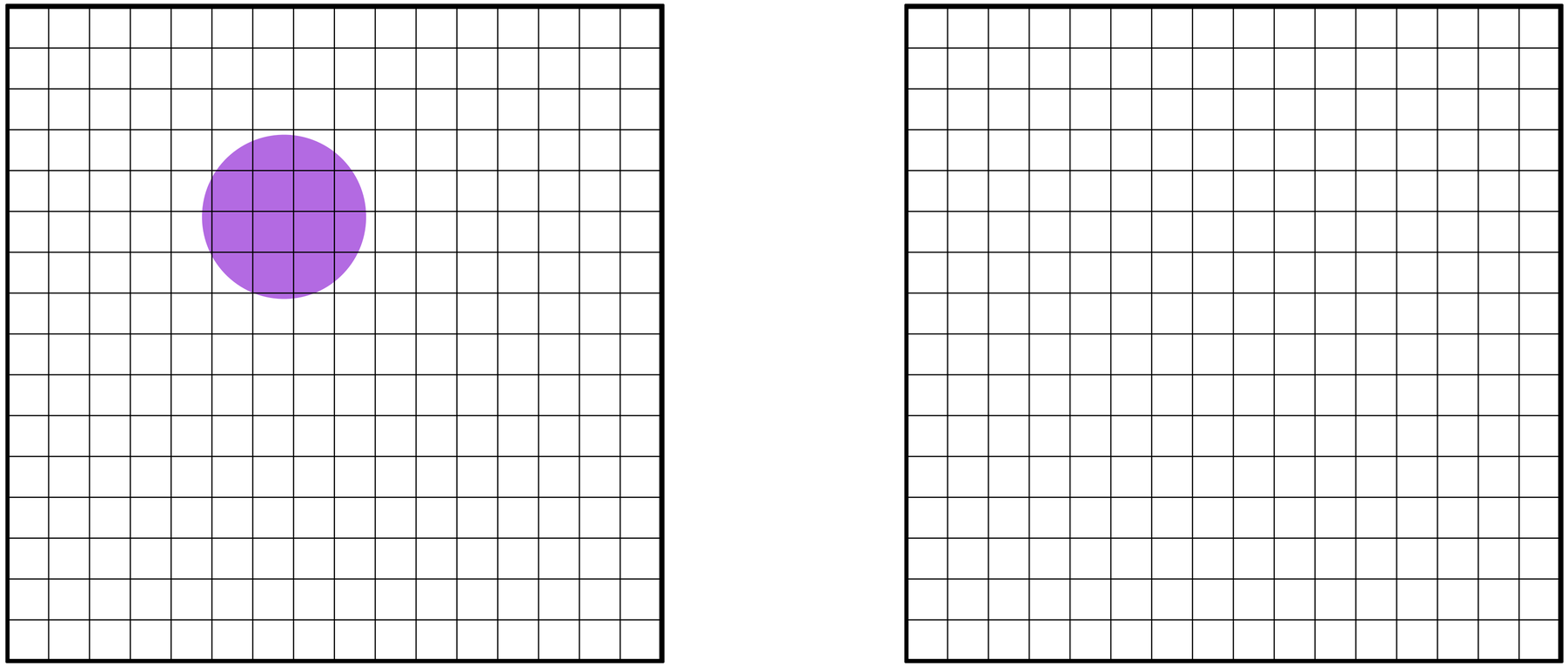
Clipping





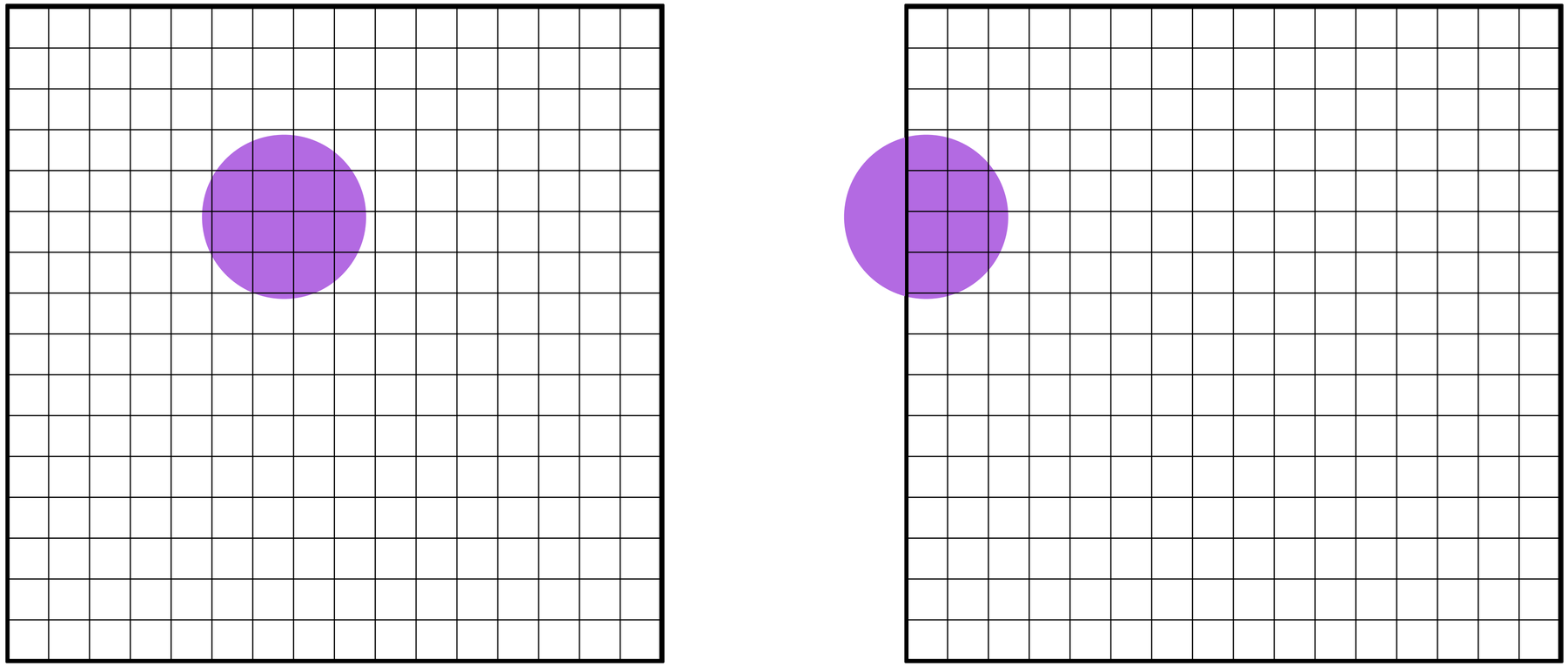
- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

Clipping



- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

Clipping

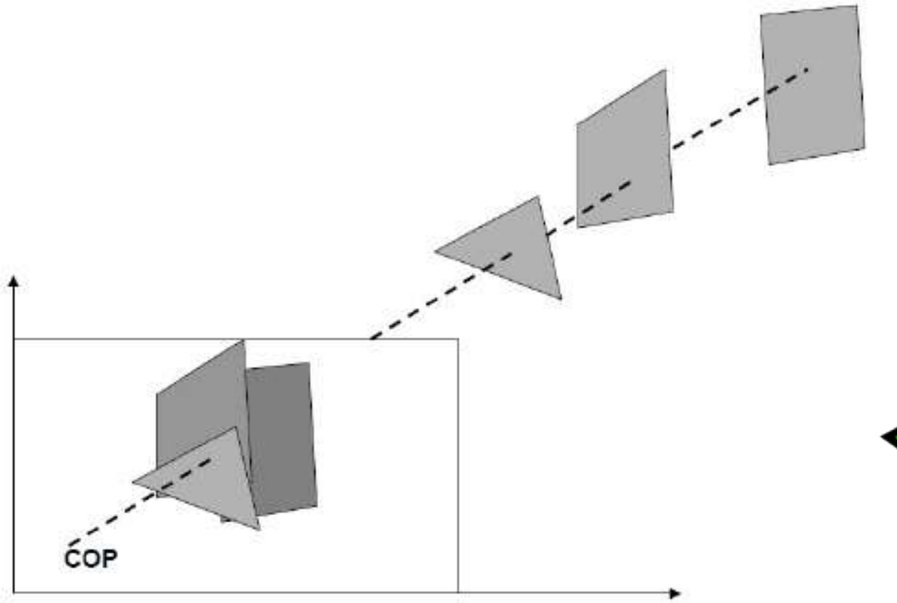


- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

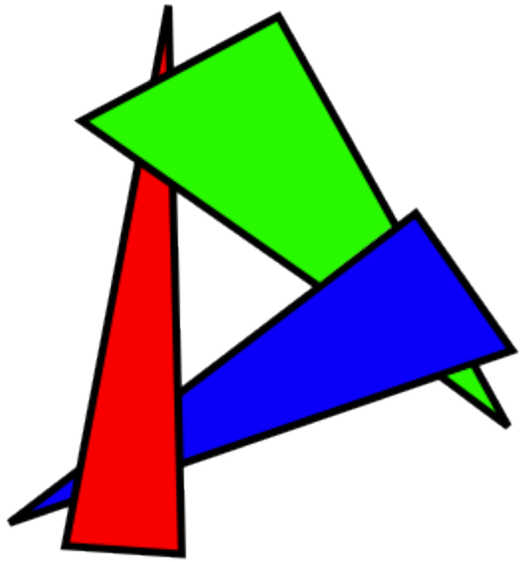
University of Victoria | Computer Science

Objects Depth Sorting

- To handle occlusion, you can sort all the objects in a scene by depth

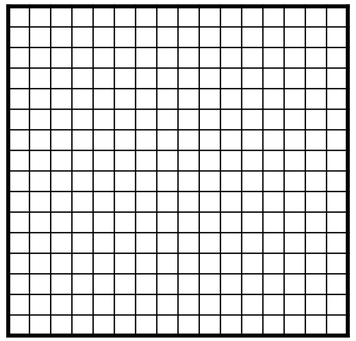


- This is not always possible!

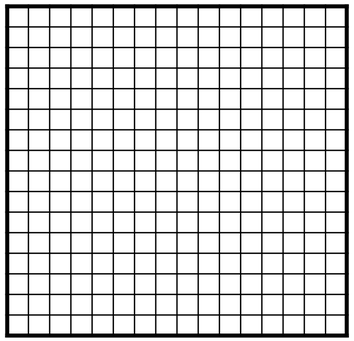


University of Victoria | Computer Science

z-buffering



Image



Depth (z)

- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

University of Victoria | Computer Science

z-buffering

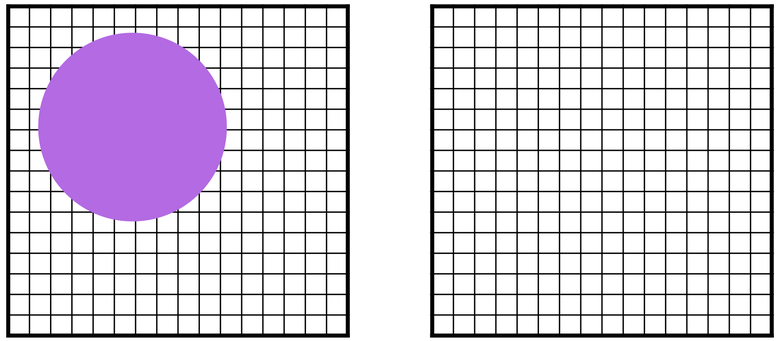


Image Depth (z)

- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

z-buffering

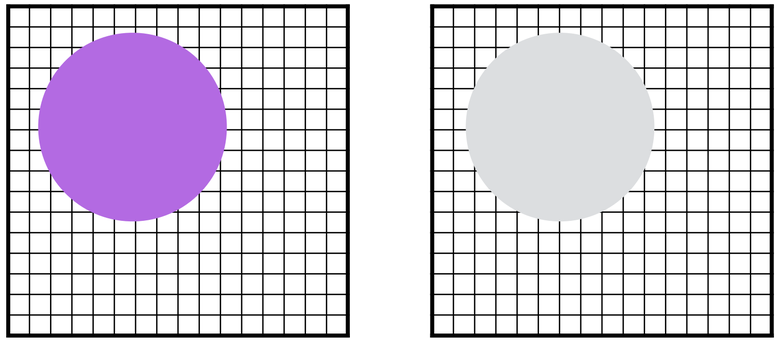
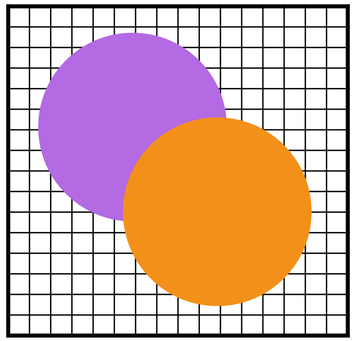


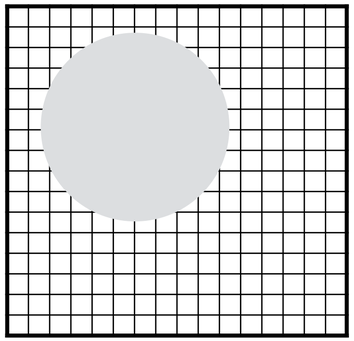
Image Depth (z)

- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

z-buffering



Image

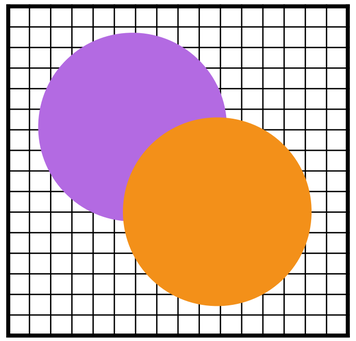


Depth (z)

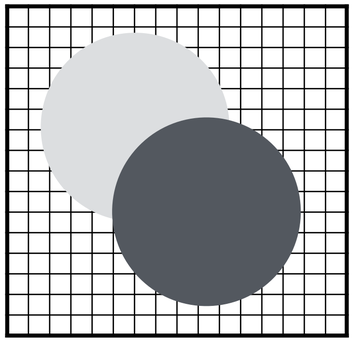
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

University of Victoria | Computer Science

z-buffering



Image



Depth (z)

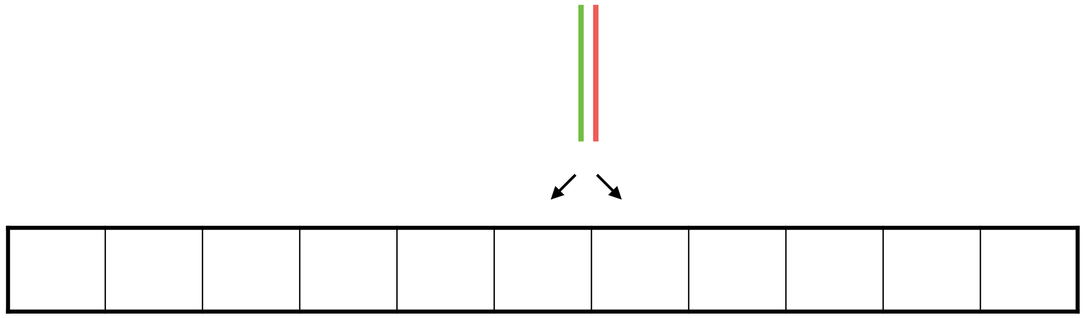
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

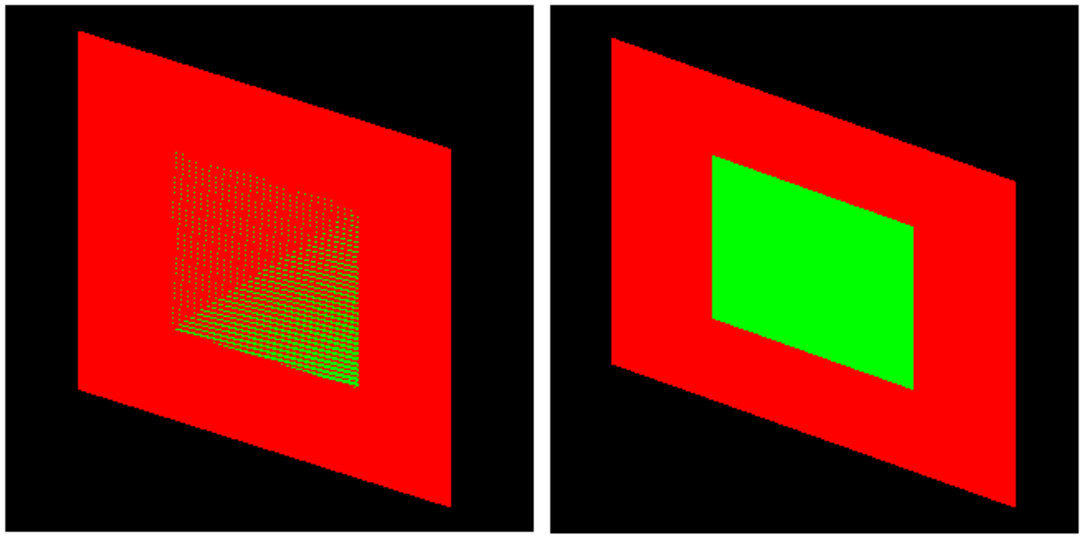
University of Victoria | Computer Science

z-buffer quantization and “z-fighting”

- The z-buffer is quantized (the number of bits is heavily dependent on the hardware platform)

- Two close object might be quantized differently, leading to strange artifacts, usually called “z-fighting”





University of Victoria | Computer Science

Super Sampling Anti-Aliasing





- Render nxn pixels instead of one
- Assign the average to the pixel

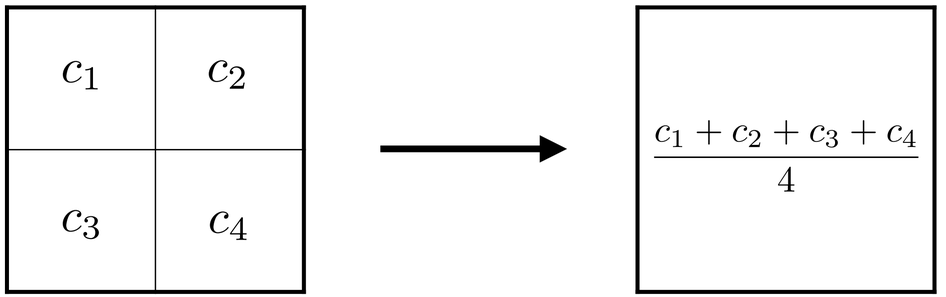
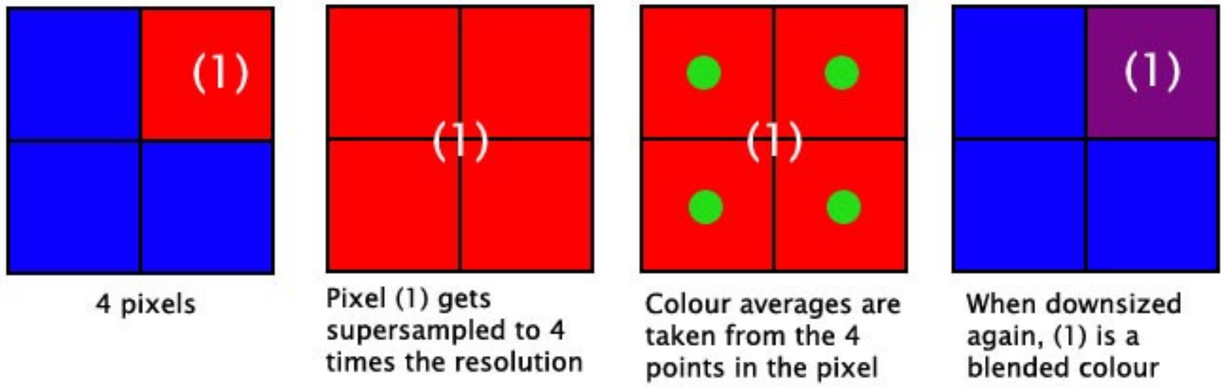


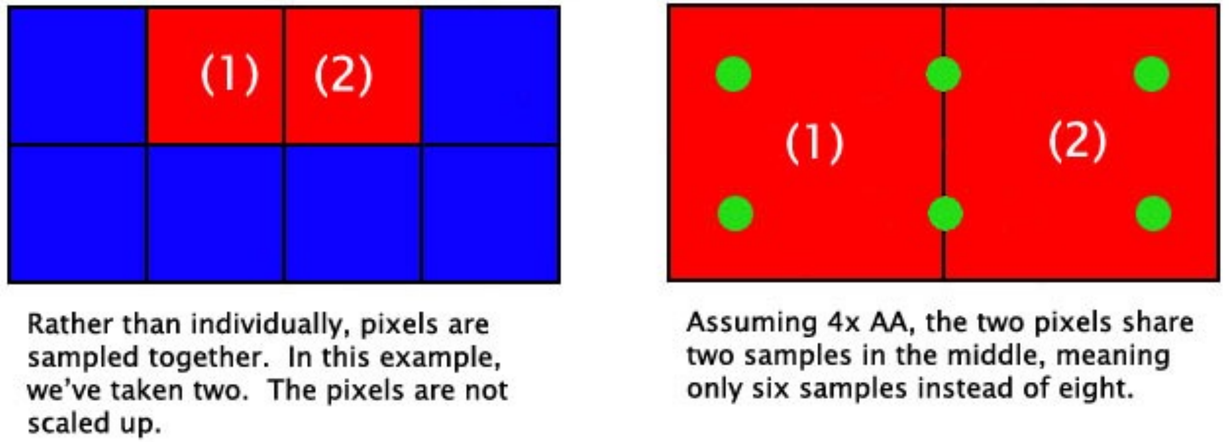
Image Copyright: Fritz Kessler

Many different names and variants

- SSAA (FSAA)
- MSAA
- CSAA
- EQAA
- FXAA
- TX AA



MSAA



Copyright: tested.com (http://www.tested.com/tech/pcs/1194-how-to-choose-the-right-anti-aliasing-mode-for-your-gpu/#)

References

Fundamentals of Computer Graphics, Fourth Edition
4th Edition **by Steve Marschner, Peter Shirley**

Chapter 8